

SOLIDWORKS API (.NET VBA C#) — Examples (43/50): Move and Copy Body using Vertex Example - Put Assembly Components in Drawing View

SOLIDWORKS API Documentation (converted from CHM)

Modify Move/Copy Body Using Vertex Example (VBA)

This example shows how to modify a move/copy body feature using a vertex.

```
'-----  
' Preconditions:  
' 1. Verify that the specified part document exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified part document.  
' 2. Selects a solid body and two vertices.  
' 3. Inserts a move/copy body feature and gets and sets some of its feature data.  
' 4. Examine the FeatureManager design tree, the graphics area, and  
'    the Immediate window.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swFeatureManager As SldWorks.FeatureManager  
Dim swFeature As SldWorks.Feature  
Dim swMoveCopyBodyFeatureData As SldWorks.MoveCopyBodyFeatureData  
Dim swVertexFrom As SldWorks.Vertex  
Dim swVertexTo As SldWorks.Vertex  
Dim swSelectionMgr As SldWorks.SelectionMgr  
Dim status As Boolean  
Dim errors As Long  
Dim warnings As Long  
Dim fileName As String  
  
Sub main()  
    Set swApp = Application.SldWorks  
  
    'Open part document  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent,  
    "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
    Set swFeatureManager = swModel.FeatureManager  
  
    'Select solid body and vertices for move/copy body feature  
    status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)  
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, 0, 0.065, True, 0, Nothing, 0)  
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, True, 0, Nothing, 0)  
    swModel.ClearSelection2 True  
    status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, False, 1, Nothing, 0)  
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, 0, 0.065, True, 4, Nothing, 0)  
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, True, 8, Nothing, 0)  
  
    'Insert move/copy body feature  
    Set swFeature = swFeatureManager.InsertMoveCopyBody2(0, 0, 0, 0, -0.085, 0, 0.065, 0, 0, 0, True, 1)  
  
    'Roll forward the feature and get and set move/copy body feature data  
    Set swMoveCopyBodyFeatureData = swFeature.GetDefinition  
    status = swMoveCopyBodyFeatureData.AccessSelections(swModel, Nothing)  
    Debug.Print "Move/copy body feature data:"  
    Debug.Print "  Number of bodies to move or rotate and copy: " & swMoveCopyBodyFeatureData.GetBodiesCount  
    Debug.Print "  Number of copies: " & swMoveCopyBodyFeatureData.NumberOfCopies  
    Debug.Print "  Transform type (0 = None, 1 = Translation, 2 = Rotation): " &  
    swMoveCopyBodyFeatureData.TransformType  
  
    'Change translation vertex  
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, True, 8, Nothing, 0)  
    Set swSelectionMgr = swModel.SelectionManager  
    Set swVertexFrom = swSelectionMgr.GetSelectedObject6(1, -1)  
    swMoveCopyBodyFeatureData.TransformReferenceEntity = swVertexFrom  
    swMoveCopyBodyFeatureData.TranslateToVertex = swVertexFrom
```

```
status = swModelDocExt.SelectByID2("", "VERTEX", 0, -0.065, 0, False, 0, Nothing, 0)
Set swVertexTo = swSelectionMgr.GetSelectedObject6(1, -1)
swMoveCopyBodyFeatureData.TranslateToVertex = swVertexTo
swMoveCopyBodyFeatureData.TransformX = 0.05

'Apply the changes and roll back the feature
swFeature.ModifyDefinition swMoveCopyBodyFeatureData, swModel, Nothing

swModel.ViewZoomtofit2
```

End Sub

Modify Move/Copy Body Using Vertex Example (VB.NET)

This example shows how to modify a move/copy body feature using a vertex.

```

'-----
' Preconditions:
' 1. Verify that the specified part document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects a solid body and two vertices.
' 3. Inserts a move/copy body feature and gets and sets some of its feature data.
' 4. Examine the FeatureManager design tree, the graphics area, and
'    the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim swMoveCopyBodyFeatureData As MoveCopyBodyFeatureData
        Dim swVertexFrom As Vertex
        Dim swVertexTo As Vertex
        Dim swSelectionMgr As SelectionMgr
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        'Open part document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\multibody\multi_inter.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "
", errors, warnings)
        swModelDocExt = swModel.Extension
        swFeatureManager = swModel.FeatureManager

        'Select solid body and vertices for move/copy body feature
        status = swModelDocExt.SelectByID("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("", "VERTEX", -0.085, 0, 0.065, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("", "VERTEX", -0.085, -0.09, 0.065, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID("", "VERTEX", -0.085, 0, 0.065, True, 4, Nothing, 0)
        status = swModelDocExt.SelectByID("", "VERTEX", -0.085, -0.09, 0.065, True, 8, Nothing, 0)

        'Insert move/copy body feature
        swFeature = swFeatureManager.InsertMoveCopyBody2(0, 0, 0, 0, -0.085, 0, 0.065, 0, 0, 0, True, 1)

        'Roll forward the feature and get and set move/copy body feature data
        swMoveCopyBodyFeatureData = swFeature.GetDefinition
        status = swMoveCopyBodyFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("Move/copy body feature data:")
        Debug.Print("  Number of bodies to move or rotate and copy: " & swMoveCopyBodyFeatureData.GetBodiesCount)
        Debug.Print("  Number of copies: " & swMoveCopyBodyFeatureData.NumberOfCopies)
        Debug.Print("  Transform type (0 = None, 1 = Translation, 2 = Rotation): " & swMoveCopyBodyFeatureData.TransformType)

        'Change translation vertex
        status = swModelDocExt.SelectByID("", "VERTEX", -0.085, -0.09, 0.065, True, 8, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swVertexFrom = swSelectionMgr.GetSelectedObject6(1, -1)
        swMoveCopyBodyFeatureData.TransformReferenceEntity = swVertexFrom
        swMoveCopyBodyFeatureData.TranslateToVertex = swVertexFrom
        status = swModelDocExt.SelectByID("", "VERTEX", 0, -0.065, 0, False, 0, Nothing, 0)
        swVertexTo = swSelectionMgr.GetSelectedObject6(1, -1)
        swMoveCopyBodyFeatureData.TranslateToVertex = swVertexTo
        swMoveCopyBodyFeatureData.TransformX = 0.05

        'Apply the changes and roll back the feature
    
```

```
swFeature.ModifyDefinition(swMoveCopyBodyFeatureData, swModel, Nothing)
```

```
swModel.ViewZoomtofit2()
```

```
End Sub
```

```
''' <summary>
```

```
''' The SldWorks swApp variable is pre-assigned for you.
```

```
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Move Annotations to Notes Area Annotation View Example (C#)

This example shows how to move all annotations to the **Notes Area** annotation view.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\button.sldprt.
// 2. Add a note to the *Top annotation view by double-clicking
//    *Top in the Annotations folder in the FeatureManager design
//    tree and clicking Insert > Annotations > Note. If prompted
//    to turn on feature dimensions display, click No.
// 3. Repeat step 2 to add a note to the *Front annotation view.
// 4. Double-click the Unassigned Items annotation view in the
//    Annotations folder in the FeatureManager design tree.
// 5. Open the Immediate window.
//
// Postconditions:
// 1. Examine the Immediate window.
// 2. Double-click each annotation view in the FeatureManager
//    design tree to verify that all annotations were moved
//    from *Front and *Top to Notes Area only. If prompted
//    to turn on feature dimensions display, click No.
//
// NOTE: Because the part is used elsewhere,
// do not save changes when closing the document.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace IAnnotationViewCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelExt = default(ModelDocExtension);
            object[] swAnnViews = null;
            object[] annotations = null;
            Annotation[] annToMove = new Annotation[2];
            AnnotationView swAnnView = default(AnnotationView);
            Annotation swAnn = default(Annotation);
            Feature swFeat = default(Feature);
            MathTransform swMathTransform = default(MathTransform);
            int i = 0;
            int j = 0;
            int k = 0;
            int l = 0;
            double[] planeArray = null;
            int nbrPlaneArray = 0;
            bool status = false;
            double[] transformArray = null;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelExt = (ModelDocExtension)swModel.Extension;

            //Get the annotation views, number of annotation views,
            //their names, and whether the annotation view is
            //hidden (i.e., not activated)
            swAnnViews = (object[])swModelExt.AnnotationViews;
            Debug.Print("Number of annotation views: " + swModelExt.AnnotationViewCount);
            for (i = 0; i <= swModelExt.AnnotationViewCount - 1; i++)
            {
                swAnnView = (AnnotationView)swAnnViews[i];
                swFeat = (Feature)swAnnView;
                Debug.Print(" " + swFeat.Name);
                status = swAnnView.Hide();
                //status dependent on whether the annotation view is activated
                //Only Unassigned Items is activated
                Debug.Print(" Hide: " + status);
                Debug.Print(" Flat-pattern view: " + swAnnView.FlatPatternView);
            }

            //Iterate through each annotation view and its annotations
            //Print each annotation name, its rotation matrix, whether
            //it is shown in the annotation view, and whether it is assigned
            //to a 3D view
            //Add all annotations to an array to move them
            k = 0;

```

```

l = 0;
Debug.Print("");
Debug.Print(" Name and number of annotations in annotation view: ");
for (i = 0; i <= swModelExt.AnnotationViewCount - 1; i++)
{
    swAnnView = (AnnotationView)swAnnViews[i];
    swAnnView.Activate();
    annotations = (object[])swAnnView.GetAnnotations2(false, false);
    swFeat = (Feature)swAnnView;
    Debug.Print("");
    Debug.Print(" " + swFeat.Name + " = " + swAnnView.AnnotationCount);
    if ((annotations != null))
    {
        for (j = 0; j <= swAnnView.AnnotationCount - 1; j++)
        {
            swAnn = (Annotation)annotations[j];
            if (k >= 0)
            {
                annToMove[k] = (Annotation)swAnn;
                k = k + 1;
            }
            planeArray = (double[])swAnn.GetPlane();
            nbrPlaneArray = planeArray.GetLength(0);
            Debug.Print(" Rotation matrix of the annotation relative to the X-
Y plane of the model: ");
            while (l < nbrPlaneArray)
            {
                Debug.Print(" " + planeArray[l] + " " + planeArray[l + 1] + " " + planeArray[l
+ 2]);
                l = l + 3;
            }
            l = 0;
            swMathTransform = (MathTransform)swAnn.GetFlipPlaneTransform();
            transformArray = (double[])swMathTransform.ArrayData;
            if (transformArray != null)
            {
                Debug.Print(" Rotation matrix if annotation plane flipped:");
                Debug.Print(" " + transformArray[0] + " " + transformArray[1] + " " + transform
Array[2]);
                Debug.Print(" " + transformArray[3] + " " + transformArray[4] + " " + transform
Array[5]);
                Debug.Print(" " + transformArray[6] + " " + transformArray[7] + " " + transform
Array[8]);
                Debug.Print(" Translation component if annotation plane flipped:");
                Debug.Print(" " + transformArray[9] + " " + transformArray[10] + " " + transform
Array[11]);
                Debug.Print("");
            }
            Debug.Print(" Annotation names:");
            Debug.Print(" " + swAnn.GetName());
            status = swAnnView.IsShown();
            Debug.Print(" Shown in this annotation view: " + status);
            status = swAnnView.UnassignedView;
            Debug.Print(" Assigned to a 3D View: " + status);
        }
    }
}

//Move all annotations to the Notes Area annotation view
Debug.Print("");
Debug.Print("Move all annotations to Notes Area annotation view:");
for (i = 0; i <= swModelExt.AnnotationViewCount - 1; i++)
{
    swAnnView = (AnnotationView)swAnnViews[i];
    swAnnView.Activate();
    swFeat = (Feature)swAnnView;
    if (swFeat.Name == "Notes Area")
    {
        status = swAnnView.MoveAnnotations(annToMove);
        Debug.Print(" Annotations moved: " + status);
        status = swAnnView.Show();
        //status should be false because annotation was activated
        Debug.Print(" Show: " + status);
    }
}
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;
}
}

```

Move Annotations to Notes Area Annotation View Example (VBA)

This example shows how to move all annotations to the **Notes Area** annotation view.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\button.sldprt.  
' 2. Add a note to the *Top annotation view by double-clicking  
'    *Top in the Annotations folder in the FeatureManager design  
'    tree and clicking Insert > Annotations > Note. If prompted  
'    to turn on feature dimensions display, click No.  
' 3. Repeat step 2 to add a note to the *Front annotation view.  
' 4. Double-click the Unassigned Items annotation view in the  
'    Annotations folder in the FeatureManager design tree.  
' 5. Open the Immediate window.  
'  
' Postconditions:  
' 1. Examine the Immediate window.  
' 2. Double-click each annotation view in the FeatureManager  
'    design tree to verify that all annotations were moved  
'    from *Front and *Top to Notes Area only. If prompted  
'    to turn on feature dimensions display, click No.  
'  
' NOTE: Because the part is used elsewhere,  
' do not save changes when closing the document.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelExt As SldWorks.ModelDocExtension  
Dim swAnnViews As Variant  
Dim annotations As Variant  
Dim annToMove(1) As SldWorks.Annotation  
Dim swAnnView As SldWorks.AnnotationView  
Dim swAnn As SldWorks.Annotation  
Dim swFeat As SldWorks.Feature  
Dim swMathTransform As SldWorks.MathTransform  
Dim i As Long  
Dim j As Long  
Dim k As Long  
Dim l As Long  
Dim planeArray() As Double  
Dim nbrPlaneArray As Long  
Dim status As Boolean  
Dim transformArray() As Double  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelExt = swModel.Extension  
  
    'Get the annotation views, number of annotation views,  
    'their names, and whether the annotation view is  
    'hidden (i.e., not activated)  
    swAnnViews = swModelExt.AnnotationViews  
    Debug.Print "Number of annotation views: " & swModelExt.AnnotationViewCount  
    Debug.Print "  Annotation view name: "  
    For i = 0 To swModelExt.AnnotationViewCount - 1  
        Set swAnnView = swAnnViews(i)  
        Set swFeat = swAnnView  
        Debug.Print "      " & swFeat.Name  
        status = swAnnView.Hide  
        'status dependent on whether the annotation view is activated  
        'Only Unassigned Items is activated  
        Debug.Print "      Hide: " & status  
        Debug.Print "      Flat-pattern view: " & swAnnView.FlatPatternView  
    Next
```

```

'Iterate through each annotation view and its annotations
'Print each annotation name, its rotation matrix, whether it is
'shown in the annotation view, and whether it is assigned to a
'3D view
'Add all annotations to an array to move them
k = 0
l = 0
Debug.Print ""
Debug.Print "   Name and number of annotations in annotation view: "
For i = 0 To swModelExt.AnnotationViewCount - 1
    Set swAnnView = swAnnViews(i)
    swAnnView.Activate
    annotations = swAnnView.GetAnnotations2(False, False)
    Set swFeat = swAnnView
    Debug.Print ""
    Debug.Print "           " & swFeat.Name & " = " & swAnnView.AnnotationCount
    If Not IsEmpty(annotations) Then
        For j = 0 To swAnnView.AnnotationCount - 1
            Set swAnn = annotations(j)
            planeArray = swAnn.GetPlane
            nbrPlaneArray = UBound(planeArray)
            Debug.Print "           Rotation matrix of the annotation relative to the X-Y plane of the model: "
            Do While l < nbrPlaneArray
                Debug.Print "           " & planeArray(l) & " " & planeArray(l + 1) & " " & planeArray(l + 2)
                l = l + 3
            Loop
            Set swMathTransform = swAnn.GetFlipPlaneTransform
            transformArray = swMathTransform.ArrayData()
            If Not IsEmpty(transformArray) Then
                Debug.Print "           Rotation matrix if annotation plane flipped:"
                Debug.Print "           " & transformArray(0) & " " & transformArray(1) & " " &
transformArray(2)
                Debug.Print "           " & transformArray(3) & " " & transformArray(4) & " " &
transformArray(5)
                Debug.Print "           " & transformArray(6) & " " & transformArray(7) & " " &
transformArray(8)
                Debug.Print "           Translation component if annotation plane flipped:"
                Debug.Print "           " & transformArray(9) & " " & transformArray(10) & " " &
transformArray(11)
                Debug.Print
            End If
            l = 0
            If k >= 0 Then
                Set annToMove(k) = swAnn
                k = k + 1
            End If
            Debug.Print "           Annotation names:"
            Debug.Print "           " & swAnn.GetName
            status = swAnnView.IsShown
            Debug.Print "           Shown in this annotation view: " & status
            status = swAnnView.UnassignedView
            Debug.Print "           Assigned to a 3D View: " & status
        Next
    End If
Next
Next

'Move all annotations to the Notes Area annotation view
Debug.Print ""
Debug.Print "Move all annotations to Notes Area annotation view:"
For i = 0 To swModelExt.AnnotationViewCount - 1
    Set swAnnView = swAnnViews(i)
    swAnnView.Activate
    Set swFeat = swAnnView
    If swFeat.Name = "Notes Area" Then
        status = swAnnView.MoveAnnotations(annToMove)
        Debug.Print "   Annotations moved: " & status
        status = swAnnView.Show
        'status should be false because annotation was activated
        Debug.Print "   Show: " & status
    End If
Next
End Sub

```

Move Annotations to Notes Area Annotation View Example (VB.NET)

This example shows how to move all annotations to the **Notes Area** annotation view.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\button.sldprt.
' 2. Add a note to the *Top annotation view by double-clicking
'    *Top in the Annotations folder in the FeatureManager design
'    tree and clicking Insert > Annotations > Note. If prompted
'    to turn on feature dimensions display, click No.
' 3. Repeat step 2 to add a note to the *Front annotation view.
' 4. Double-click the Unassigned Items annotation view in the
'    Annotations folder in the FeatureManager design tree.
' 5. Open the Immediate window.
'
' Postconditions:
' 1. Examine the Immediate window.
' 2. Double-click each annotation view in the FeatureManager
'    design tree to verify that all annotations were moved
'    from *Front and *Top to Notes Area only. If prompted
'    to turn on feature dimensions display, click No.
'
' NOTE: Because the part is used elsewhere, do not
'       save changes when closing it.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelExt As ModelDocExtension
        Dim swAnnViews As Object
        Dim annotations As Object
        Dim annToMove(1) As Annotation
        Dim swAnnView As AnnotationView
        Dim swAnn As Annotation
        Dim swFeat As Feature
        Dim swMathTransform As MathTransform
        Dim i As Integer
        Dim j As Integer
        Dim k As Integer
        Dim l As Integer
        Dim planeArray() As Double
        Dim nbrPlaneArray As Integer
        Dim status As Boolean
        Dim transformArray() As Double

        swModel = swApp.ActiveDoc
        swModelExt = swModel.Extension

        'Get the annotation views, number of annotation views,
        'their names, and whether the annotation views
        'is hidden (i.e., not activated)
        swAnnViews = swModelExt.AnnotationViews
        Debug.Print("Number of annotation views: " & swModelExt.AnnotationViewCount)
        For i = 0 To swModelExt.AnnotationViewCount - 1
            swAnnView = swAnnViews(i)
            swFeat = swAnnView.Feature
            Debug.Print(" " & swFeat.Name)
            status = swAnnView.Hide
            'status dependent on whether the annotation view is activated
            'Only Unassigned Items is activated
            Debug.Print(" " & status)
            Debug.Print(" " & swAnnView.FlatPatternView)
        Next

        'Iterate through each annotation view and its annotations
        'Print each annotation name, its rotation matrix, whether it
        'is shown in the annotation view, and whether it is assigned
        'to a 3D view
        'Add all annotations to an array to move them
        k = 0
        l = 0
        Debug.Print("")
        Debug.Print(" Name and number of annotations in annotation view: ")
    
```

```

For i = 0 To swModelExt.AnnotationViewCount - 1
    swAnnView = swAnnViews(i)
    swAnnView.Activate()
    annotations = swAnnView.GetAnnotations2(False, False)
    swFeat = swAnnView
    Debug.Print("")
    Debug.Print(" " & swFeat.Name & " = " & swAnnView.AnnotationCount)
    If Not annotations Is Nothing Then
        For j = 0 To swAnnView.AnnotationCount - 1
            swAnn = annotations(j)
            If k >= 0 Then
                annToMove(k) = swAnn
                k = k + 1
            End If
            planeArray = swAnn.GetPlane
            Debug.Print(" " & swAnn.Name & " Rotation matrix of the annotation relative to the X-
Y plane of the model: ")
            nbrPlaneArray = UBound(planeArray)
            Do While l < nbrPlaneArray
                Debug.Print(" " & planeArray(l) & " " & planeArray(l + 1) & " " & planeArray(l + 2)
            )
                l = l + 3
            Loop

            swMathTransform = swAnn.GetFlipPlaneTransform
            transformArray = swMathTransform.ArrayData()
            If Not transformArray Is Nothing Then
                Debug.Print(" " & transformArray(0) & " " & transformArray(1) & " " & transformArra
                Debug.Print(" " & transformArray(3) & " " & transformArray(4) & " " & transformArra
                Debug.Print(" " & transformArray(6) & " " & transformArray(7) & " " & transformArra
                Debug.Print(" " & transformArray(9) & " " & transformArray(10) & " " & transformArra
            End If

            l = 0
            Debug.Print(" " & swAnn.Name & " Annotation names:")
            Debug.Print(" " & swAnn.GetName)
            status = swAnnView.IsShown
            Debug.Print(" " & status & " Shown in this annotation view: ")
            status = swAnnView.UnassignedView
            Debug.Print(" " & status & " Assigned to a 3D View: ")
        Next
    End If
Next

'Move all annotations to the Notes Area annotation view
Debug.Print("")
Debug.Print("Move all annotations to Notes Area annotation view:")
For i = 0 To swModelExt.AnnotationViewCount - 1
    swAnnView = swAnnViews(i)
    swAnnView.Activate()
    swFeat = swAnnView
    If swFeat.Name = "Notes Area" Then
        status = swAnnView.MoveAnnotations(annToMove)
        Debug.Print(" " & status & " Annotations moved: ")
        status = swAnnView.Show()
        'status should be false because annotation was activated
        Debug.Print(" " & status & " Show: ")
    End If
Next

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Move Assembly Components to New Folder Example (C#)

This example shows how to move selected assembly components to a newly created folder in the FeatureManager design tree.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly to open exists.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly document.
// 2. Selects the valve<1> and valve_guide<1> components.
// 3. Creates a folder named Folder1 in the FeatureManager design tree.
// 4. Moves the valve<1> and valve_guide<1> components to Folder1,
//    which you can verify by expanding Folder1.
// 5. Examine the Immediate window.
//
// NOTE: Because the assembly document is used by elsewhere,
// do not save any changes.
//-----

using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace FeatureFolderLocation_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 modelDoc2;
        AssemblyDoc assemblyDoc;
        FeatureManager featureMgr;
        ModelDocExtension modelDocExt;
        SelectionMgr selectionMgr;
        Feature feature;
        object selObj;
        Feature feat;
        Feature folderFeat;
        int errors;
        int warnings;
        int count;
        Component2 componentToMove;
        object[] componentsToMove;
        int i;
        bool retVal;

        public void Main()
        {
            //Open assembly document
            swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\motionstudies\\valve_cam.sldasm", (int)swDocumentTypes_e.swDocASSEMBLY,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            modelDoc2 = (ModelDoc2)swApp.ActiveDoc;
            assemblyDoc = (AssemblyDoc)modelDoc2;

            //Select and get the two valve-related components to move to the new folder
            modelDocExt = modelDoc2.Extension;
            selectionMgr = (SelectionMgr)modelDoc2.SelectionManager;
            modelDocExt.SelectByID2("valve-1@valve_cam", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            selObj = selectionMgr.GetSelectedObject6(1, -1);
            modelDocExt.SelectByID2("valve_guide-1@valve_cam", "COMPONENT", 0, 0, 0, true, 0, null,
0);
        }
    }
}
```

```

selectionMgr.GetSelectedObject6(2, -1);
count = selectionMgr.GetSelectedObjectCount2(0);
componentsToMove = new object[count];
for (i = 0; i <= count - 1; i++)
{
    componentToMove = (Component2)selectionMgr.GetSelectedObjectsComponent4(i + 1, 0);
    componentsToMove[i] = componentToMove;
}

//Create the folder where to move the selected components
featureMgr = modelDoc2.FeatureManager;
feature =
featureMgr.InsertFeatureTreeFolder2((int)swFeatureTreeFolderType_e.swFeatureTreeFolder_EmptyBefore);
feature = (Feature)assemblyDoc.FeatureByName("Folder1");

//Move the selected components to the new folder
retVal = assemblyDoc.ReorderComponents(componentsToMove, feature,
(int)swReorderComponentsWhere_e.swReorderComponents_LastInFolder);

modelDocExt.SelectByID2("valve-1@valve_cam", "COMPONENT", 0, 0, 0, true, 0, null, 0);
feat = (Feature)selectionMgr.GetSelectedObject6(1, -1);

featureMgr = modelDoc2.FeatureManager;
folderFeat = featureMgr.FeatureFolderLocation(feat);

Debug.Print("Component valve-1@valve_cam folder feature: " + folderFeat.Name);

}

public SldWorks swApp;

}

}

```

Move Assembly Components to New Folder Example (VBA)

This example shows how to move selected assembly components to a newly created folder in the FeatureManager design tree.

```
' -----  
' Preconditions:  
' 1. Verify that the specified assembly to open exists.  
' 2. Open an Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified assembly document.  
' 2. Selects the valve<1> and valve_guide<1> components.  
' 3. Creates a folder named Folder1 in the FeatureManager design tree.  
' 4. Moves the valve<1> and valve_guide<1> components to Folder1,  
'    which you can verify by expanding Folder1.  
' 5. Examine the Immediate window.  
'  
' NOTE: Because the assembly document is used by elsewhere,  
' do not save any changes.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim modelDoc2 As SldWorks.modelDoc2  
Dim assemblyDoc As SldWorks.assemblyDoc  
Dim featureMgr As SldWorks.FeatureManager  
Dim modelDocExt As SldWorks.ModelDocExtension  
Dim selectionMgr As SldWorks.selectionMgr  
Dim feature As SldWorks.feature  
Dim selObj As Object  
Dim feat As SldWorks.feature  
Dim folderFeat As SldWorks.feature  
Dim errors As Long  
Dim warnings As Long  
Dim status As Long  
Dim count As Long  
Dim componentToMove As SldWorks.Component2  
Dim componentsToMove() As Object  
Dim i As Long  
Dim retVal As Boolean  
Sub Main()  
  
    Set swApp = Application.SldWorks  
    'Open assembly document  
    swApp.OpenDoc6 "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\motionstudies\valve_cam.sldasm", swDocASSEMBLY, swOpenDocOptions_Silent, "",  
errors, warnings  
    Set modelDoc2 = swApp.ActiveDoc  
    Set assemblyDoc = modelDoc2  
  
    'Select and get the two valve-related components to move to the new folder  
    Set modelDocExt = modelDoc2.Extension  
    Set selectionMgr = modelDoc2.SelectionManager  
    status = modelDocExt.SelectByID2("valve-1@valve_cam", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)  
    Set selObj = selectionMgr.GetSelectedObject6(1, -1)  
    status = modelDocExt.SelectByID2("valve_guide-1@valve_cam", "COMPONENT", 0, 0, 0, True, 0,  
Nothing, 0)  
    Set selObj = selectionMgr.GetSelectedObject6(2, -1)  
    count = selectionMgr.GetSelectedObjectCount2(0)  
    ReDim componentsToMove(count - 1)  
    For i = 0 To count - 1  
        Set componentToMove = selectionMgr.GetSelectedObjectsComponent4(i + 1, 0)  
        Set componentsToMove(i) = componentToMove  
    Next  
End Sub
```

```
'Create the folder where to move the selected components
Set featureMgr = modelDoc2.FeatureManager
Set feature = featureMgr.InsertFeatureTreeFolder2(swFeatureTreeFolder_EmptyBefore)
Set feature = assemblyDoc.FeatureByName("Folder1")

'Move the selected components to the new folder
retVal = assemblyDoc.ReorderComponents(componentsToMove, feature,
swReorderComponents_LastInFolder)

status = modelDocExt.SelectByID2("valve-1@valve_cam", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
Set feat = selectionMgr.GetSelectedObject6(1, -1)

Set featureMgr = modelDoc2.FeatureManager
Set folderFeat = featureMgr.FeatureFolderLocation(feat)

Debug.Print "Component valve-1@valve_cam folder feature: " & folderFeat.Name
End Sub
```

Move Assembly Components to New Folder Example (VB.NET)

This example shows how to move selected assembly components to a newly created folder in the FeatureManager design tree.

```
'-----  
' Preconditions:  
' 1. Verify that the specified assembly to open exists.  
' 2. Open an Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified assembly document.  
' 2. Selects the valve<1> and valve_guide<1> components.  
' 3. Creates a folder named Folder1 in the FeatureManager design tree.  
' 4. Moves the valve<1> and valve_guide<1> components to Folder1,  
'    which you can verify by expanding Folder1.  
' 5. Examine the Immediate window.  
'  
' NOTE: Because the assembly document is used by elsewhere,  
' do not save any changes.  
'-----
```

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Imports System.Runtime.InteropServices

Imports System

Imports System.Diagnostics

Partial Class SolidWorksMacro

```
  
    Dim modelDoc2 As ModelDoc2  
  
    Dim assemblyDoc As AssemblyDoc  
  
    Dim featureMgr As FeatureManager  
  
    Dim modelDocExt As ModelDocExtension  
  
    Dim selectionMgr As SelectionMgr  
  
    Dim feature As Feature  
  
    Dim selObj As Object  
  
    Dim feat As Feature  
  
    Dim folderFeat As Feature  
  
    Dim errors As Integer  
  
    Dim warnings As Integer  
  
    Dim status As Integer  
  
    Dim count As Integer  
  
    Dim componentToMove As Component2  
  
    Dim componentsToMove() As Object  
  
    Dim i As Integer  
  
    Dim retVal As Boolean  
  
    Sub Main()  

```

```

'Open assembly document

swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\motionstudies\valve_cam.sldasm", swDocumentTypes_e.swDocASSEMBLY,
swOpenDocOptions_e.swOpenDocOptions_Silent, "",
errors, warnings)

modelDoc2 = swApp.ActiveDoc

assemblyDoc = modelDoc2

'Select and get the two
valve-related components to move to the new folder

modelDocExt = modelDoc2.Extension

selectionMgr = modelDoc2.SelectionManager

status = modelDocExt.SelectByID2("valve-1@valve_cam", "COMPONENT",
0, 0, 0, True,
0, Nothing,
0)

selObj = selectionMgr.GetSelectedObject6(1, -1)

status = modelDocExt.SelectByID2("valve_guide-1@valve_cam", "COMPONENT",
0, 0, 0, True,
0, Nothing,
0)

selObj = selectionMgr.GetSelectedObject6(2, -1)

count = selectionMgr.GetSelectedObjectCount2(0)

ReDim componentsToMove(count - 1)

For i = 0 To count - 1

    componentToMove = selectionMgr.GetSelectedObjectsComponent4(i
+ 1, 0)

    componentsToMove(i) = componentToMove

Next

'Create
the folder where to move the selected components

featureMgr = modelDoc2.FeatureManager

feature =
featureMgr.InsertFeatureTreeFolder2(swFeatureTreeFolderType_e.swFeatureTreeFolder_EmptyBefore)

feature = assemblyDoc.FeatureByName("Folder1")

'Move the selected components to
the new folder

retVal = assemblyDoc.ReorderComponents(componentsToMove,
feature, swReorderComponentsWhere_e.swReorderComponents_LastInFolder)

status = modelDocExt.SelectByID2("valve-1@valve_cam", "COMPONENT",
0, 0, 0, True,
0, Nothing,
0)

feat = selectionMgr.GetSelectedObject6(1, -1)

featureMgr = modelDoc2.FeatureManager

folderFeat = featureMgr.FeatureFolderLocation(feat)

```

```
Debug.Print("Component valve-1@valve_cam folder feature: " & folderFeat.Name)
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Move Bodies Example (C#)

This example shows how to move all of the bodies in a part document.

```
//-----  
// Preconditions:  
// 1. Specified part document to open exists.  
// 2. Run the macro.  
//  
// Postconditions: All of the bodies in the part document  
// are moved to the specified location.  
//  
// NOTE: Because this part is used elsewhere, do not save  
// any changes when closing it.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
  
namespace MoveBodiesCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
  
        public void SelectBodies(SldWorks swApp, ModelDoc2 swModel, object[] bodyArr)  
        {  
            // Select and mark the bodies to move  
            SelectionMgr swSelMgr = default(SelectionMgr);  
            SelectData swSelData = default(SelectData);  
            Body2 swBody = default(Body2);  
            bool status = false;  
            int i = 0;  
  
            swSelMgr = (SelectionMgr)swModel.SelectionManager;  
            swSelData = (SelectData)swSelMgr.CreateSelectData();  
  
            if ((bodyArr == null))  
                return;  
            for (i = 0; i <= bodyArr.GetUpperBound(0); i++)  
            {  
  
                swBody = (Body2)bodyArr[i];  
                swSelData.Mark = 1;  
                status = swBody.Select2(true, swSelData);  
            }  
        }  
  
        public void Main()  
        {  
  
            ModelDoc2 swModel = default(ModelDoc2);  
            PartDoc swPart = default(PartDoc);  
            object[] bodyArr = null;  
            FeatureManager swFeatMgr = default(FeatureManager);  
            Feature swFeat = default(Feature);  
            string fileName = null;  
            int errors = 0;  
            int warnings = 0;  
  
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\multibody\\multi_inter.sldprt";  
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);  
            swPart = (PartDoc)swModel;  
            swFeatMgr = (FeatureManager)swModel.FeatureManager;  
  
            swModel.ClearSelection2(true);  
  
            // Get the bodies to move  
            bodyArr = (object[])swPart.GetBodies2((int)swBodyType_e.swAllBodies, false);  
            SelectBodies(swApp, swModel, bodyArr);  
  
            // Move the bodies  
            swFeat = (Feature)swFeatMgr.InsertMoveCopyBody2(0.1, 0.2, 0.3, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, false, 1);  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>
```

```
    public SldWorks swApp;  
}
```

Move Bodies Example (VBA)

This example shows how to move all of the bodies in a part document.

```
' -----  
' Preconditions:  
' 1. Specified part document to open exists.  
' 2. Run the macro.  
'  
' Postconditions: All of the bodies in the part document  
' are moved to the specified location.  
'  
' NOTE: Because this part is used elsewhere, do not save  
' any changes when closing it.  
' -----
```

Option Explicit

```
Sub SelectBodies(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, bodyArr As Variant)  
' Select and mark the bodies to move  
  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swSelData As SldWorks.SelectData  
    Dim body As Variant  
    Dim swBody As SldWorks.Body2  
    Dim status As Boolean  
  
    Set swSelMgr = swModel.SelectionManager  
    Set swSelData = swSelMgr.CreateSelectData  
  
    If IsEmpty(bodyArr) Then Exit Sub  
    For Each body In bodyArr  
        Set swBody = body  
        swSelData.Mark = 1  
        status = swBody.Select2(True, swSelData)  
    Next body  
  
End Sub  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swPart As SldWorks.PartDoc  
    Dim bodyArr As Variant  
    Dim swFeatMgr As SldWorks.FeatureManager  
    Dim swFeat As SldWorks.Feature  
    Dim fileName As String  
    Dim errors As Long  
    Dim warnings As Long  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,  
warnings)  
    Set swPart = swModel  
    Set swFeatMgr = swModel.FeatureManager  
  
    swModel.ClearSelection2 True  
  
    ' Get the bodies to move  
    bodyArr = swPart.GetBodies2(swAllBodies, false)  
    SelectBodies swApp, swModel, bodyArr  
  
    ' Move the bodies  
    Set swFeat = swFeatMgr.InsertMoveCopyBody2(0.1, 0.2, 0.3, 0#, 0#, 0#, 0#, 0#, 0#, 0#, False, 1)  
  
End Sub
```

Move_Bodies_Example_VBNET (Topic)

SOLIDWORKS API Help

Move Bodies Example (VB.NET)

This example shows how to move all of the bodies in a part document.

' Preconditions:

- ' 1. Specified part document to open exists.
- ' 2. Run the macro.

' Postconditions: All of the bodies in the part document
' are moved to the specified location.

' **NOTE:** Because this part is used elsewhere, do not save
' any changes when closing it.

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

Sub SelectBodies(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal bodyArr As Object)

' Select and mark the bodies to move

Dim swSelMgr As SelectionMgr
Dim swSelData As SelectData
Dim body As Object
Dim swBody As Body2
Dim status As Boolean

swSelMgr = swModel.SelectionManager
swSelData = swSelMgr.CreateSelectData

If (bodyArr Is Nothing) Then Exit Sub
For Each body In bodyArr
swBody = body
swSelData.Mark = 1
status = swBody.Select2(True, swSelData)
Next body

End Sub

Sub Main()

Dim swModel As ModelDoc2
Dim swPart As PartDoc
Dim bodyArr As Object
Dim swFeatMgr As FeatureManager
Dim swFeat As Feature
Dim fileName As String
Dim errors As Integer
Dim warnings As Integer

fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt"
swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
swPart = swModel
swFeatMgr = swModel.FeatureManager

swModel.ClearSelection2(True)

' Get the bodies to move
bodyArr = swPart.GetBodies2(swBodyType_e.swAllBodies, False)
SelectBodies(swApp, swModel, bodyArr)

' Move the bodies
swFeat = swFeatMgr.InsertMoveCopyBody2(0.1, 0.2, 0.3, 0.0#, 0.0#, 0.0#, 0.0#, 0.0#, 0.0#, 0.0#, False, 1)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

Move Copy Sketch Entities Example (C#)

This example shows how to move, copy, and move and copy sketch entities.

```
//-----  
// Preconditions: Verify that the specified part template exists.  
//  
// Postconditions:  
// 1. Insert a breakpoint at the following lines of code:  
//  
//     swFeature = swPart.FeatureByName("Sketch1")  
//  
//     swModel.EditUndo2(1)  
//  
// 2. Run the macro.  
// 3. Examine the sketch at each breakpoint, then press F5.  
//     1. Opens a new part document.  
//     2. Opens a sketch and sketches a line and a circle.  
//     3. Selects the line and circle.  
//     4. Moves the line and circle.  
//     5. Moves and copies the line and circle once.  
//     6. Moves and copies the line and circle twice.  
// 4. Close the part document without saving it.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
  
namespace MoveOrCopyCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            ModelDocExtension swModelDocExt = default(ModelDocExtension);  
            PartDoc swPart = default(PartDoc);  
            Feature swFeature = default(Feature);  
            SketchManager swSketchMgr = default(SketchManager);  
            SketchSegment swSketchSegment = default(SketchSegment);  
            long lIdx = 0;  
            bool bCopy = false;  
            int lNumCopies = 0;  
            double[] aBasePoint = new double[3];  
            double[] aMoveVector = new double[3];  
            int errors = 0;  
            bool status = false;  
  
            // Open a new part document and sketch a line and a circle  
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2016\\templates\\Part.prt  
dot", 0, 0, 0);  
            swApp.ActivateDoc3("Part1", true, 0, ref errors);  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swPart = (PartDoc)swModel;  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
            swSketchMgr = (SketchManager)swModel.SketchManager;  
            swSketchMgr.InsertSketch(true);  
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(-0.096389, 0.032667, 0.0, -0.062943, 0.019437, 0.  
0);  
            swSketchSegment = (SketchSegment)swSketchMgr.CreateCircle(-0.084504, 0.013823, 0.0, -0.087932, 0.006083,  
0.0);  
  
            swFeature = (Feature)swPart.FeatureByName("Sketch1");  
            status = swFeature.Select2(false, 0);  
            swModel.EditSketch();  
            aBasePoint[0] = 0.0;  
            aBasePoint[1] = 0.0;  
            aBasePoint[2] = 0.0;  
            aMoveVector[0] = 0.1;  
            aMoveVector[1] = 0.0;  
            aMoveVector[2] = 0.0;
```

```

        for (lIdx = 0; lIdx <= 2; lIdx++)
        {
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", -0.0752087432116777, 0.0368667656031986,
0.0146398923143701, true, 0, null, 0);
            status = swModelDocExt.SelectByID("Arc1", "SKETCHSEGMENT", -0.0802420935887737, 0.0333695230163339,
0.019671897706856, true, 0, null, 0);
            switch ((lIdx))
            {
                case 0:
                    // Move
                    bCopy = false;
                    lNumCopies = 0;
                    break;
                case 1:
                    // Move and copy once
                    bCopy = true;
                    lNumCopies = 1;
                    break;
                case 2:
                    // Move and copy twice
                    bCopy = true;
                    lNumCopies = 2;
                    break;
            }
            if (!bCopy)
            {
                lNumCopies = 0;
            }
            swModelDocExt.MoveOrCopy(bCopy, lNumCopies, true, aBasePoint[0], aBasePoint[1], 0.0, aBasePoint[0] +
aMoveVector[0], aBasePoint[1] + aMoveVector[1], aBasePoint[2] + aMoveVector[2]);
            swModel.ClearSelection2(true);
            // Undo so that you can do it again, but differently
            swModel.EditUndo2(1);
        }
        swSketchMgr.InsertSketch(true);

    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}

```

Move Copy Sketch Entities Example (VBA)

This example shows how to move, copy, and move and copy sketch entities.

```
'-----  
' Preconditions: Verify that the specified part template exists.  
'  
' Postconditions:  
' 1. Step through the macro by pressing F8.  
'    1. Opens a new part document.  
'    2. Opens a sketch and sketches a line and a circle.  
'    3. Examine the sketch.  
'    4. Selects the line and circle.  
'    5. Moves the line and circle.  
'    6. Examine the sketch.  
'    7. Moves and copies the line and circle once.  
'    8. Examine the sketch.  
'    9. Moves and copies the line and circle twice.  
'   10. Examine the sketch.  
' 2. Close the part document without saving it.  
'-----
```

Option Explicit

```
Dim swApp           As SldWorks.SldWorks  
Dim swModel         As SldWorks.ModelDoc2  
Dim swModelDocExt   As SldWorks.ModelDocExtension  
Dim swPart          As SldWorks.PartDoc  
Dim swFeature       As SldWorks.Feature  
Dim swSketchMgr     As SldWorks.SketchManager  
Dim swSketchSegment As SldWorks.SketchSegment  
Dim lIdx            As Long  
Dim bCopy           As Boolean  
Dim lNumCopies      As Long  
Dim aBasePoint(2)  As Double  
Dim aMoveVector(2) As Double  
Dim errors          As Long  
Dim status          As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
  
    ' Open a new part document and sketch a line and a circle  
    Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)  
    swApp.ActivateDoc3 "Part1", True, False, errors  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
    Set swModelDocExt = swModel.Extension  
    Set swSketchMgr = swModel.SketchManager  
    swSketchMgr.InsertSketch True  
    Set swSketchSegment = swSketchMgr.CreateLine(-0.096389, 0.032667, 0#, -0.062943, 0.019437, 0#)  
    Set swSketchSegment = swSketchMgr.CreateCircle(-0.084504, 0.013823, 0#, -0.087932, 0.006083, 0#)  
  
    Set swFeature = swPart.FeatureByName("Sketch1")  
    status = swFeature.Select2(False, 0)  
    swModel.EditSketch  
  
    aBasePoint(0) = 0#  
    aBasePoint(1) = 0#  
    aBasePoint(2) = 0#  
  
    aMoveVector(0) = 0.1  
    aMoveVector(1) = 0#  
    aMoveVector(2) = 0#  
  
    For lIdx = 0 To 2  
        swModel.ClearSelection2 True  
        status = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", -7.52087432116777E-02, 3.68667656031986E-02,  
1.46398923143701E-02, True, 0, Nothing, 0)
```

```
status = swModelDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", -8.02420935887737E-02, 3.33695230163339E-02,  
0.019671897706856, True, 0, Nothing, 0)
```

```
Select Case (lIdx)
```

```
Case 0
```

```
' Move
```

```
    bCopy = False
```

```
    lNumCopies = 0
```

```
Case 1
```

```
' Move and copy once
```

```
    bCopy = True
```

```
    lNumCopies = 1
```

```
Case 2
```

```
' Move and copy twice
```

```
    bCopy = True
```

```
    lNumCopies = 2
```

```
End Select
```

```
If (Not bCopy) Then
```

```
    lNumCopies = 0
```

```
End If
```

```
swModelDocExt.MoveOrCopy bCopy, lNumCopies, True, aBasePoint(0), aBasePoint(1), 0#, aBasePoint(0) +  
aMoveVector(0), aBasePoint(1) + aMoveVector(1), aBasePoint(2) + aMoveVector(2)
```

```
swModel.ClearSelection2 True
```

```
' Undo so that you can do it again, but differently
```

```
swModel.EditUndo2 1
```

```
Next lIdx
```

```
swSketchMgr.InsertSketch True
```

```
End Sub
```

Move Copy Sketch Entities Example (VB.NET)

This example shows how to move, copy, and move and copy sketch entities.

```
'-----  
' Preconditions: Verify that the specified part template exists.  
'  
' Postconditions:  
' 1. Insert a breakpoint at the following lines of code:  
'  
'     swFeature = swPart.FeatureByName("Sketch1")  
'  
'     swModel.EditUndo2(1)  
'  
' 2. Run the macro.  
' 3. Examine the sketch at each breakpoint, then press F5.  
'    1. Opens a new part document.  
'    2. Opens a sketch and sketches a line and a circle.  
'    3. Selects the line and circle.  
'    4. Moves the line and circle.  
'    5. Moves and copies the line and circle once.  
'    6. Moves and copies the line and circle twice.  
' 4. Close the part document without saving it.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

Partial Class SolidWorksMacro

Public Sub main()

```
Dim swModel As ModelDoc2  
Dim swModelDocExt As ModelDocExtension  
Dim swPart As PartDoc  
Dim swFeature As Feature  
Dim swSketchMgr As SketchManager  
Dim swSketchSegment As SketchSegment  
Dim lIdx As Integer  
Dim bCopy As Boolean  
Dim lNumCopies As Integer  
Dim aBasePoint(2) As Double  
Dim aMoveVector(2) As Double  
Dim errors As Integer  
Dim status As Boolean
```

' Open a new part document and sketch a line and a circle

```
swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2013\templates\Part.prtdot", 0, 0, 0)
```

```
swApp.ActivateDoc3("Part1", True, False, errors)
```

```
swModel = swApp.ActiveDoc
```

```
swPart = swModel
```

```
swModelDocExt = swModel.Extension
```

```
swSketchMgr = swModel.SketchManager
```

```
swSketchMgr.InsertSketch(True)
```

```
swSketchSegment = swSketchMgr.CreateLine(-0.096389, 0.032667, 0.0#, -0.062943, 0.019437, 0.0#)
```

```
swSketchSegment = swSketchMgr.CreateCircle(-0.084504, 0.013823, 0.0#, -0.087932, 0.006083, 0.0#)
```

```
swFeature = swPart.FeatureByName("Sketch1")
```

```
status = swFeature.Select2(False, 0)
```

```
swModel.EditSketch()
```

```
aBasePoint(0) = 0.0#
```

```
aBasePoint(1) = 0.0#
```

```
aBasePoint(2) = 0.0#
```

```
aMoveVector(0) = 0.1
```

```
aMoveVector(1) = 0.0#
```

```
aMoveVector(2) = 0.0#
```

```
For lIdx = 0 To 2
```

```
swModel.ClearSelection2(True)
```

```
status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", -0.0752087432116777, 0.0368667656031986, 0.0  
146398923143701, True, 0, Nothing, 0)
```

```
status = swModelDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", -0.0802420935887737, 0.0333695230163339, 0.01  
9671897706856, True, 0, Nothing, 0)
```

```
Select Case (lIdx)
```

```
Case 0
```

```

        ' Move
        bCopy = False
        lNumCopies = 0
    Case 1
        ' Move and copy once
        bCopy = True
        lNumCopies = 1
    Case 2
        ' Move and copy twice
        bCopy = True
        lNumCopies = 2
    End Select
    If (Not bCopy) Then
        lNumCopies = 0
    End If
    swModelDocExt.MoveOrCopy(bCopy, lNumCopies, True, aBasePoint(0), aBasePoint(1), 0.0#, aBasePoint(0) + aMoveVector(0), aBasePoint(1) + aMoveVector(1), aBasePoint(2) + aMoveVector(2))
    swModel.ClearSelection2(True)
    ' Undo so that you can do it again, but differently
    swModel.EditUndo2(1)
Next lIdx
swSketchMgr.InsertSketch(True)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Move Freeze Bar Example (C#)

This example shows how to move the freeze bar to another location in the FeatureManager design tree.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\FreezeBarNeedsRebuild2.sldprt.
// 2. Inspect the FeatureManager design tree:
//    * The rebuild indicator is displayed in the FeatureManager design tree.
//    * The freeze bar is below Boss-Extrude1 in the FeatureManager design tree.
//    * Boss-Extrude1 is frozen in the FeatureManager design tree.
//    * Boss-Extrude2 is unfrozen in the FeatureManager design tree.
//    * Boss-Extrude1 has freeze updates pending.
//    * Boss-Extrude2 has no freeze updates pending.
//    * The model needs to be rebuilt.
//
// Postconditions:
// 1. Removes the rebuild indicator from the part.
// 2. Inspect the Immediate window to see which features have hidden locks.
// 3. Press F5 to continue.
//    * Freeze bar moved to the top of the FeatureManager design tree.
//    * Boss-Extrude1 is unfrozen.
// 4. Press F5 to continue.
//    * Freeze bar moved to the bottom of the FeatureManager design tree.
//    * Boss-Extrude1 and Boss-Extrude2 are frozen.
// 5. Press F5 to continue.
//    * Freeze bar moved to after Boss-Extrude2.
//    * Boss-Extrude1 and Boss-Extrude2 are frozen.
// 6. Press F5 to continue.
//    * Freeze bar moved to the top of the FeatureManager design tree.
//    * Boss-Extrude1 and Boss-Extrude2 are unfrozen.
// 7. Press F5 to continue.
//    * The model does not need to be rebuilt.
//    * Boss-Extrude1 and Boss-Extrude2 have no freeze updates pending.
// 8. Press F5 to close the model.
// 9. Inspect the Immediate window.
//
// NOTE: Because the model is used elsewhere,
// do not save changes when closing it.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
namespace MoveFreezeBar_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModDocExt;
        int lRet;
        FeatureManager featMgr;
        SelectionMgr selMgr;
        Feature featFrozen;
        Feature featUnFrozen;
        Feature feat;
        bool boolstatus;
        Object[] vfeats;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
```

```

swModDocExt = swModel.Extension;
featMgr = swModel.FeatureManager;
selMgr = (SelectionMgr)swModel.SelectionManager;

swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swUserEnableFreezeBar,
true);
swModDocExt.ShowPartRebuildIndicators = false;

Debug.Print("Number of Features is " + swModel.GetFeatureCount());

vfeats = (Object[])featMgr.GetFeatures(true);

for (int i = 0; i < vfeats.GetLength(0); i++ )
{
    feat = (Feature)vfeats[i];
    Debug.Print("Feature name from FeatureManager design tree is " + feat.Name);
    Debug.Print(feat.Name + " has a hidden lock? " + feat.IsHiddenLock());
}

boolstatus = swModDocExt.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, false, 0,
null, 0);
boolstatus = swModDocExt.SelectByID2("Boss-Extrude2", "BODYFEATURE", 0, 0, 0, true, 0,
null, 0);

feat = featMgr.GetFreezeLocation();
if ((feat != null))
{
    Debug.Print("");
    Debug.Print("Freeze bar location is after " + feat.Name);
    Debug.Print(feat.Name + " is frozen? " + feat.IsFrozen());
}

lRet = swModDocExt.NeedsRebuild2;
Debug.Print("Needs rebuild? " + lRet);

featFrozen = (Feature)selMgr.GetSelectedObject6(1, -1);
Debug.Print("Feature " + featFrozen.Name + " has frozen updates pending? " +
featFrozen.HasFrozenUpdatePending());
featUnFrozen = (Feature)selMgr.GetSelectedObject6(2, -1);
Debug.Print("Feature " + featUnFrozen.Name + " has frozen updates pending? " +
featUnFrozen.HasFrozenUpdatePending());
Debug.Print("");

swModel.ClearSelection2(true);

System.Diagnostics.Debugger.Break();

if ((featFrozen.HasFrozenUpdatePending()))
{
    lRet = featMgr.EditFreeze2((int)swMoveFreezeBarTo_e.swMoveFreezeBarToBeforeFeature,
featFrozen.Name, true, true);
    Debug.Print("Freeze bar moved to the top of the FeatureManager design tree. Press
F5.");
    System.Diagnostics.Debugger.Break();
    lRet = featMgr.EditFreeze2((int)swMoveFreezeBarTo_e.swMoveFreezeBarToEnd, "", true,
true);
    Debug.Print("Freeze bar moved to the bottom of the FeatureManager design tree. Press
F5.");
    System.Diagnostics.Debugger.Break();
    lRet = featMgr.EditFreeze2((int)swMoveFreezeBarTo_e.swMoveFreezeBarToAfterFeature,
featUnFrozen.Name, true, true);
    Debug.Print("Freeze bar moved to after Boss-Extrude2. Press F5.");
    System.Diagnostics.Debugger.Break();
    lRet = featMgr.EditFreeze2((int)swMoveFreezeBarTo_e.swMoveFreezeBarToTop, "", true,
true);
    Debug.Print("Freeze bar moved to the top of the FeatureManager design tree. Press
F5.");
    System.Diagnostics.Debugger.Break();
}
else
{
    lRet = featMgr.EditFreeze2((int)swMoveFreezeBarTo_e.swMoveFreezeBarToBeforeFeature,
featFrozen.Name, true, true);

```

```

        lRet = featMgr.EditFreeze2((int)swMoveFreezeBarTo_e.swMoveFreezeBarToEnd, "", true,
true);
        lRet = featMgr.EditFreeze2((int)swMoveFreezeBarTo_e.swMoveFreezeBarToBeforeFeature,
featUnFrozen.Name, true, true);
        lRet = featMgr.EditFreeze2((int)swMoveFreezeBarTo_e.swMoveFreezeBarToAfterFeature,
featUnFrozen.Name, true, true);
        lRet = featMgr.EditFreeze2((int)swMoveFreezeBarTo_e.swMoveFreezeBarToTop, "", true,
true);
    }

    lRet = swModDocExt.NeedsRebuild2;
    Debug.Print("Needs rebuild? " + lRet);

    Debug.Print("Feature " + featFrozen.Name + " has frozen updates pending? " +
featFrozen.HasFrozenUpdatePending());
    Debug.Print("Feature " + featUnFrozen.Name + " has frozen updates pending? " +
featUnFrozen.HasFrozenUpdatePending());

    feat = featMgr.GetFreezeLocation();
    // feat = Null if freeze bar is at the top of FeatureManager design tree
    if ((feat != null))
    {
        Debug.Print(feat.Name + " is frozen? " + feat.IsFrozen());
    }

    Debug.Print("Press F5 to close the model.");
    System.Diagnostics.Debugger.Break();

    swApp.CloseDoc("");
}

public SldWorks swApp;
}
}

```

Move Freeze Bar Example (VBA)

This example shows how to move the freeze bar to another location in the FeatureManager design tree.

```
' -----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\FreezeBarNeedsRebuild2.sldprt.
' 2. Inspect the FeatureManager design tree:
'   * The rebuild indicator is displayed in the FeatureManager design tree.
'   * The freeze bar is below Boss-Extrude1 in the FeatureManager design tree.
'   * Boss-Extrude1 is frozen in the FeatureManager design tree.
'   * Boss-Extrude2 is unfrozen in the FeatureManager design tree.
'   * Boss-Extrude1 has freeze updates pending.
'   * Boss-Extrude2 has no freeze updates pending.
'   * The model needs to be rebuilt.
'
' Postconditions:
' 1. Removes the rebuild indicator from the part.
' 2. Inspect the Immediate window to see which features have hidden locks.
' 3. Press F5 to continue.
'   * Freeze bar moved to the top of the FeatureManager design tree.
'   * Boss-Extrude1 is unfrozen.
' 4. Press F5 to continue.
'   * Freeze bar moved to the bottom of the FeatureManager design tree.
'   * Boss-Extrude1 and Boss-Extrude2 are frozen.
' 5. Press F5 to continue.
'   * Freeze bar moved to after Boss-Extrude2.
'   * Boss-Extrude1 and Boss-Extrude2 are frozen.
' 6. Press F5 to continue.
'   * Freeze bar moved to the top of the FeatureManager design tree.
'   * Boss-Extrude1 and Boss-Extrude2 are unfrozen.
' 7. Press F5 to continue.
'   * The model does not need to be rebuilt.
'   * Boss-Extrude1 and Boss-Extrude2 have no freeze updates pending.
' 8. Press F5 to close the model.
' 9. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModDocExt As SldWorks.ModelDocExtension
Dim lRet As Long
Dim featMgr As SldWorks.FeatureManager
Dim selMgr As SldWorks.SelectionMgr
Dim featFrozen As SldWorks.Feature
Dim featUnFrozen As SldWorks.Feature
Dim feat As SldWorks.Feature
Dim boolstatus As Boolean
Dim bRetVal As Boolean
Dim vfeats As Variant
Dim vfeat As Variant

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModDocExt = swModel.Extension
    Set featMgr = swModel.FeatureManager
    Set selMgr = swModel.SelectionManager

    swApp.SetUserPreferenceToggle swUserEnableFreezeBar, True
    swModDocExt.ShowPartRebuildIndicators = False
```

```

Debug.Print "Number of Features is " & swModel.GetFeatureCount

vfeats = featMgr.GetFeatures(True)

For Each vfeat In vfeats
    Set feat = vfeat
    Debug.Print "Feature name from FeatureManager design tree is " & feat.Name
    Debug.Print feat.Name & " has a hidden lock? " & feat.IsHiddenLock
Next

boolstatus = swModDocExt.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing,
0)
boolstatus = swModDocExt.SelectByID2("Boss-Extrude2", "BODYFEATURE", 0, 0, 0, True, 0, Nothing,
0)

Set feat = featMgr.GetFreezeLocation
If Not feat Is Nothing Then
    Debug.Print ""
    Debug.Print "Freeze bar location is after " & feat.Name
    Debug.Print feat.Name & " is frozen? " & feat.IsFrozen
End If

lRet = swModDocExt.NeedsRebuild2
Debug.Print "Needs rebuild? " & lRet

Set featFrozen = selMgr.GetSelectedObject6(1, -1)
Debug.Print "Feature " & featFrozen.Name & " has freeze updates pending? " &
featFrozen.HasFrozenUpdatePending
Set featUnFrozen = selMgr.GetSelectedObject6(2, -1)
Debug.Print "Feature " & featUnFrozen.Name & " has freeze updates pending? " &
featUnFrozen.HasFrozenUpdatePending
Debug.Print ""

swModel.ClearSelection2 True

Stop

If (featFrozen.HasFrozenUpdatePending) Then
    lRet = featMgr.EditFreeze2(swMoveFreezeBarToBeforeFeature, featFrozen.Name, True, True)
    Debug.Print "Freeze bar moved to the top of the FeatureManager design tree. Press F5."
    Stop
    lRet = featMgr.EditFreeze2(swMoveFreezeBarToEnd, "", True, True)
    Debug.Print "Freeze bar moved to the bottom of the FeatureManager design tree. Press F5."
    Stop
    lRet = featMgr.EditFreeze2(swMoveFreezeBarToAfterFeature, featUnFrozen.Name, True, True)
    Debug.Print "Freeze bar moved to after Boss-Extrude2. Press F5."
    Stop
    lRet = featMgr.EditFreeze2(swMoveFreezeBarToTop, "", True, True)
    Debug.Print "Freeze bar moved to the top of the FeatureManager design tree. Press F5."
    Stop
Else
    lRet = featMgr.EditFreeze2(swMoveFreezeBarToBeforeFeature, featFrozen.Name, True, True)
    lRet = featMgr.EditFreeze2(swMoveFreezeBarToEnd, "", True, True)
    lRet = featMgr.EditFreeze2(swMoveFreezeBarToBeforeFeature, featUnFrozen.Name, True, True)
    lRet = featMgr.EditFreeze2(swMoveFreezeBarToAfterFeature, featUnFrozen.Name, True, True)
    lRet = featMgr.EditFreeze2(swMoveFreezeBarToTop, "", True, True)
End If

Debug.Print ""
lRet = swModDocExt.NeedsRebuild2
Debug.Print "Needs rebuild? " & lRet

Debug.Print "Feature " & featFrozen.Name & " has freeze updates pending? " &
featFrozen.HasFrozenUpdatePending
Debug.Print "Feature " & featUnFrozen.Name & " has freeze updates pending? " &
featUnFrozen.HasFrozenUpdatePending

```

```
        Set feat = featMgr.GetFreezeLocation ' feat = Nothing if freeze bar is at the top of
FeatureManager design tree
    If Not feat Is Nothing Then
        Debug.Print feat.Name & " is frozen?" & feat.IsFrozen
    End If

    Debug.Print "Press F5 to close the model."
    Stop

    swApp.CloseDoc ("")
End Sub
```

Move Freeze Bar Example (VB.NET)

This example shows how to move the freeze bar to another location in the FeatureManager design tree.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\FreezeBarNeedsRebuild2.sldprt.  
' 2. Inspect the FeatureManager design tree:  
'    * The rebuild indicator is displayed in the FeatureManager design tree.  
'    * The freeze bar is below Boss-Extrude1 in the FeatureManager design tree.  
'    * Boss-Extrude1 is frozen in the FeatureManager design tree.  
'    * Boss-Extrude2 is unfrozen in the FeatureManager design tree.  
'    * Boss-Extrude1 has freeze updates pending.  
'    * Boss-Extrude2 has no freeze updates pending.  
'    * The model needs to be rebuilt.  
'  
' Postconditions:  
' 1. Removes the rebuild indicator from the part.  
' 2. Inspect the Immediate window to see which features have hidden locks.  
' 3. Press F5 to continue.  
'    * Freeze bar moved to the top of the FeatureManager design tree.  
'    * Boss-Extrude1 is unfrozen.  
' 4. Press F5 to continue.  
'    * Freeze bar moved to the bottom of the FeatureManager design tree.  
'    * Boss-Extrude1 and Boss-Extrude2 are frozen.  
' 5. Press F5 to continue.  
'    * Freeze bar moved to after Boss-Extrude2.  
'    * Boss-Extrude1 and Boss-Extrude2 are frozen.  
' 6. Press F5 to continue.  
'    * Freeze bar moved to the top of the FeatureManager design tree.  
'    * Boss-Extrude1 and Boss-Extrude2 are unfrozen.  
' 7. Press F5 to continue.  
'    * The model does not need to be rebuilt.  
'    * Boss-Extrude1 and Boss-Extrude2 have no freeze updates pending.  
' 8. Press F5 to close the model.  
' 9. Inspect the Immediate window.  
'  
' NOTE: Because the model is used elsewhere,  
' do not save changes when closing it.  
' -----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2  
    Dim swModDocExt As ModelDocExtension  
    Dim lRet As Integer  
    Dim featMgr As FeatureManager  
    Dim selMgr As SelectionMgr  
    Dim featFrozen As Feature  
    Dim featUnFrozen As Feature  
    Dim feat As Feature  
    Dim boolstatus As Boolean  
    Dim bRetVal As Boolean  
    Dim vfeats As Object  
    Dim vfeat As Object
```

```
    Sub main()
```

```
        swModel = swApp.ActiveDoc  
        swModDocExt = swModel.Extension  
        featMgr = swModel.FeatureManager
```

```

selMgr = swModel.SelectionManager

swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swUserEnableFreezeBar, True)
swModDocExt.ShowPartRebuildIndicators = False

Debug.Print("Number of Features is " & swModel.GetFeatureCount)

vfeats = featMgr.GetFeatures(True)

For Each vfeat In vfeats
    feat = vfeat
    Debug.Print("Feature name from FeatureManager design tree is " & feat.Name)
    Debug.Print(feat.Name & " has a hidden lock? " & feat.IsHiddenLock)
Next

boolstatus = swModDocExt.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0,
Nothing, 0)
boolstatus = swModDocExt.SelectByID2("Boss-Extrude2", "BODYFEATURE", 0, 0, 0, True, 0,
Nothing, 0)

feat = featMgr.GetFreezeLocation
If Not feat Is Nothing Then
    Debug.Print("")
    Debug.Print("Freeze bar location is after " & feat.Name)
    Debug.Print(feat.Name & " is frozen? " & feat.IsFrozen)
End If

lRet = swModDocExt.NeedsRebuild2
Debug.Print("Needs rebuild? " & lRet)

featFrozen = selMgr.GetSelectedObject6(1, -1)
Debug.Print("Feature " & featFrozen.Name & " has freeze updates pending? " &
featFrozen.HasFrozenUpdatePending)
featUnFrozen = selMgr.GetSelectedObject6(2, -1)
Debug.Print("Feature " & featUnFrozen.Name & " has freeze updates pending? " &
featUnFrozen.HasFrozenUpdatePending)
Debug.Print("")

swModel.ClearSelection2(True)

Stop

If (featFrozen.HasFrozenUpdatePending) Then
    lRet = featMgr.EditFreeze2(swMoveFreezeBarTo_e.swMoveFreezeBarToBeforeFeature,
featFrozen.Name, True, True)
    Debug.Print("Freeze bar moved to the top of the FeatureManager design tree. Press F5.")
    Stop
    lRet = featMgr.EditFreeze2(swMoveFreezeBarTo_e.swMoveFreezeBarToEnd, "", True, True)
    Debug.Print("Freeze bar moved to the bottom of the FeatureManager design tree. Press
F5.")
    Stop
    lRet = featMgr.EditFreeze2(swMoveFreezeBarTo_e.swMoveFreezeBarToAfterFeature,
featUnFrozen.Name, True, True)
    Debug.Print("Freeze bar moved to after Boss-Extrude2. Press F5.")
    Stop
    lRet = featMgr.EditFreeze2(swMoveFreezeBarTo_e.swMoveFreezeBarToTop, "", True, True)
    Debug.Print("Freeze bar moved to the top of the FeatureManager design tree. Press F5.")
    Stop
Else
    lRet = featMgr.EditFreeze2(swMoveFreezeBarTo_e.swMoveFreezeBarToBeforeFeature,
featFrozen.Name, True, True)
    lRet = featMgr.EditFreeze2(swMoveFreezeBarTo_e.swMoveFreezeBarToEnd, "", True, True)
    lRet = featMgr.EditFreeze2(swMoveFreezeBarTo_e.swMoveFreezeBarToBeforeFeature,
featUnFrozen.Name, True, True)
    lRet = featMgr.EditFreeze2(swMoveFreezeBarTo_e.swMoveFreezeBarToAfterFeature,
featUnFrozen.Name, True, True)
    lRet = featMgr.EditFreeze2(swMoveFreezeBarTo_e.swMoveFreezeBarToTop, "", True, True)
End If

Debug.Print("")
lRet = swModDocExt.NeedsRebuild2
Debug.Print("Needs rebuild? " & lRet)

```

```

        Debug.Print("Feature " & featFrozen.Name & " has freeze updates pending? " &
featFrozen.HasFrozenUpdatePending)
        Debug.Print("Feature " & featUnFrozen.Name & " has freeze updates pending? " &
featUnFrozen.HasFrozenUpdatePending)

        feat = featMgr.GetFreezeLocation ' feat = Nothing if freeze bar is at the top of
FeatureManager design tree
        If Not feat Is Nothing Then
            Debug.Print(feat.Name & " is frozen? " & feat.IsFrozen)
        End If

        Debug.Print("Press F5 to close the model.")
        Stop

        swApp.CloseDoc("")

    End Sub

    Public swApp As SldWorks

End Class

```

Move Rollback Bar Using Feature Example (C#)

This example shows how to move the rollback bar above and below a selected feature in a part document.

```
//-----
// Preconditions: Verify that the part to open exists.
// 2. Run the macro.
//
// Postconditions:
// 1. Opens the part document.
// 2. Selects the Sweep1 feature.
// 3. Moves the rollback bar to feature above the
//    Sweep1 feature; i.e., the Revolve1 feature.
// 4. Examine the graphics area to verify, then press
//    F5
// 5. Moves the rollback bar to below the Sweep1 feature.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace EditRollbackPartDocCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            PartDoc swPart = default(PartDoc);
            FeatureManager swFeatureMgr = default(FeatureManager);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\cstick.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.s
wOpenDocOptions_Silent, "", ref errors, ref warnings);
            swPart = (PartDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Move rollback bar to above Sweep1
            status = swModelDocExt.SelectByID2("Sweep1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swPart.EditRollback();

            System.Diagnostics.Debugger.Break();
            //Examine graphics area, then press F5
            // Move rollback bar to below Sweep1

            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            status = swFeatureMgr.EditRollback((int)swMoveRollbackBarTo_e.swMoveRollbackBarToBeforeFeature, "");
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Move Rollback Bar Using Feature Example (VBA)

This example shows how to move the rollback bar above and below a selected feature in a part document.

```
' -----  
' Preconditions: Verify that the part to open exists.  
' 2. Run the macro.  
'  
' Postconditions:  
' 1. Opens the part document.  
' 2. Selects the Sweep1 feature.  
' 3. Moves the rollback bar to feature above the  
'    Sweep1 feature; i.e., the Revolve1 feature.  
' 4. Examine the graphics area to verify, then press  
'    F5  
' 5. Moves the rollback bar to below the Sweep1 feature.  
' -----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swPart As SldWorks.PartDoc  
Dim swFeatureMgr As SldWorks.FeatureManager  
Dim fileName As String  
Dim status As Boolean  
Dim errors As Long, warnings As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cstick.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent,  
    "", errors, warnings)  
    Set swPart = swModel  
    Set swModelDocExt = swModel.Extension  
  
    ' Move rollback bar to above Sweep1  
    status = swModelDocExt.SelectByID2("Sweep1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)  
    swPart.EditRollback  
  
    Stop  
    'Examine graphics area, then press F5  
  
    ' Move rollback bar to below Sweep1  
    Set swFeatureMgr = swModel.FeatureManager  
    status = swFeatureMgr.EditRollback(swMoveRollbackBarTo_e.swMoveRollbackBarToBeforeFeature, "")  
  
End Sub
```

Move_Rollback_Bar_Using_Feature_Example_VBNET (Topic)

SOLIDWORKS API Help

Move Rollback Bar Using Feature Example (VB.NET)

This example shows how to move the rollback bar above and below a selected feature in a part document.

```
'-----  
' Preconditions: Verify that the part to open exists.  
' 2. Run the macro.  
'-----  
' Postconditions:  
' 1. Opens the part document.  
' 2. Selects the Sweep1 feature.  
' 3. Moves the rollback bar to feature above the  
'    Sweep1 feature; i.e., the Revolve1 feature.  
' 4. Examine the graphics area to verify, then press  
'    F5  
' 5. Moves the rollback bar to below the Sweep1 feature.  
'-----  
Imports SolidWorks.Interop.swconst  
Imports SolidWorks.Interop.sldworks  
Imports System.Runtime.InteropServices  
Imports System  
  
Partial Class SolidWorksMacro  
  
    Public Sub Main()  
  
        Dim swModel As ModelDoc2  
        Dim swModelDocExt As ModelDocExtension  
        Dim swPart As PartDoc  
        Dim swFeatureMgr As FeatureManager  
        Dim fileName As String  
        Dim status As Boolean  
        Dim errors As Integer, warnings As Integer  
  
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cstick.sldprt"  
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "  
", errors, warnings)  
        swPart = swModel  
        swModelDocExt = swModel.Extension  
        ' Move rollback bar to above Sweep1  
        status = swModelDocExt.SelectByID2("Sweep1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)  
        swPart.EditRollback()  
        Stop  
        'Examine graphics area, then press F5  
        ' Move rollback bar to below Sweep1  
        swFeatureMgr = swModel.FeatureManager  
        status = swFeatureMgr.EditRollback(swMoveRollbackBarTo_e.swMoveRollbackBarToBeforeFeature, "")  
  
    End Sub  
  
    ''' <summary>  
    ''' The SldWorks swApp variable is pre-assigned for you.  
    ''' </summary>  
    Public swApp As SldWorks  
  
End Class
```

Move_Selected_Face_Example_VB (Topic)

SOLIDWORKS API Help

Move Selected Face Example (VBA)

This example shows how to move the selected face in a part.

```
'-----
```

```
'  
' Preconditions: Part is open.
```

```
'  
' Postconditions: The selected face is moved.  
'  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
```

```
Dim swModel As SldWorks.ModelDoc2
```

```
Dim swModelDocExt As SldWorks.ModelDocExtension
```

```
Dim swFeatMgr As SldWorks.FeatureManager
```

```
Dim swFeat As SldWorks.Feature
```

```
Dim boolstatus As Boolean
```

```
Sub main()
```

```
Set swApp = Application.SldWorks
```

```
Set swModel = swApp.ActiveDoc
```

```
Set swModelDocExt = swModel.Extension
```

```
Set swFeatMgr = swModel.FeatureManager
```

```
' Select face to move
```

```
boolstatus = swModelDocExt.SelectByID2("", "FACE", -0.06133102397996, 0.049999999999999, 0.02353079473198, False, 1, Nothing, 0)
```

```
' Select the direction reference
```

```
boolstatus = swModelDocExt.SelectByID2("", "EDGE", -0.05000883624581, 0.02773250193934, 0.04000883624559, True, 2, Nothing, 0)
```

```
' Move the selected face
```

```
Set swFeat = swFeatMgr.InsertMoveFace(1, False, 0.034907, 0.05)
```

```
swModel.ViewZoomtofit2
```

```
End Sub
```

Multiselect Same and Different Objects Example (C#)

This example shows how to select multiple faces, multiple edges, and multiple faces and edges.

```
//-----
// Preconditions:
// 1. Verify that the part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the part.
// 2. Selects and adds three faces to an array of faces.
// 3. Selects and adds two edges to an array of edges.
// 4. Selects one face and three edges and adds them to an
//    array of entities.
// 5. Multiselects each array of faces, edges, and entities.
// 6. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System.Diagnostics;
using System.Windows.Forms;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public PartDoc swPart;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Face2[] faceArray = new Face2[3];
            Edge[] edgeArray = new Edge[2];
            Entity[] entitiesArray = new Entity[4];
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            int nbrSelections = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\box.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.s
wOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            //Select three faces and add them to an array of faces
            status = swModelDocExt.SelectByID("", "FACE", 0.00339003579642849, 0.0165689832048201, 0.046052156634573
3, false, 0, null, 0);
            faceArray[0] = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            status = swModelDocExt.SelectByID("", "FACE", -0.00464858017755887, 0.02999999999999159, -0.0021315888220
1014, false, 0, null, 0);
            faceArray[1] = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            status = swModelDocExt.SelectByID("", "FACE", -0.0310506911185371, 0.02999999999999159, -0.01660017836445
6, false, 0, null, 0);
            faceArray[2] = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            swModel.ClearSelection2(true);

            //Select two edges and them to an array of edges
            status = swModelDocExt.SelectByID("", "EDGE", -0.0103092369793103, 0.0304904435424191, 0.045718956530038
2, false, 0, null, 0);
            edgeArray[0] = (Edge)swSelMgr.GetSelectedObject6(1, -1);
            status = swModelDocExt.SelectByID("", "EDGE", 0.0437175784318242, 0.0301364202527452, 0.028332486890065,
false, 0, null, 0);
            edgeArray[1] = (Edge)swSelMgr.GetSelectedObject6(1, -1);
            swModel.ClearSelection2(true);

            //Select one face and three edges and add them to an array of entities
            status = swModelDocExt.SelectByID("", "FACE", -0.00725501009526397, 0.02999999999999159, -0.0023637937484
2954, false, 0, null, 0);
            entitiesArray[0] = (Entity)swSelMgr.GetSelectedObject6(1, -1);
            status = swModelDocExt.SelectByID("", "EDGE", 0.0435139962395397, 0.0148277472299014, 0.0462522660892546
, false, 0, null, 0);
            entitiesArray[1] = (Entity)swSelMgr.GetSelectedObject6(1, -1);
```

```

        status = swModelDocExt.SelectByID("", "EDGE", 0.0441251049156222, -0.000180110278279244, 0.0139372949385
006, false, 0, null, 0);
        entitiesArray[2] = (Entity)swSelMgr.GetSelectedObject6(1, -1);
        status = swModelDocExt.SelectByID("", "EDGE", 0.0303098420922652, 0.000354499639115602, 0.04581131481154
44, false, 0, null, 0);
        entitiesArray[3] = (Entity)swSelMgr.GetSelectedObject6(1, -1);
        swModel.ClearSelection2(true);

        //Multiselect faces, edges, and entities using IModelDocExtension::MultiSelect2
        Debug.Print("Number of selections in:");
        nbrSelections = swModelDocExt.MultiSelect2(faceArray, false, null);
        Debug.Print(" face array: " + nbrSelections);
        swModel.ClearSelection2(true);
        nbrSelections = swModelDocExt.MultiSelect2(edgeArray, false, null);
        Debug.Print(" edge array: " + nbrSelections);
        swModel.ClearSelection2(true);
        nbrSelections = swModelDocExt.MultiSelect2(entitiesArray, false, null);
        Debug.Print(" entities (face and edges) array: " + nbrSelections);
        swModel.ClearSelection2(true);

    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>

    public SldWorks swApp;

}
}

```

Multiselect Same and Different Objects Example (VBA)

This example shows how to select multiple faces, multiple edges, and multiple faces and edges.

```
'-----  
' Preconditions:  
' 1. Verify that the part to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the part.  
' 2. Selects and adds three faces to an array of faces.  
' 3. Selects and adds two edges to an array of edges.  
' 4. Selects one face and three edges and adds them to an  
'    array of entities.  
' 5. Multiselects each array of faces, edges, and entities.  
' 6. Examine the Immediate window.  
'  
' NOTE: Because the part is used elsewhere do not save  
' changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim faceArray(2) As SldWorks.Face2  
Dim edgeArray(1) As SldWorks.Edge  
Dim entitiesArray(3) As SldWorks.Entity  
Dim fileName As String  
Dim status As Boolean  
Dim errors As Long  
Dim warnings As Long  
Dim nbrSelections As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent,  
    "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
    Set swSelMgr = swModel.SelectionManager  
  
    'Select three faces and add them to an array of faces  
    status = swModelDocExt.SelectByID2("", "FACE", 3.39003579642849E-03, 1.65689832048201E-02, 4.60521566345733E-02,  
False, 0, Nothing, 0)  
    Set faceArray(0) = swSelMgr.GetSelectedObject6(1, -1)  
    status = swModelDocExt.SelectByID2("", "FACE", -4.64858017755887E-03, 2.9999999999159E-02, -2.13158882201014E-  
03, False, 0, Nothing, 0)  
    Set faceArray(1) = swSelMgr.GetSelectedObject6(1, -1)  
    status = swModelDocExt.SelectByID2("", "FACE", -3.10506911185371E-02, 2.9999999999159E-02, -0.016600178364456,  
False, 0, Nothing, 0)  
    Set faceArray(2) = swSelMgr.GetSelectedObject6(1, -1)  
    swModel.ClearSelection2 True  
  
    'Select two edges and them to an array of edges  
    status = swModelDocExt.SelectByID2("", "EDGE", -1.03092369793103E-02, 3.04904435424191E-02, 4.57189565300382E-02,  
False, 0, Nothing, 0)  
    Set edgeArray(0) = swSelMgr.GetSelectedObject6(1, -1)  
    status = swModelDocExt.SelectByID2("", "EDGE", 4.37175784318242E-02, 3.01364202527452E-02, 0.028332486890065,  
False, 0, Nothing, 0)  
    Set edgeArray(1) = swSelMgr.GetSelectedObject6(1, -1)  
    swModel.ClearSelection2 True  
  
    'Select one face and three edges and add them to an array of entities  
    status = swModelDocExt.SelectByID2("", "FACE", -7.25501009526397E-03, 2.9999999999159E-02, -2.36379374842954E-  
03, False, 0, Nothing, 0)  
    Set entitiesArray(0) = swSelMgr.GetSelectedObject6(1, -1)  
    status = swModelDocExt.SelectByID2("", "EDGE", 4.35139962395397E-02, 1.48277472299014E-02, 4.62522660892546E-02,
```

```

False, 0, Nothing, 0)
    Set entitiesArray(1) = swSelMgr.GetSelectedObject6(1, -1)
    status = swModelDocExt.SelectByID2("", "EDGE", 4.41251049156222E-02, -1.80110278279244E-04, 1.39372949385006E-02,
False, 0, Nothing, 0)
    Set entitiesArray(2) = swSelMgr.GetSelectedObject6(1, -1)
    status = swModelDocExt.SelectByID2("", "EDGE", 3.03098420922652E-02, 3.54499639115602E-04, 4.58113148115444E-02,
False, 0, Nothing, 0)
    Set entitiesArray(3) = swSelMgr.GetSelectedObject6(1, -1)
    swModel.ClearSelection2 True

    'Multiselect faces, edges, and entities using IModelDocExtension::MultiSelect2
    Debug.Print "Number of selections in:"
    nbrSelections = swModelDocExt.MultiSelect2(faceArray, False, Nothing)
    Debug.Print "  face array: " & nbrSelections
    swModel.ClearSelection2 True
    nbrSelections = swModelDocExt.MultiSelect2(edgeArray, False, Nothing)
    Debug.Print "  edge array: " & nbrSelections
    swModel.ClearSelection2 True
    nbrSelections = swModelDocExt.MultiSelect2(entitiesArray, False, Nothing)
    Debug.Print "  entities (face and edges) array: " & nbrSelections
    swModel.ClearSelection2 True

End Sub

```

Multiselect Same and Different Objects Example (VB.NET)

This example shows how to select multiple faces, multiple edges, and multiple faces and edges.

```

'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Selects and adds three faces to an array of faces.
' 3. Selects and adds two edges to an array of edges.
' 4. Selects one face and three edges and adds them to an
'    array of entities.
' 5. Multiselects each array of faces, edges, and entities.
' 6. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere do not save
'       changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim faceArray(2) As Face2
        Dim edgeArray(1) As Edge
        Dim entitiesArray(3) As Entity
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim nbrSelections As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "
", errors, warnings)
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager

        'Select three faces and add them to an array of faces
        status = swModelDocExt.SelectByID("", "FACE", 0.00339003579642849, 0.0165689832048201, 0.0460521566345733, F
alse, 0, Nothing, 0)
        faceArray(0) = swSelMgr.GetSelectedObject6(1, -1)
        status = swModelDocExt.SelectByID("", "FACE", -0.00464858017755887, 0.02999999999999159, -0.00213158882201014
, False, 0, Nothing, 0)
        faceArray(1) = swSelMgr.GetSelectedObject6(1, -1)
        status = swModelDocExt.SelectByID("", "FACE", -0.0310506911185371, 0.02999999999999159, -0.016600178364456, F
alse, 0, Nothing, 0)
        faceArray(2) = swSelMgr.GetSelectedObject6(1, -1)
        swModel.ClearSelection2(True)

        'Select two edges and them to an array of edges
        status = swModelDocExt.SelectByID("", "EDGE", -0.0103092369793103, 0.0304904435424191, 0.0457189565300382, F
alse, 0, Nothing, 0)
        edgeArray(0) = swSelMgr.GetSelectedObject6(1, -1)
        status = swModelDocExt.SelectByID("", "EDGE", 0.0437175784318242, 0.0301364202527452, 0.028332486890065, Fal
se, 0, Nothing, 0)
        edgeArray(1) = swSelMgr.GetSelectedObject6(1, -1)
        swModel.ClearSelection2(True)

        'Select one face and three edges and add them to an array of entities
        status = swModelDocExt.SelectByID("", "FACE", -0.00725501009526397, 0.02999999999999159, -0.00236379374842954
, False, 0, Nothing, 0)
        entitiesArray(0) = swSelMgr.GetSelectedObject6(1, -1)
        status = swModelDocExt.SelectByID("", "EDGE", 0.0435139962395397, 0.0148277472299014, 0.0462522660892546, Fa
lse, 0, Nothing, 0)
        entitiesArray(1) = swSelMgr.GetSelectedObject6(1, -1)
        status = swModelDocExt.SelectByID("", "EDGE", 0.0441251049156222, -0.000180110278279244, 0.0139372949385006,
False, 0, Nothing, 0)
        entitiesArray(2) = swSelMgr.GetSelectedObject6(1, -1)
        status = swModelDocExt.SelectByID("", "EDGE", 0.0303098420922652, 0.000354499639115602, 0.0458113148115444,
False, 0, Nothing, 0)
    
```

```

entitiesArray(3) = swSelMgr.GetSelectedObject6(1, -1)
swModel.ClearSelection2(True)

'Multiselect faces, edges, and entities using IModelDocExtension::MultiSelect2
Debug.Print("Number of selections in:")
nbrSelections = swModelDocExt.MultiSelect2(faceArray, False, Nothing)
Debug.Print("  face array: " & nbrSelections)
swModel.ClearSelection2(True)
nbrSelections = swModelDocExt.MultiSelect2(edgeArray, False, Nothing)
Debug.Print("  edge array: " & nbrSelections)
swModel.ClearSelection2(True)
nbrSelections = swModelDocExt.MultiSelect2(entitiesArray, False, Nothing)
Debug.Print("  entities (face and edges) array: " & nbrSelections)
swModel.ClearSelection2(True)

```

End Sub

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Offset Edges to Create 3D Sketch Example (C#)

This example shows how to offset edges to create a 3D sketch on a face.

```
//-----
// Preconditions: Verify that the part to open exists.
//
// Postconditions:
// 1. Opens the part.
// 2. Selects the edges to offset.
// 3. Creates a 3D sketch on the face whose edges were selected.
// 4. Examine the graphics area.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchManager = default(SketchManager);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\lesson1\\tutoriala.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.s
wOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Select the edges
            status = swModelDocExt.SelectByID("", "EDGE", 0.06, 0.12, 0.03, true, 0, null, 0);
            status = swModelDocExt.SelectByID("", "EDGE", 0, 0.12, 0.015, true, 0, null, 0);
            status = swModelDocExt.SelectByID("", "EDGE", 0.06, 0.12, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID("", "EDGE", 0.12, 0.12, 0.015, true, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID("", "EDGE", 0.06, 0.12, 0.03, true, 1, null, 0);
            status = swModelDocExt.SelectByID("", "EDGE", 0, 0.12, 0.015, true, 1, null, 0);
            status = swModelDocExt.SelectByID("", "EDGE", 0.06, 0.12, 0, true, 1, null, 0);
            status = swModelDocExt.SelectByID("", "EDGE", 0.12, 0.12, 0.015, true, 1, null, 0);

            //Create a 3D sketch
            status = swModelDocExt.SketchOffsetOnSurface(0.01, false, true, false);

            swModel.ClearSelection2(true);

            //Close the sketch
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.InsertSketch(true);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Offset Edges to Create 3D Sketch Example (VBA)

This example shows how to offset edges to create a 3D sketch on a face.

```
'-----  
' Preconditions: Verify that the part to open exists.  
'  
' Postconditions:  
' 1. Opens the part.  
' 2. Selects the edges to offset.  
' 3. Creates a 3D sketch on the face whose edges were selected.  
' 4. Examine the graphics area.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSketchManager As SldWorks.SketchManager  
Dim status As Boolean  
Dim errors As Long  
Dim warnings As Long  
Dim fileName As String  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\lesson1\tutor1a.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent,  
    "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
  
    'Select the edges  
    status = swModelDocExt.SelectByID("", "EDGE", 0.06, 0.12, 0.03, True, 0, Nothing, 0)  
    status = swModelDocExt.SelectByID("", "EDGE", 0, 0.12, 0.015, True, 0, Nothing, 0)  
    status = swModelDocExt.SelectByID("", "EDGE", 0.06, 0.12, 0, True, 0, Nothing, 0)  
    status = swModelDocExt.SelectByID("", "EDGE", 0.12, 0.12, 0.015, True, 0, Nothing, 0)  
    swModel.ClearSelection2 True  
    status = swModelDocExt.SelectByID("", "EDGE", 0.06, 0.12, 0.03, True, 1, Nothing, 0)  
    status = swModelDocExt.SelectByID("", "EDGE", 0, 0.12, 0.015, True, 1, Nothing, 0)  
    status = swModelDocExt.SelectByID("", "EDGE", 0.06, 0.12, 0, True, 1, Nothing, 0)  
    status = swModelDocExt.SelectByID("", "EDGE", 0.12, 0.12, 0.015, True, 1, Nothing, 0)  
  
    'Create a 3D sketch  
    status = swModelDocExt.SketchOffsetOnSurface(0.01, False, True, False)  
  
    swModel.ClearSelection2 True  
  
    'Close the sketch  
    Set swSketchManager = swModel.SketchManager  
    swSketchManager.InsertSketch True  
  
End Sub
```

Offset Edges to Create 3D Sketch Example (VB.NET)

This example shows how to offset edges to create a 3D sketch on a face.

```

'-----
' Preconditions: Verify that the part to open exists.
'
' Postconditions:
' 1. Opens the part.
' 2. Selects the edges to offset.
' 3. Creates a 3D sketch on the face whose edges were selected.
' 4. Examine the graphics area.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchManager As SketchManager
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\lesson1\tutor1a.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "
", errors, warnings)
        swModelDocExt = swModel.Extension

        'Select the edges
        status = swModelDocExt.SelectByID2("", "EDGE", 0.06, 0.12, 0.03, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "EDGE", 0, 0.12, 0.015, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "EDGE", 0.06, 0.12, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "EDGE", 0.12, 0.12, 0.015, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("", "EDGE", 0.06, 0.12, 0.03, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "EDGE", 0, 0.12, 0.015, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "EDGE", 0.06, 0.12, 0, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "EDGE", 0.12, 0.12, 0.015, True, 1, Nothing, 0)

        'Create a 3D sketch
        status = swModelDocExt.SketchOffsetOnSurface(0.01, False, True, False)

        swModel.ClearSelection2(True)

        'Close the sketch
        swSketchManager = swModel.SketchManager
        swSketchManager.InsertSketch(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Offset Selected Edges in Active Sketch Example (VBA)

This example shows how to offset the selected edges to generate geometry in the active sketch.

```
' -----  
' Preconditions:  
' 1. Open a part.  
' 2. Open a new sketch on a plane.  
' 3. Select one or more edges in the part coincident  
'    to the plane.  
'  
' Postconditions:  
' 1. Offsets the selected edges.  
' 2. Examine the sketch in the graphics area.  
' -----
```

Option Explicit

```
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSketchMgr As SldWorks.SketchManager  
    Dim swSketch As SldWorks.Sketch  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSketchMgr = swModel.SketchManager  
    Set swSketch = swModel.GetActiveSketch2  
  
    bRet = swSketchMgr.SketchUseEdge(False)
```

End Sub

Only Show Selected Components Example (VBA)

This example shows how to only show the selected components in an assembly.

```
' -----  
' Preconditions:  
' 1. Open an assembly.  
' 2. Select one or more components.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Hides all components except the selected components.  
' 2. Examine the Immediate window, graphics area, and the  
'    FeatureManager design tree.  
'  
' NOTES:  
' * If a subassembly is selected, then all child components  
'   (parts and sub-subassemblies) are shown.  
' * If a sub-subassembly is selected, all parents are recursively  
'   shown at the top-level assembly.  
' -----
```

Option Explicit

```
Function IsCompInSelList(swSelCompArr() As SldWorks.Component2, swComp As SldWorks.Component2) As Boolean
```

```
    Dim vSelCompArr As Variant  
    Dim vSelComp As Variant  
  
    vSelCompArr = swSelCompArr  
    For Each vSelComp In vSelCompArr  
        If vSelComp Is swComp Then  
            IsCompInSelList = True  
            Exit Function  
        End If  
    Next vSelComp  
  
    IsCompInSelList = False
```

```
End Function
```

```
Sub ShowParentComp(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swComp As SldWorks.Component2)
```

```
    Dim swParentComp As SldWorks.Component2  
  
    Set swParentComp = swComp.GetParent  
    If Not swParentComp Is Nothing Then  
        ' This is a component in a subassembly, so set  
        ' subassembly to be visible because subassembly's visibility  
        ' overrides component's visibility; that is, show parent  
        swParentComp.Visible = swComponentVisible  
        ' Show grandparent; that is, recurse up assembly tree  
        ShowParentComp swApp, swModel, swParentComp  
    End If
```

```
End Sub
```

```
Sub ShowComp(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swComp As SldWorks.Component2)
```

```
    Dim vChildArr As Variant  
    Dim vChild As Variant  
    Dim swChildComp As SldWorks.Component2  
  
    ' Show this component  
    swComp.Visible = swComponentVisible  
  
    vChildArr = swComp.GetChildren  
    For Each vChild In vChildArr  
        Set swChildComp = vChild  
        ' Show all children  
        swChildComp.Visible = swComponentVisible  
        ' Show all grandchildren; that is, recurse down assembly tree
```

```

        ShowComp swApp, swModel, swChildComp
    Next vChild

End Sub

Sub HideAllComp(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swComp As SldWorks.Component2)

    Dim vChildArr As Variant
    Dim vChild As Variant
    Dim swChildComp As SldWorks.Component2

    ' Hide this component
    swComp.Visible = swComponentHidden

    vChildArr = swComp.GetChildren
    For Each vChild In vChildArr
        Set swChildComp = vChild
        ' Hide each child component; that is, recurse down assembly tree
        HideAllComp swApp, swModel, swChildComp
    Next vChild

End Sub

Function GetSelCompList(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2) As SldWorks.Component2()

    Dim swSelCompArr() As SldWorks.Component2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swComp As SldWorks.Component2
    Dim nSelCount As Long
    Dim i As Long

    ReDim swSelCompArr(0)
    Set swSelMgr = swModel.SelectionManager
    nSelCount = swSelMgr.GetSelectedObjectCount2(-1): Debug.Assert nSelCount >= 1
    For i = 1 To nSelCount
        Set swComp = swSelMgr.GetSelectedObjectsComponent4(i, -1)
        If Not swComp Is Nothing Then
            Set swSelCompArr(UBound(swSelCompArr)) = swComp
            ReDim Preserve swSelCompArr(UBound(swSelCompArr) + 1)
        End If
    Next i

    Debug.Assert UBound(swSelCompArr) > 0
    ReDim Preserve swSelCompArr(UBound(swSelCompArr) - 1)

    GetSelCompList = swSelCompArr

End Function

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDocExt As SldWorks.ModelDocExtension
    Dim swFeatMgr As SldWorks.FeatureManager
    Dim swAssy As SldWorks.AssemblyDoc
    Dim swConfigMgr As SldWorks.ConfigurationManager
    Dim swConfig As SldWorks.Configuration
    Dim swRootComp As SldWorks.Component2
    Dim swSelCompArr() As SldWorks.Component2
    Dim vSelCompArr As Variant
    Dim vSelComp As Variant
    Dim swSelComp As SldWorks.Component2
    Dim nSelCount As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDocExt = swModel.Extension
    Set swFeatMgr = swModel.FeatureManager
    Set swAssy = swModel
    Set swConfigMgr = swModel.ConfigurationManager
    Set swConfig = swConfigMgr.ActiveConfiguration
    Set swRootComp = swConfig.GetRootComponent3(True)

    ' Temporarily disable FeatureManager design tree updates to increase performance
    swFeatMgr.EnableFeatureTree = False

    swSelCompArr = GetSelCompList(swApp, swModel)
    vSelCompArr = swSelCompArr

    HideAllComp swApp, swModel, swRootComp

```

```
For Each vSelComp In vSelCompArr
    Set swSelComp = vSelComp
    ShowComp swApp, swModel, swSelComp
    ShowParentComp swApp, swModel, swSelComp
Next vSelComp

' Restore selected components
nSelCount = swDocExt.MultiSelect2(vSelCompArr, True, Nothing): Debug.Assert UBound(vSelCompArr) + 1 = nSelCount

swFeatMgr.EnableFeatureTree = True
```

End Sub

Open Advanced Dialog on Document Open Example (C#)

This example shows how to display an advanced dialog box before opening a document.

```
//-----  
// Preconditions: Verify that the specified model document to open exists.  
//  
// Postconditions:  
// 1. Displays the Configure Document dialog box before the model  
//    document opens.  
// 2. Click OK to close the dialog box.  
// 3. Close the document without saving.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
  
partial class SolidWorksMacro  
{  
    public void Main()  
    {  
        ModelDoc2 myDoc = default(ModelDoc2);  
        DocumentSpecification openDocParams = default(DocumentSpecification);  
        openDocParams = (DocumentSpecification)swApp.GetOpenDocSpec("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\motionstudies\\valve_cam.sldasm");  
        openDocParams.DocumentType = (int)swDocumentTypes_e.swDocASSEMBLY;  
        openDocParams.InteractiveAdvancedOpen = true;  
        myDoc = swApp.OpenDoc7(openDocParams);  
    }  
    public SldWorks swApp;  
}
```

Open Advanced Dialog on Document Open Example (VBA)

This example shows how to open an advanced dialog box before opening a document.

```
'-----  
' Preconditions: Verify that the specified model document to open exists.  
,  
' Postconditions:  
' 1. Displays the Configure Document dialog box before the model  
'   document opens.  
' 2. Click OK to close the dialog box.  
' 3. Close the document without saving.  
'-----
```

Option Explicit

Sub main()

Dim swApp As SldWorks.SldWorks

Dim myDoc As SldWorks.ModelDoc2

Dim openDocParams As SldWorks.DocumentSpecification

Set swApp = Application.SldWorks

Set openDocParams = swApp.**GetOpenDocSpec**("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\motionstudies\valve_cam.sldasm")

openDocParams.DocumentType = SwConst.swDocASSEMBLY

openDocParams.**InteractiveAdvancedOpen** = True

Set myDoc = swApp.**OpenDoc7**(openDocParams)

End Sub

Open Advanced Dialog on Document Open Example (VB.NET)

This example shows how to open an advanced dialog box before opening a document.

```
' -----  
' Preconditions: Verify that the specified model document to open exists.  
'  
' Postconditions:  
' 1. Displays the Configure Document dialog box before the model  
'    document opens.  
' 2. Click OK to close the dialog box.  
' 3. Close the document without saving.  
' -----
```

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Imports System

Partial Class SolidWorksMacro

Sub main()

Dim myDoc As ModelDoc2

Dim openDocParams As DocumentSpecification

openDocParams = swApp.**GetOpenDocSpec**("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\motionstudies\valve_cam.sldasm")

openDocParams.DocumentType = swDocumentTypes_e.swDocASSEMBLY

openDocParams.**InteractiveAdvancedOpen** = True

myDoc = swApp.**OpenDoc7**(openDocParams)

End Sub

Public swApp As SldWorks

End Class

Load All External References Example (VBA)

This example shows how to specify to open all external references.

```
'-----  
' Preconditions: Open the Immediate window.  
'  
' Postconditions:  
' 1. Changes your SOLIDWORKS Design load all external references system  
'    setting to 1 (swLoadExternalReferences_e.swLoadExternalReferences_All),  
'    if not already set.  
' 2. Examine the Immediate window.  
'  
' NOTE: This macro changes your SOLIDWORKS Design load external references  
' setting.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim bRet As Boolean
```

```
    Set swApp = Application.SldWorks
```

```
    'Get current external references setting
```

```
    Debug.Print "Load external references current setting = " &  
swApp.GetUserPreferenceIntegerValue(swLoadExternalReferences)
```

```
    ' Open all external references for part, document, and drawing documents
```

```
    bRet = swApp.SetUserPreferenceIntegerValue(swLoadExternalReferences,  
swLoadExternalReferences_e.swLoadExternalReferences_All)
```

```
    ' Get new external references setting
```

```
    Debug.Print "Load external references new setting = " &  
swApp.GetUserPreferenceIntegerValue(swLoadExternalReferences)
```

End Sub

Open and Exit Sketch Example (VBA)

This example shows how to open a sketch, sketch a line, and close the sketch.

```
' -----  
' Preconditions:  
' 1. Open a part.  
' 2. Select a planar face or a sketch.  
'  
' Postconditions:  
' 1. Opens a sketch, sketches a line, and closes the sketch.  
' 2. Examine the graphics area and FeatureManager design tree.  
' -----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSkSeg As SldWorks.SketchSegment  
    Dim swSkLine As SldWorks.SketchLine  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    ' Open a sketch on the selected planar face  
    ' or the selected sketch  
    swModel.InsertSketch2 True  
  
    ' Sketch a line  
    Set swSkSeg = swModel.CreateLine2(0#, 0#, 0#, 0.1, 0.1, 0#)  
    Set swSkLine = swSkSeg  
  
    ' Exit the sketch  
    swModel.InsertSketch2 True  
  
End Sub
```

Open And Save Document to 3DEXPERIENCE Example (C#)

This example shows how to open and save a **3DEXPERIENCE** document with SOLIDWORKS Design connected to the 3DEXPERIENCE platform.

```
//-----  
// Preconditions:  
// 1. Open SOLIDWORKS Design connected to the 3DEXPERIENCE platform.  
// 2. Open a new part.  
// 3. Open the Immediate window.  
// 4. Run the macro to the Break.  
// 5. Close the document in the user interface.  
// 6. Press F5 to re-load the document from  
//    your collaborative space in 3DEXPERIENCE.  
//  
// Postconditions:  
// 1. NonActiveSave.sldprt is saved to a collaborative space  
//    on the 3DEXPERIENCE platform and re-opened.  
// 2. Use 3DEXPERIENCE 3DSearch to find NonActiveSave.sldprt  
//    in your collaborative space.  
// NOTE: Before running this example again, change strFileName to a  
//        unique file name or delete NonActiveSave.sldprt.  
//-----
```

```
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Diagnostics;
```

```
namespace OpenAndSave_CSharp  
{  
    public partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel;  
            ModelDocExtension swModExt;  
            SaveTo3DExperienceOptions swSaveTo3DExp;  
            PLMObjectSpecification swPLMObjectSpecification;  
            DocumentSpecification swDocSpecification;  
            string strPLMId;  
            string strFileName, strComment;  
            bool bStat;  
            int lError = 0;  
            int IWarning = 0;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swModExt = swModel.Extension;
```

```

if (swApp.ApplicationType == (int) swApplicationType_e.swApplicationType_3DEXPERIENCE)
{
    swSaveTo3DExp = (SaveTo3DExperienceOptions)swApp.GetSaveTo3DExperienceOptions();

    strFileName = "NonActiveSave.sldprt";
    strComment = "No comment";
    swSaveTo3DExp.FileName = strFileName;
    bStat = swSaveTo3DExp.SetRevisionComments(strComment);

    bStat = swModExt.SaveTo3DExperience(swSaveTo3DExp, ref lError, ref IWarning);

    strPLMid = swModExt.GetPLMID();
    Debug.Print(swModel.GetPathName() + " " + strPLMid);

    System.Diagnostics.Debugger.Break(); // Close the document in the user interface

    // Re-open the document from 3DEXPERIENCE
    swDocSpecification = (DocumentSpecification)swApp.GetOpenDocSpec("");
    swDocSpecification.DocumentType = (int) swDocumentTypes_e.swDocPART;
    swPLMObjectSpecification =
    (PLMObjectSpecification)swDocSpecification.PLMObjectSpecification;
    swPLMObjectSpecification.PLMID = strPLMid;

    swModel = swApp.OpenDoc7(swDocSpecification);
}

}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}

}

```

Open And Save Document to 3DEXPERIENCE Example (VBA)

This example shows how to open and save a **3DEXPERIENCE** document with SOLIDWORKS Design connected to the 3DEXPERIENCE platform.

```
'-----  
' Preconditions:  
' 1. Open SOLIDWORKS Design connected to the 3DEXPERIENCE platform.  
' 2. Open a new part.  
' 3. Open the Immediate window.  
' 4. Run the macro to the Stop.  
' 5. Close the document in the user interface.  
' 6. Press F5 to re-load the document from your collaborative space  
'    in 3DEXPERIENCE.  
,
```

```
' Postconditions:  
' 1. NonActiveSave.sldprt is saved to a collaborative space  
'    on the 3DEXPERIENCE platform and re-opened.  
' 2. Use 3DEXPERIENCE 3DSearch to find NonActiveSave.sldprt  
'    in your collaborative space.  
,
```

```
' NOTE: Before running this example again, change strFileName to  
'       a unique file name or delete NonActiveSave.sldprt.  
'-----
```

```
Option Explicit  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModExt As SldWorks.ModelDocExtension  
Dim swSaveTo3DExp As SldWorks.SaveTo3DExperienceOptions  
Dim swPLMObjectSpecification As SldWorks.PLMObjectSpecification  
Dim swDocSpecification As SldWorks.DocumentSpecification  
Dim strPartTitle As String  
Dim strPLMid As String  
Dim strFileName, strComment As String  
Dim bStat As Boolean  
Dim IError As Long  
Dim IWarning As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModExt = swModel.Extension  
  
    If swApp.ApplicationType = swApplicationType_3DEXPERIENCE Then  
  
        Set swSaveTo3DExp = swApp.GetSaveTo3DExperienceOptions  
  
        strFileName = "NonActiveSave.sldprt"  
        strComment = "No comment"  
        swSaveTo3DExp.FileName = strFileName  
        bStat = swSaveTo3DExp.SetRevisionComments(strComment)  
  
        bStat = swModExt.SaveTo3DExperience(swSaveTo3DExp, IError, IWarning)  
  
        strPLMid = swModExt.GetPLMID  
        Debug.Print swModel.GetPathName & " " & strPLMid  
  
        Stop 'Close the document in the user interface  
  
        'Re-open the document from 3DEXPERIENCE  
        Set swDocSpecification = swApp.GetOpenDocSpec("")  
        Set swPLMObjectSpecification = swDocSpecification.PLMObjectSpecification  
        swPLMObjectSpecification.PLMID = strPLMid  
  
        Set swModel = swApp.OpenDoc7(swDocSpecification)  
  
    End If  
End Sub
```

Open And Save Document to 3DEXPERIENCE Example (VB.NET)

This example shows how to open and save a **3DEXPERIENCE** document with SOLIDWORKS Design connected to the 3DEXPERIENCE platform.

```
' -----  
' Preconditions:  
' 1. Open SOLIDWORKS Design connected to the 3DEXPERIENCE platform.  
' 2. Open a new part.  
' 3. Open the Immediate window.  
' 4. Run the macro to the Stop.  
' 5. Close the document in the user interface.  
' 6. Press F5 to re-load the document from your  
'    collaborative space In 3DEXPERIENCE.  
'  
' Postconditions:  
' 1. NonActiveSave.sldprt is saved to a collaborative space  
'    on the 3DEXPERIENCE platform and re-opened.  
' 2. Use 3DEXPERIENCE 3DSearch to find NonActiveSave.sldprt  
'    in your collaborative space.  
'  
' NOTE: Before running this example again, change strFileName to  
'       a unique file name or delete NonActiveSave.sldprt.  
' -----
```

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Partial Class SolidWorksMacro

```
    Dim swModel As ModelDoc2  
    Dim swModExt As ModelDocExtension  
    Dim swSaveTo3DExp As SaveTo3DExperienceOptions  
    Dim swPLMObjectSpecification As PLMObjectSpecification  
    Dim swDocSpecification As DocumentSpecification  
    Dim strPartTitle As String  
    Dim strPLMid As String  
    Dim strFileName, strComment As String  
    Dim bStat As Boolean  
    Dim lError As Integer  
    Dim IWarning As Integer  
  
    Sub main()  
  
        swModel = swApp.ActiveDoc  
        swModExt = swModel.Extension
```

```
If swApp.ApplicationType = swApplicationType_e.swApplicationType_3DEXPERIENCE Then
```

```
    swSaveTo3DExp = swApp.GetSaveTo3DExperienceOptions
```

```
    strFileName = "NonActiveSave.sldprt"
```

```
    strComment = "No comment"
```

```
    swSaveTo3DExp.FileName = strFileName
```

```
    bStat = swSaveTo3DExp.SetRevisionComments(strComment)
```

```
    bStat = swModExt.SaveTo3DExperience(swSaveTo3DExp, lError, IWarning)
```

```
    strPLMid = swModExt.GetPLMID
```

```
    Debug.Print(swModel.GetPathName() & " " & strPLMid)
```

```
Stop 'Close the document in the user interface
```

```
'Re-open the document from 3DEXPERIENCE
```

```
swDocSpecification = swApp.GetOpenDocSpec("")
```

```
swDocSpecification.DocumentType = swDocumentTypes_e.swDocPART
```

```
swPLMObjectSpecification = swDocSpecification.PLMObjectSpecification
```

```
swPLMObjectSpecification.PLMID = strPLMid
```

```
swModel = swApp.OpenDoc7(swDocSpecification)
```

```
End If
```

```
End Sub
```

```
''' <summary>
```

```
''' The SldWorks swApp variable is pre-assigned for you.
```

```
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Open Assembly Component Example (C++ COM)

This example shows how to open a component that is part of the current assembly document.

```
// Your code
```

```
...
```

```
LPMODELDOC m_ModelDoc = NULL;
```

```
HRESULT hres = NOERROR;
```

```
// Retrieve IModelDoc pointer
```

```
hres = UserApp->getSWApp()->get_IActiveDoc( &m_ModelDoc );
```

```
if( m_ModelDoc == NULL )
```

```
    return;
```

```
LPASSEMBLYDOC m_AssemblyDoc = NULL;
```

```
hres = m_ModelDoc->QueryInterface(IID_IAssemblyDoc, (LPVOID *)&m_AssemblyDoc);
```

```
// If current document is not an assembly then return
```

```
if(hres!=S_OK || m_AssemblyDoc==NULL)
```

```
    return;
```

```
VARIANT_BOOL selOK;
```

```
hres = m_ModelDoc->SelectByID(_T("Lever-1@LeverAssem"), _T("COMPONENT"), 0, 0, 0, &selOK);
```

```
// Open the component file
```

```
hres = m_AssemblyDoc->OpenCompFile();
```

```
// Make the Lever component the active document
```

```
// Note: Use the correct filename extension
```

```
hres = m_pSOLIDWORKS->IActivateDoc(_T("Lever.SLDPRT"), &m_ModelDoc);
```

```
// Your code
```

```
...
```

```
// Releasing
```

```
m_AssemblyDoc->Release();
```

```
m_ModelDoc->Release();
```

Open Assembly Document Example (C#)

This example shows how to specify how to open an assembly document.

```
//-----
// Preconditions: Verify that the specified assembly document
// to open exists.
//
// Postconditions:
// 1. If the Large Design Review dialog is displayed, click OK
// to close it.
// 2. Opens the specified assembly document and loads
// and displays the specified component.
// 3. Examine the graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DocumentSpecification swDocSpecification = default(DocumentSpecification);
            string[] componentsArray = new string[1];
            object[] components = null;
            string name = null;
            int errors = 0;
            int warnings = 0;

            //Set the specifications
            swDocSpecification = (DocumentSpecification)swApp.GetOpenDocSpec("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\bowl and chute.sldasm");
            componentsArray[0] = "food bowl-1@bowl and chute";
            components = (object[])componentsArray;
            swDocSpecification.ComponentList = components;
            swDocSpecification.Selective = true;
            name = swDocSpecification.FileName;
            swDocSpecification.DocumentType = (int)swDocumentTypes_e.swDocASSEMBLY;
            swDocSpecification.DisplayState = "Default_Display State-1";
            swDocSpecification.UseLightWeightDefault = false;
            swDocSpecification.LightWeight = true;
            swDocSpecification.Silent = true;
            swDocSpecification.IgnoreHiddenComponents = true;

            //Open the assembly document as per the specifications
            swModel = (ModelDoc2)swApp.OpenDoc7(swDocSpecification);
            errors = swDocSpecification.Error;
            warnings = swDocSpecification.Warning;
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Open Assembly Document Example (VBA)

This example shows how to specify how to open an assembly document.

```
' -----  
' Preconditions: Verify that the specified assembly document  
' to open exists.  
'  
' Postconditions:  
' 1. If the Large Design Review dialog is displayed, click OK  
'    to close it.  
' 2. Opens the specified assembly document and loads  
'    and displays the specified component.  
' 3. Examine the graphics area.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
' -----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDocSpecification As SldWorks.DocumentSpecification  
Dim componentsArray(0) As String  
Dim components As Variant  
Dim name As String  
Dim errors As Long  
Dim warnings As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
  
    'Set the specifications  
    Set swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\advdrawings\bowl and chute.sldasm")  
    componentsArray(0) = "food bowl-1@bowl and chute"  
    components = componentsArray  
    swDocSpecification.ComponentList = components  
    swDocSpecification.Selective = True  
    name = swDocSpecification.FileName  
    swDocSpecification.DocumentType = swDocASSEMBLY  
    swDocSpecification.DisplayState = "Default_Display State-1"  
    swDocSpecification.UseLightWeightDefault = False  
    swDocSpecification.LightWeight = True  
    swDocSpecification.Silent = True  
    swDocSpecification.IgnoreHiddenComponents = True  
  
    'Open the assembly document as per the specifications  
    Set swModel = swApp.OpenDoc7(swDocSpecification)  
    errors = swDocSpecification.error  
    warnings = swDocSpecification.Warning  
  
End Sub
```

Open Assembly Document Example (VB.NET)

This example shows how to specify how to open an assembly document.

```
'-----
' Preconditions: Verify that the specified assembly document
' to open exists.
'
' Postconditions:
' 1. If the Large Design Review dialog is displayed, click OK
'    to close it.
' 2. Opens the specified assembly document and loads
'    and displays the specified component.
' 3. Examine the graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swDocSpecification As DocumentSpecification
        Dim componentsArray(0) As String
        Dim components() As Object
        Dim name As String
        Dim errors As Integer
        Dim warnings As Integer

        'Set the specifications
        swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\bowl and chute.sldasm")
        componentsArray(0) = "food bowl-1@bowl and chute"
        components = componentsArray
        swDocSpecification.ComponentList = components
        swDocSpecification.Selective = True
        name = swDocSpecification.FileName
        swDocSpecification.DocumentType = swDocumentTypes_e.swDocASSEMBLY
        swDocSpecification.DisplayState = "Default_Display State-1"
        swDocSpecification.UseLightWeightDefault = False
        swDocSpecification.LightWeight = True
        swDocSpecification.Silent = True
        swDocSpecification.IgnoreHiddenComponents = True

        'Open the assembly document as per the specifications
        swModel = swApp.OpenDoc7(swDocSpecification)
        errors = swDocSpecification.Error
        warnings = swDocSpecification.Warning

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Open an Assembly in Large Design Review Mode Example (C#)

This example shows how to open an assembly in Large Design Review mode.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly to open in Large Design Review mode exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Click OK in the Large Design Review dialog.
// 2. Opens the assembly.
// 3. Creates a section view.
// 4. Creates four snapshots in the DisplayManager:
//    * Home
//    * ASnap
//    * Snap2
//    * Snap3
// 5. Click OK in the Name Snapshot dialog.
// 6. Selects and fully resolves a component.
// 7. Inspect the Immediate window for snapshot information and inspect
//    the graphics area.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        AssemblyDoc Part;
        ModelDoc2 Model;
        Snapshot snap;
        object[] snaps;
        Snapshot snap1;
        Snapshot snap2;
        Snapshot snap3;
        int i;
        bool boolstatus;
        int longstatus;
        int longwarnings;

        public void Main()
        {
            string large_assembly_document;
            large_assembly_document = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\advdrawings\\motor casing.sldasm";
            Part = (AssemblyDoc)swApp.OpenDoc6(large_assembly_document, (int)swDocumentTypes_e.swDocASSEMBLY, (int)sw
OpenDocOptions_e.swOpenDocOptions_ViewOnly, "", ref longstatus, ref longwarnings);
            Model = (ModelDoc2)Part;

            // Create section view
            SectionViewData sViewData = default(SectionViewData);
            sViewData = Model.ModelViewManager.CreateSectionViewData();
            sViewData.FirstPlane = null;
            sViewData.FirstReverseDirection = false;
            sViewData.FirstOffset = -0.00508;
            sViewData.FirstRotationX = 0;
            sViewData.FirstRotationY = 0;
            sViewData.FirstColor = 16711680;
            sViewData.ShowSectionCap = true;
            sViewData.KeepCapColor = true;

            ModelViewManager mvmgr = default(ModelViewManager);
            mvmgr = Model.ModelViewManager;
            boolstatus = mvmgr.CreateSectionView(sViewData);
            Model.ClearSelection2(true);
            Model.ShowNamedView2("Front", 1);
            Model.ShowNamedView2("Dimetric", 9);

            // Add a named snapshot
            snap1 = mvmgr.AddSnapShot("ASnap");
            // Open dialog box to name the next snapshot
        }
    }
}
```

```

snap2 = mvmgr.AddSnapShot("?");
// Add a snapshot with the next default name
snap3 = mvmgr.AddSnapShot("");

snap1.Comment = "<TS> This is a comment for ASnap.";

snaps = (object[])mvmgr.GetSnapShots();

for (i = 0; i <= snaps.GetUpperBound(0); i++)
{
    snap = (SnapShot)snaps[i];
    snap.Activate();
    Debug.Print("Snapshot name: " + snap.Name);
    Debug.Print("      Comment: " + snap.Comment);
    Debug.Print("      ViewID: " + snap.ViewId);
}

// Selects a component
boolstatus = Model.Extension.SelectByID2("motor casing-
1@motor casing", "COMPONENT", 0, 0, 0, false, 0, null, 0);

// Fully resolve only the selected components
Part.SelectiveOpen(true, false);

}

public SldWorks swApp;

}

```


Open an Assembly in Large Design Review Mode Example (VB.NET)

This example shows how to open an assembly in Large Design Review mode.

```
' -----  
' Preconditions:  
' 1. Verify that the specified assembly to open in Large Design Review mode  
'    exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Click OK in the Large Design Review dialog.  
' 2. Opens the assembly.  
' 3. Creates a section view.  
' 4. Creates four snapshots in the DisplayManager:  
'    * Home  
'    * ASnap  
'    * Snap2  
'    * Snap3  
' 5. Click OK in the Name Snapshot dialog.  
' 6. Selects and fully resolves a component.  
' 7. Inspect the Immediate window for snapshot information and inspect  
'    the graphics area.  
'  
' NOTE: Because the assembly is used elsewhere, do not save changes.  
' -----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
    Dim Part As AssemblyDoc  
    Dim snap As Snapshot  
    Dim snaps As Object  
    Dim snap1 As Snapshot  
    Dim snap2 As Snapshot  
    Dim snap3 As Snapshot  
    Dim i As Integer  
    Dim boolstatus As Boolean  
    Dim longwarnings As Integer  
    Dim longstatus As Integer  
  
    Sub main()  
  
        ' Open a large assembly in Large Design Review mode  
        Dim large_assembly_document As String  
        large_assembly_document = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\advdrawings\motor casing.sldasm"  
        Set Part = swApp.OpenDoc6(large_assembly_document, swDocumentTypes_e.swDocASSEMBLY,  
swOpenDocOptions_e.swOpenDocOptions_ViewOnly, "", longstatus, longwarnings)  
  
        ' Create section view  
        Dim sViewData As SectionViewData  
        sViewData = Part.ModelViewManager.CreateSectionViewData()  
        sViewData.FirstPlane = Nothing  
        sViewData.FirstReverseDirection = False  
        sViewData.FirstOffset = -0.00508  
        sViewData.FirstRotationX = 0  
        sViewData.FirstRotationY = 0  
        sViewData.FirstColor = 16711680  
        sViewData.ShowSectionCap = True  
        sViewData.KeepCapColor = True
```

```

Dim mvmgr As ModelViewManager
mvmgr = Part.ModelViewManager
boolstatus = mvmgr.CreateSectionView(sViewData)
Part.ClearSelection2(True)
Part.ShowNamedView2("Front", 1)
Part.ShowNamedView2("Dimetric", 9)

' Add a named snapshot
snap1 = mvmgr.AddSnapShot("ASnap")
' Open dialog box to name the next snapshot
snap2 = mvmgr.AddSnapShot("?")
' Add a snapshot with the next default name
snap3 = mvmgr.AddSnapShot("")

snap1.Comment = "<TS> This is a comment for ASnap."

snaps = mvmgr.GetSnapShots

For i = 0 To UBound(snaps)
    snap = snaps(i)
    snap.Activate()
    Debug.Print("Snapshot name: " & snap.Name)
    Debug.Print("      Comment: " & snap.Comment)
    Debug.Print("      ViewID: " & snap.ViewId)
Next

' Selects a component
Dim model As ModelDoc2
model = Part
boolstatus = model.Extension.SelectByID2("motor casing-1@motor casing", "COMPONENT", 0, 0, 0, False, 0,
Nothing, 0)

' Fully resolve only the selected component
Part.SelectiveOpen (True, False)

End Sub

Public swApp As SldWorks

End Class

```

Open Document Example (C#)

This example shows how to programmatically open a document and set the SOLIDWORKS Design working directory.

```
//-----  
// Preconditions:  
// 1. Verify that the specified document to open exists.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Opens the specified document.  
// 2. Sets the SOLIDWORKS Design working directory to the document directory.  
// 3. Examine the Immediate window.  
//-----  
using Microsoft.VisualBasic;  
using System;  
using System.Collections;  
using System.Collections.Generic;  
using System.Data;  
using System.Diagnostics;  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
namespace SetWorkingDirectory_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        ModelDoc2 doc;  
        int fileerror;  
        int filewarning;  
  
        public void Main()  
        {  
            swApp.Visible = true;  
  
            // Get the current working directory before opening the document  
            Debug.Print("Current working directory is " + swApp.GetCurrentWorkingDirectory());  
  
            doc = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\api\\chair.sldprt", (int)swDocumentTypes_e.swDocPART,  
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref fileerror, ref filewarning);  
  
            // Opening a document with SldWorks::OpenDoc6 does not set the working directory  
            Debug.Print("Current working directory is still " + swApp.GetCurrentWorkingDirectory());  
  
            // Set the working directory to the document directory  
            swApp.SetCurrentWorkingDirectory(doc.GetPathName().Substring(0,  
doc.GetPathName().LastIndexOf("\\\\")));  
  
            Debug.Print("Current working directory is now " + swApp.GetCurrentWorkingDirectory());  
        }  
  
        public SldWorks swApp;  
    }  
}
```

Open Document Example (VBA)

This example shows how to programmatically open a document and set the SOLIDWORKS Design working directory.

```
' -----  
' Preconditions:  
' 1. Verify that the specified document to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified document.  
' 2. Sets the SOLIDWORKS Design working directory to the document directory.  
' 3. Examine the Immediate window.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim doc As SldWorks.ModelDoc2  
Dim fileerror As Long  
Dim filewarning As Long  
Sub main()  
    Set swApp = Application.SldWorks  
    swApp.Visible = True  
  
    ' Get the current working directory before opening the document  
    Debug.Print "Current working directory is " & swApp.GetCurrentWorkingDirectory  
  
    Set doc = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\api\chair.sldprt", swDocPART, swOpenDocOptions_Silent, "", fileerror,  
filewarning)  
  
    ' Opening a document with SldWorks::OpenDoc6 does not set the working directory  
    Debug.Print "Current working directory is still " & swApp.GetCurrentWorkingDirectory  
  
    ' Set the working directory to the document directory  
    swApp.SetCurrentWorkingDirectory (Left(doc.GetPathName, InStrRev(doc.GetPathName, "\")))  
  
    Debug.Print "Current working directory is now " & swApp.GetCurrentWorkingDirectory  
End Sub
```

Open Document Example (VB.NET)

This example shows how to programmatically open a document and set the SOLIDWORKS Design working directory.

```
'-----  
' Preconditions:  
' 1. Verify that the specified document to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified document.  
' 2. Sets the SOLIDWORKS Design working directory to the document directory.  
' 3. Examine the Immediate window.  
'-----  
  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Dim doc As ModelDoc2  
    Dim fileerror As Integer  
    Dim filewarning As Integer  
  
    Sub main()  
  
        swApp.Visible = True  
  
        '  
        Get the current working directory before opening the document  
        Debug.Print("Current working directory is " & swApp.GetCurrentWorkingDirectory)  
  
        doc = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\api\chair.sldprt", swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "",  
fileerror, filewarning)  
  
        ' Opening a document with SldWorks::OpenDoc6  
        does not set the working directory  
        Debug.Print("Current working directory is still " & swApp.GetCurrentWorkingDirectory)  
  
        ' Set the working directory to the  
        document directory  
        swApp.SetCurrentWorkingDirectory(Left(doc.GetPathName, InStrRev(doc.GetPathName, "\")))  
  
        Debug.Print("Current working  
directory is now " & swApp.GetCurrentWorkingDirectory)  
  
    End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Open Document Silently Example (VBA)

This example shows how to open a document silently; that is, without dialog boxes.

```
'-----  
' Preconditions:  
' 1. Verify that the document specified to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified document silently (i.e., no dialogs  
'    are displayed).  
' 2. Examine the Immediate window.  
'-----
```

Option Explicit

Sub main()

```
Const sDocFileName As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cstick.sldprt"  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.modelDoc  
Dim nDocType As Long  
Dim nErrors As Long  
Dim nWarnings As Long
```

```
Set swApp = CreateObject("SldWorks.Application")
```

```
' Determine type of SOLIDWORKS Design file based on file extension
```

```
If InStr(LCase(sDocFileName), ".sldprt") > 0 Then
```

```
    nDocType = swDocPART
```

```
ElseIf InStr(LCase(sDocFileName), ".sldasm") > 0 Then
```

```
    nDocType = swDocASSEMBLY
```

```
ElseIf InStr(LCase(sDocFileName), ".slddrw") > 0 Then
```

```
    nDocType = swDocDRAWING
```

```
Else
```

```
    ' Probably not a SOLIDWORKS file
```

```
    nDocType = swDocNONE
```

```
    ' So cannot open the file
```

```
    Exit Sub
```

```
End If
```

```
Set swModel = swApp.OpenDoc6(sDocFileName, nDocType, swOpenDocOptions_Silent, "", nErrors, nWarnings)
```

```
Debug.Print "File = " & swModel.GetPathName
```

```
Debug.Print " Error (0 = no errors) = " & nErrors
```

```
Debug.Print " Warnings (2 = document is read-only) = " & nWarnings
```

End Sub

Open Drawing Document Read-only Example (VBA)

This example shows how to open a drawing document as read-only.

'-----

' Preconditions: Verify that the specified drawing
' document exists.

'

' Postconditions: Opens the specified drawing document
' as read-only.

'-----

Option Explicit

Dim swApp As SldWorks.SldWorks

Dim swModel As SldWorks.ModelDoc2

Dim swSelMgr As SldWorks.SelectionMgr

Dim swDocSpecification As SldWorks.DocumentSpecification

Dim sName As String

Dim longstatus As Long, longwarnings As Long

Sub main()

Set swApp = Application.SldWorks

Set swDocSpecification = swApp.**GetOpenDocSpec**("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\AutoCAD\7550-021.slddrw")

sName = swDocSpecification.**FileName**

swDocSpecification.**DocumentType** = swDocDRAWING

swDocSpecification.**ReadOnly** = True

swDocSpecification.**Silent** = False

Set swModel = swApp.**OpenDoc7**(swDocSpecification)

longstatus = swDocSpecification.**Error**

longwarnings = swDocSpecification.**Warning**

End Sub

Open File Example (VBA)

This example shows how to prompt the user for the name of the file to open. This example also shows how to set up filters by file types.

```
'-----  
' Preconditions: Open the Immediate window.  
'  
' Postconditions:  
' 1. Displays the File to Attach dialog and lists the file  
'    types specified in the filter.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Public swApp As SldWorks.SldWorks  
Public swModel As SldWorks.ModelDoc2  
  
Sub main()  
    Dim Filter As String  
    Dim fileName As String  
    Dim fileConfig As String  
    Dim fileDispName As String  
    Dim fileDisplayState As String  
    Dim fileOptions As Long  
  
    Set swApp = Application.SldWorks  
    ' The Filter string has three filters  
    ' associated with it; note the use of  
    ' the | character between filters  
    Filter = "SOLIDWORKS Files (*.sldprt; *.sldasm; *.slddrw)|*.sldprt;*.sldasm;*.slddrw|Filter name  
(*.*)|*.fil|All Files (*.*)|*.*|"  
    fileName = swApp.GetOpenFileName2("File to Attach", "", Filter, fileOptions, fileConfig, fileDispName,  
fileDisplayState)  
    Debug.Print fileName  
  
End Sub
```

Open Part from Assembly Example (VBA)

This example shows how to open a part document from an assembly document.

```
' -----  
' Preconditions:  
' 1. Open an assembly.  
' 2. Select a vertex, face, or an edge of a component.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the part document to which the selected vertex,  
'    face, or edge belongs.  
' 2. Examine the Immediate window and graphics area.  
' -----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModelDocExt As SldWorks.ModelDocExtension  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swCompEnt As SldWorks.Entity  
    Dim swPartFeat As SldWorks.Feature  
    Dim swComp As SldWorks.Component2  
    Dim swCompModel As SldWorks.ModelDoc2  
    Dim swPart As SldWorks.PartDoc  
    Dim swPartEnt As SldWorks.Entity  
    Dim swConfigMgr As SldWorks.ConfigurationManager  
    Dim swCompModelConfig As Configuration  
    Dim nRetVal As Long  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
  
    Set swCompEnt = swSelMgr.GetSelectedObject6(1, 0)  
    Set swComp = swSelMgr.GetSelectedObjectsComponent3(1, 0)  
    Set swCompModel = swComp.GetModelDoc  
    Set swConfigMgr = swCompModel.ConfigurationManager  
    Set swCompModelConfig = swConfigMgr.ActiveConfiguration  
    Set swModelDocExt = swCompModel.Extension  
    Set swPartEnt = swModelDocExt.GetCorrespondingEntity(swCompEnt)  
    Set swCompModel = swApp.ActivateDoc2(swCompModel.GetPathName, True, nRetVal): Debug.Assert 0 = nRetVal  
  
    bRet = swPartEnt.Select4(False, Nothing)  
  
    Debug.Print "File = " + swModel.GetPathName  
    Debug.Print "  Component      = " + swComp.Name2 + " <" + swComp.ReferencedConfiguration + ">" + " [" +  
    swComp.GetPathName + "]"  
    Debug.Print "  Model          = " + swCompModel.GetPathName + " <" + swCompModelConfig.Name + ">"  
  
End Sub
```

Open Specified Sheet in Drawing Document Example (C#)

This example shows how to open a specific sheet when programmatically opening a drawing document.

```
//-----
// Preconditions:
// 1. Verify that the specified drawing to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified sheet in the specified drawing
//    document as view-only.
// 2. Examine the drawing and Immediate window.
//
// NOTE: Because this drawing document is used
// elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace Macro1.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DocumentSpecification swDocSpecification = default(DocumentSpecification);
            string sName = null;
            long longstatus = 0;
            long longwarnings = 0;

            // Drawing document path and name
            swDocSpecification =
(DocumentSpecification)swApp.GetOpenDocSpec("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\advdrawings\\foodprocessor.slddrw");
            sName = swDocSpecification.FileName;

            // Sheet name
            swDocSpecification.SheetName = "Sheet2";
            swDocSpecification.DocumentType = (int)swDocumentTypes_e.swDocDRAWING;
            swDocSpecification.ReadOnly = true;
            swDocSpecification.Silent = false;

            // Open the specified sheet in the specified drawing document
            swModel = swApp.OpenDoc7(swDocSpecification);
            longstatus = swDocSpecification.Error;
            longwarnings = swDocSpecification.Warning;

            Debug.Print ("Name of active sheet? " + swDocSpecification.SheetName);
            Debug.Print ("Drawing read-only? " + swDocSpecification.ReadOnly);

        }
        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Open Specified Sheet in Drawing Document Example (VBA)

This example shows how to open a specific sheet when programmatically opening a drawing document.

```
' -----  
' Preconditions:  
' 1. Verify that the specified drawing to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified sheet in the specified drawing  
'    document as view-only.  
' 2. Examine the drawing and Immediate window.  
'  
' NOTE: Because this drawing document is used  
' elsewhere, do not save changes.  
' -----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDocSpecification As SldWorks.DocumentSpecification  
Dim sName As String  
Dim longstatus As Long, longwarnings As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
  
    ' Drawing document path and name  
    Set swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\advdrawings\foodprocessor.slddrw")  
    sName = swDocSpecification.FileName  
  
    ' Sheet name  
    swDocSpecification.SheetName = "Sheet2"  
    swDocSpecification.DocumentType = swDocDRAWING  
    swDocSpecification.ReadOnly = True  
    swDocSpecification.Silent = False  
  
    ' Open the specified sheet in the specified drawing document  
    Set swModel = swApp.OpenDoc7(swDocSpecification)  
    longstatus = swDocSpecification.Error  
    longwarnings = swDocSpecification.Warning  
  
    Debug.Print "Name of active sheet? " & swDocSpecification.SheetName  
    Debug.Print "Drawing read-only? " & swDocSpecification.ReadOnly  
  
End Sub
```

Open Specified Sheet in Drawing Document Example (VB.NET)

This example shows how to open a specific sheet when programmatically opening a drawing document.

```
' -----  
' Preconditions:  
' 1. Verify that the specified drawing to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified sheet in the specified drawing  
'    document as view-only.  
' 2. Examine the drawing and Immediate window.  
'  
' NOTE: Because this drawing document is used  
' elsewhere, do not save changes.  
' -----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
    Public Sub main()  
        Dim swModel As ModelDoc2  
        Dim swDocSpecification As DocumentSpecification  
        Dim sName As String  
        Dim longstatus As Long, longwarnings As Long  
  
        ' Drawing document path and name  
        swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\advdrawings\foodprocessor.slddrw")  
        sName = swDocSpecification.FileName  
        ' Sheet name  
        swDocSpecification.SheetName = "Sheet2"  
        swDocSpecification.DocumentType = swDocumentTypes_e.swDocDRAWING  
        swDocSpecification.ReadOnly = True  
        swDocSpecification.Silent = False  
  
        ' Open the specified sheet in the specified drawing document  
        swModel = swApp.OpenDoc7(swDocSpecification)  
        longstatus = swDocSpecification.Error  
        longwarnings = swDocSpecification.Warning  
  
        Debug.Print("Name of active sheet? " & swDocSpecification.SheetName)  
        Debug.Print("Drawing read-only? " & swDocSpecification.ReadOnly)  
  
    End Sub  
  
    ''' <summary>  
    ''' The SldWorks swApp variable is pre-assigned for you.  
    ''' </summary>  
    Public swApp As SldWorks  
  
End Class
```

Override Layer Color for Area Hatch Example (VBA)

This example shows how to set the color of an area hatch to override the color a layer.

```
'-----  
' Preconditions:  
' 1. Verify that the specified drawing exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the drawing.  
' 2. Hatches a face in the drawing.  
' 3. Sets the color of the hatch to override the color the layer.  
' 4. Inspect the Immediate window.  
' 5. Click outside the drawing view and inspect the hatch.  
'  
' NOTE: Because the drawing is used elsewhere, do not save changes.  
'-----
```

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swView As SldWorks.View  
Dim swSketch As SldWorks.Sketch  
Dim vSketchHatch As Variant  
Dim swSketchHatch As SldWorks.SketchHatch  
Dim swFace As SldWorks.Face2  
Dim vID As Variant  
Dim i As Long  
Dim bRet As Boolean  
Dim longstatus As Long, longwarnings As Long  
  
Option Explicit  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
  
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\api\box.slddrw", swDocumentTypes_e.swDocDRAWING, 0, "", longstatus, longwarnings)  
    swApp.ActivateDoc2 "box - Sheet1", False, longstatus  
    Set swModel = swApp.ActiveDoc  
  
    bRet = swModel.ActivateView("Drawing View1")  
  
    bRet = swModel.Extension.SelectByID2("", "FACE", 0.246685509728212, 0.236217308689246, 1.49999999999864E-02,  
True, 0, Nothing, 0)  
    swModel.InsertHatchedFace  
    swModel.ClearSelection2 True  
  
    bRet = swModel.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)  
  
    Set swSelMgr = swModel.SelectionManager  
    Set swView = swSelMgr.GetSelectedObject6(1, -1)  
    Set swSketch = swView.GetSketch  
    swModel.EditSketch  
  
    swModel.ClearSelection2 True  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print " " & swView.Name  
  
    vSketchHatch = swSketch.GetSketchHatches  
  
    If Not IsEmpty(vSketchHatch) Then  
  
        For i = 0 To UBound(vSketchHatch)  
  
            Set swSketchHatch = vSketchHatch(i)  
            Set swFace = swSketchHatch.GetFace  
  
            bRet = swSketchHatch.Select4(True, Nothing)  
            vID = swSketchHatch.GetID
```

```

Debug.Print "      Hatch ID(" & i & "): [" & vID(0) & "," & vID(1) & "]"
Debug.Print "      Angle: " & swSketchHatch.Angle
Debug.Print "      Color: " & swSketchHatch.Color
Debug.Print "      Layer: " & swSketchHatch.Layer
Debug.Print "      Layer override (bitmask)? " & swSketchHatch.LayerOverride
Debug.Print "      Pattern: " & swSketchHatch.Pattern
Debug.Print "      Scale: " & swSketchHatch.Scale2
Debug.Print "      Solid fill? " & swSketchHatch.SolidFill
Debug.Print " "

```

```

'Override layer color; change to red
Dim overRide As Long
swSketchHatch.Color = RGB(255, 0, 0)
overRide = swSketchHatch.LayerOverride
' Extract the first bit and get its value
If (overRide And 1) Then
    Debug.Print "      Hatch color overrode layer color!"
Else
    Debug.Print "      Hatch color did not override layer color!"
End If

```

```

Next i

```

```

End If

```

```

End Sub

```

Pack and Go an Assembly (C#)

This example shows how to get the names of the path and files of an assembly document, add a prefix and suffix to the names, and save the files to a different path using the Pack and Go interface.

```
//-----  
// Preconditions:  
// 1. Specified assembly exists.  
// 2. The folder, c:\temp, exists.  
// 3. Open the Immediate window.  
// 4. Run the macro.  
//  
// Postconditions:  
// 1. Prints names of the current path and filenames  
//    of the assembly documents to Immediate window.  
// 2. Prints names of the default path and filenames to which to  
//    save assembly documents to the Immediate window.  
// 3. Specifies the Pack and Go destination folder.  
// 4. Specifies that all files get saved to the root directory of the  
//    Pack and Go destination folder.  
// 5. Adds prefix and suffix to user-named filenames.  
// 6. Prints names of user-specified path and user-named filenames to  
//    Immediate window.  
// 7. Creates user-named files in user-specified path using Pack and Go.  
// 8. Examine c:\temp to verify.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
using System.Runtime.InteropServices;  
  
namespace PackAndGoCSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModelDoc = default(ModelDoc2);  
            ModelDocExtension swModelDocExt = default(ModelDocExtension);  
            PackAndGo swPackAndGo = default(PackAndGo);  
            string openFile = null;  
            bool status = false;  
            int warnings = 0;  
            int errors = 0;  
            int i = 0;  
            int namesCount = 0;  
            string myPath = null;  
            int[] statuses = null;  
  
            // Open assembly  
            openFile = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\advdrawings\\handle.sldasm";  
            swModelDoc = (ModelDoc2)swApp.OpenDoc6(openFile, (int)swDocumentTypes_e.swDocASSEMBLY,  
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);  
            swModelDocExt = (ModelDocExtension)swModelDoc.Extension;  
  
            // Get Pack and Go object  
            Debug.Print("Pack and Go");  
            swPackAndGo = (PackAndGo)swModelDocExt.GetPackAndGo();  
  
            // Get number of documents in assembly  
            namesCount = swPackAndGo.GetDocumentNamesCount();  
            Debug.Print("    Number of model documents: " + namesCount);  
        }  
    }  
}
```

```

// Include any drawings, SOLIDWORKS Simulation results, and SOLIDWORKS Toolbox components
swPackAndGo.IncludeDrawings = true;
Debug.Print(" Include drawings: " + swPackAndGo.IncludeDrawings);
swPackAndGo.IncludeSimulationResults = true;
Debug.Print(" Include SOLIDWORKS Simulation results: " + swPackAndGo.IncludeSimulationResults);
swPackAndGo.IncludeToolboxComponents = true;
Debug.Print(" Include SOLIDWORKS Toolbox components: " + swPackAndGo.IncludeToolboxComponents);

// Get current paths and filenames of the assembly's documents
object fileNames;
object[] pgFileNames = new object[namesCount - 1];
status = swPackAndGo.GetDocumentNames(out fileNames);
pgFileNames = (object[])fileNames;

Debug.Print("");
Debug.Print(" Current path and filenames: ");
if ((pgFileNames != null))
{
    for (i = 0; i <= pgFileNames.GetUpperBound(0); i++)
    {
        Debug.Print(" The path and filename is: " + pgFileNames[i]);
    }
}

// Get current save-to paths and filenames of the assembly's documents
object pgFileStatus;
status = swPackAndGo.GetDocumentSaveToNames(out fileNames, out pgFileStatus);
pgFileNames = (object[])fileNames;
Debug.Print("");
Debug.Print(" Current default save-to filenames: ");
if ((pgFileNames != null))
{
    for (i = 0; i <= pgFileNames.GetUpperBound(0); i++)
    {
        Debug.Print(" The path and filename is: " + pgFileNames[i]);
    }
}

// Set folder where to save the files
myPath = "C:\\temp\\";
status = swPackAndGo.SetSaveToName(true, myPath);

// Flatten the Pack and Go folder structure; save all files to the root directory
swPackAndGo.FlattenToSingleFolder = true;

// Add a prefix and suffix to the filenames
swPackAndGo.AddPrefix = "SW_";
swPackAndGo.AddSuffix = "_PackAndGo";

// Verify document paths and filenames after adding prefix and suffix
object getFileNames;
object getDocumentStatus;
string[] pgGetFileNames = new string[namesCount - 1];

status = swPackAndGo.GetDocumentSaveToNames(out getFileNames, out getDocumentStatus);
pgGetFileNames = (string[])getFileNames;
Debug.Print("");
Debug.Print(" My Pack and Go path and filenames after adding prefix and suffix: ");
for (i = 0; i <= namesCount - 1; i++)
{
    Debug.Print(" My path and filename is: " + pgGetFileNames[i]);
}

// Pack and Go
statuses = (int[])swModelDocExt.SavePackAndGo(swPackAndGo);
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

```

```
        public SldWorks swApp;  
    }  
}
```

Pack and Go an Assembly (VBA)

This example shows how to get the names of the path and files of an assembly document, add a prefix and suffix to the names, and save the files to a different path using the Pack and Go interface.

```
' -----  
' Preconditions:  
' 1. Specified assembly exists.  
' 2. The folder, c:\temp, exists.  
' 3. Open the Immediate window.  
' 4. Run the macro.  
'  
' Postconditions:  
' 1. Prints names of the current path and filenames  
'   of the assembly documents to the Immediate window.  
' 2. Prints names of the default path and filenames to which to  
'   save assembly documents to Immediate window.  
' 3. Specifies the Pack and Go destination folder.  
' 4. Specifies that all files get saved to the root directory of the  
'   Pack and Go destination folder.  
' 5. Adds prefix and suffix to user-named filenames.  
' 6. Prints names of user-specified path and user-named filenames to  
'   Immediate window.  
' 7. Creates user-named files in user-specified path using Pack and Go.  
' 8. Examine c:\temp to verify.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModelDoc As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swPackAndGo As SldWorks.PackAndGo  
Dim openFile As String  
Dim pgFileNames As Variant  
Dim pgFileStatus As Variant  
Dim pgGetFileNames As Variant  
Dim pgDocumentStatus As Variant  
Dim status As Boolean  
Dim warnings As Long  
Dim errors As Long  
Dim i As Long  
Dim namesCount As Long  
Dim myPath As String  
Dim statuses As Variant  
  
Sub main()  
    Set swApp = Application.SldWorks  
    ' Open assembly  
    openFile = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\advdrawings\handle.sldasm"  
    Set swModelDoc = swApp.OpenDoc6(openFile, swDocASSEMBLY, swOpenDocOptions_Silent, "",  
errors, warnings)  
    Set swModelDocExt = swModelDoc.Extension  
    ' Get Pack and Go object  
    Debug.Print "Pack and Go"  
    Set swPackAndGo = swModelDocExt.GetPackAndGo  
    ' Get number of documents in assembly  
    namesCount = swPackAndGo.GetDocumentNamesCount  
    Debug.Print "  Number of model documents: " & namesCount  
    ' Include any drawings, SOLIDWORKS Simulation results, and SOLIDWORKS Toolbox components  
    swPackAndGo.IncludeDrawings = True  
    Debug.Print "  Include drawings: " & swPackAndGo.IncludeDrawings  
    swPackAndGo.IncludeSimulationResults = True
```

```

Debug.Print " Include SOLIDWORKS Simulation results: " &
swPackAndGo.IncludeSimulationResults
swPackAndGo.IncludeToolboxComponents = True
Debug.Print " Include SOLIDWORKS Toolbox components: " &
swPackAndGo.IncludeToolboxComponents

' Get current paths and filenames of the assembly's documents
status = swPackAndGo.GetDocumentNames(pgFileNames)
Debug.Print ""
Debug.Print " Current path and filenames: "
If (Not (IsEmpty(pgFileNames))) Then
    For i = 0 To UBound(pgFileNames)
        Debug.Print " The path and filename is: " & pgFileNames(i)
    Next i
End If

' Get current save-to paths and filenames of the assembly's documents
status = swPackAndGo.GetDocumentSaveToNames(pgFileNames, pgFileStatus)
Debug.Print ""
Debug.Print " Current default save-to filenames: "
If (Not (IsEmpty(pgFileNames))) Then
    For i = 0 To UBound(pgFileNames)
        Debug.Print " The path and filename is: " & pgFileNames(i)
    Next i
End If

' Set folder where to save the files
myPath = "C:\temp\"
status = swPackAndGo.SetSaveToName(True, myPath)

' Flatten the Pack and Go folder structure; save all files to the root directory
swPackAndGo.FlattenToSingleFolder = True

' Add a prefix and suffix to the new Pack and Go filenames
swPackAndGo.AddPrefix = "SW_"
swPackAndGo.AddSuffix = "_PackAndGo"

' Verify document paths and filenames after adding prefix and suffix
ReDim pgGetFileNames(namesCount - 1)
ReDim pgDocumentStatus(namesCount - 1)
status = swPackAndGo.GetDocumentSaveToNames(pgGetFileNames, pgDocumentStatus)
Debug.Print ""
Debug.Print " My Pack and Go path and filenames after adding prefix and suffix: "
For i = 0 To (namesCount - 1)
    Debug.Print " My path and filename is: " & pgGetFileNames(i)
Next i

' Pack and Go
statuses = swModelDocExt.SavePackAndGo(swPackAndGo)
End Sub

```

Pack and Go an Assembly (VB.NET)

This example shows how to get the names of the path and files of an assembly document, add a prefix and suffix to the names, and save the files to a different path using the Pack and Go interface.

```
' -----  
' Preconditions:  
' 1. Specified assembly exists.  
' 2. The folder, c:\temp, exists.  
' 3. Open the Immediate window.  
' 4. Run the macro.  
'  
' Postconditions:  
' 1. Prints names of the current path and filenames  
'   of the assembly documents to the Immediate window.  
' 2. Prints names of the default path and filenames to which to  
'   save assembly documents to Immediate window.  
' 3. Specifies the Pack and Go destination folder.  
' 4. Specifies that all files get saved to the root directory of the  
'   Pack and Go destination folder.  
' 5. Adds prefix and suffix to user-named filenames.  
' 6. Prints names of user-specified path and user-named filenames to  
'   Immediate window.  
' 7. Creates user-named files in user-specified path using Pack and Go.  
' 8. Examine c:\temp to verify.  
' -----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub Main()
```

```
        Dim swModelDoc As ModelDoc2  
        Dim swModelDocExt As ModelDocExtension  
        Dim swPackAndGo As PackAndGo  
        Dim openFile As String  
        Dim pgFileNames As Object = Nothing  
        Dim pgFileStatus As Object = Nothing  
        Dim pgSetFileNames() As String  
        Dim pgGetFileNames As Object  
        Dim pgDocumentStatus As Object  
        Dim status As Boolean  
        Dim warnings As Integer  
        Dim errors As Integer  
        Dim i As Integer  
        Dim namesCount As Integer  
        Dim myPath As String  
        Dim statuses As Object
```

```
        ' Open assembly
```

```
        openFile = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\advdrawings\handle.sldasm"  
        swModelDoc = swApp.OpenDoc6(openFile, swDocumentTypes_e.swDocASSEMBLY,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
        swModelDocExt = swModelDoc.Extension
```

```
        ' Get Pack and Go object
```

```
        Debug.Print("Pack and Go")  
        swPackAndGo = swModelDocExt.GetPackAndGo
```

```
        ' Get number of documents in assembly
```

```
        namesCount = swPackAndGo.GetDocumentNamesCount  
        Debug.Print(" Number of model documents: " & namesCount)
```

```

' Include any drawings, SOLIDWORKS Simulation results, and SOLIDWORKS Toolbox components
swPackAndGo.IncludeDrawings = True
Debug.Print(" Include drawings: " & swPackAndGo.IncludeDrawings)
swPackAndGo.IncludeSimulationResults = True
Debug.Print(" Include SOLIDWORKS Simulation results: " &
swPackAndGo.IncludeSimulationResults)
swPackAndGo.IncludeToolboxComponents = True
Debug.Print " Include SOLIDWORKS Toolbox components: " & swPackAndGo.IncludeToolboxComponents

' Get current paths and filenames of the assembly's documents
status = swPackAndGo.GetDocumentNames(pgFileNames)
Debug.Print("")
Debug.Print(" Current path and filenames: ")
If Not pgFileNames Is Nothing Then
    For i = 0 To UBound(pgFileNames)
        Debug.Print(" The path and filename is: " & pgFileNames(i))
    Next i
End If

' Get current save-to paths and filenames of the assembly's documents
status = swPackAndGo.GetDocumentSaveToNames(pgFileNames, pgFileStatus)
Debug.Print("")
Debug.Print(" Current default save-to filenames: ")
If Not pgFileNames Is Nothing Then
    For i = 0 To UBound(pgFileNames)
        Debug.Print(" The path and filename is: " & pgFileNames(i))
    Next i
End If

' Set folder where to save the files
myPath = "C:\temp\"
status = swPackAndGo.SetSaveToName(True, myPath)

' Flatten the Pack and Go folder structure; save all files to the root directory
swPackAndGo.FlattenToSingleFolder = True

' Add a prefix and suffix to the filenames
swPackAndGo.AddPrefix = "SW_"
swPackAndGo.AddSuffix = "_PackAndGo"

' Verify document paths and filenames after adding prefix and suffix
ReDim pgGetFileNames(namesCount - 1)
ReDim pgDocumentStatus(namesCount - 1)
status = swPackAndGo.GetDocumentSaveToNames(pgGetFileNames, pgDocumentStatus)
Debug.Print("")
Debug.Print(" My Pack and Go path and filenames after adding prefix and suffix: ")
For i = 0 To (namesCount - 1)
    Debug.Print(" My path and filename is: " & pgGetFileNames(i))
Next i

' Pack and Go
statuses = swModelDocExt.SavePackAndGo(swPackAndGo)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Pack and Go Part and Linked Equation Example (C#)

This example shows how to determine if a part document includes any equations and whether those equations are linked files. The example also shows how to add the part document and any linked equations to Pack and Go.

```
//-----
// Preconditions:
// 1. Verify that the specified part and equation documents
//    exist.
// 2. Verify that c:\temp exists.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Adds the part and linked equation documents
//    to Pack and Go and copies the files to
//    c:\temp.
// 3. To verify, examine c:\temp.
//
// NOTE: Because the model is used elsewhere,
// do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace EquationsPackAndGoCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            EquationMgr swEqnMgr = default(EquationMgr);
            PackAndGo swPackAndGo = default(PackAndGo);
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            int i = 0;
            int nCount = 0;
            bool eqnLinked = false;
            object fileNames = null;
            object[] pgFileNames = null;
            bool status = false;
            int[] statuses = null;
            string myPath = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\microphonehousing.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.s
wOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" ");

            // Get Equation manager object
            swEqnMgr = (EquationMgr)swModel.GetEquationMgr();

            // List the equations, get whether they're linked
            // to files, make sure any equations are up to date,
            // and get the paths where they're stored
            nCount = swEqnMgr.GetCount();
            for (i = 0; i <= nCount - 1; i++)
            {
                Debug.Print(" Equation #" + i + " = " + swEqnMgr.get_Equation(i));

                eqnLinked = swEqnMgr.LinkToFile;
                Debug.Print(" Equation linked to file? " + eqnLinked);

                if (eqnLinked)
                {
                    //Make sure equations are up to date
                    swEqnMgr.UpdateValuesFromExternalEquationFile();

                    //Get path and name of linked equation file
                    Debug.Print(" Path and filename of linked equation: " + swEqnMgr.FilePath);
                }
            }
        }
    }
}
```

```

        Debug.Print(" ");

        // Get Pack and Go object
        Debug.Print("    Pack and Go");
        swPackAndGo = swModelDocExt.GetPackAndGo();

        // Get current paths and names of the documents
        status = swPackAndGo.GetDocumentNames(out fileNames);
        pgFileNames = (object[])fileNames;
        Debug.Print("    Add these SOLIDWORKS Design files to Pack and Go: ");
        if ((pgFileNames != null))
        {
            for (i = 0; i <= pgFileNames.GetUpperBound(0); i++)
            {
                Debug.Print("        The file to pack up is: " + pgFileNames[i]);
            }
        }

        // Set document paths and names for Pack and Go
        status = swPackAndGo.SetDocumentSaveToNames(pgFileNames);

        // Override path where to save documents
        myPath = "c:\\temp\\";
        status = swPackAndGo.SetSaveToName(true, myPath);

        // Pack and Go both SOLIDWORKS Design and non-SOLIDWORKS-Design files
        statuses = (int[])swModelDocExt.SavePackAndGo(swPackAndGo);
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>

    public SldWorks swApp;
}
}

```

Pack and Go Part and Linked Equation Example(VBA)

This example shows how to determine if a part document includes any equations and whether those equations are linked files. The example also shows how to add the part document and any linked equations to Pack and Go.

```
'-----  
' Preconditions:  
' 1. Verify that the specified part and equation documents  
'    exist.  
' 2. Verify that c:\temp exists.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified part document.  
' 2. Adds the part and linked equation documents  
'    to Pack and Go and copies the files to  
'    c:\temp.  
' 3. To verify, examine c:\temp.  
'  
' NOTE: Because the model is used elsewhere,  
' do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swEqnMgr As SldWorks.EquationMgr  
Dim swPackAndGo As SldWorks.PackAndGo  
Dim fileName As String  
Dim errors As Long  
Dim warnings As Long  
Dim i As Long  
Dim nCount As Long  
Dim eqnLinked As Boolean  
Dim pgFileNames As Variant  
Dim status As Boolean  
Dim statuses As Variant  
Dim myPath As String
```

Sub main()

```
Set swApp = Application.SldWorks  
  
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\microphonehousing.sldprt"  
Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)  
Set swModelDocExt = swModel.Extension  
Debug.Print "File = " & swModel.GetPathName  
Debug.Print " "  
  
' Get Equation manager object  
Set swEqnMgr = swModel.GetEquationMgr  
  
' List the equations, get whether they're linked  
' to files, make sure any equations are up to date,  
' and get the paths where they're stored  
nCount = swEqnMgr.GetCount  
For i = 0 To nCount - 1  
    Debug.Print " Equation #" & i & " = " & swEqnMgr.Equation(i)  
  
    eqnLinked = swEqnMgr.LinkToFile  
    Debug.Print " Equation linked to file? " & eqnLinked  
  
    If eqnLinked Then  
        'Make sure equations are up to date  
        swEqnMgr.UpdateValuesFromExternalEquationFile  
  
        'Get path and name of linked equation file  
        Debug.Print " Path and file name of linked equation: " & swEqnMgr.FilePath  
    End If  
Next i
```

```

Debug.Print " "

' Get Pack and Go object
Debug.Print " Pack and Go"
Set swPackAndGo = swModelDocExt.GetPackAndGo

' Get current paths and names of the documents
status = swPackAndGo.GetDocumentNames(pgFileNames)
Debug.Print " Add these SOLIDWORKS Design files to Pack and Go: "
If (Not (IsEmpty(pgFileNames))) Then
For i = 0 To UBound(pgFileNames)
    Debug.Print " The file to pack up is: " & pgFileNames(i)
Next i
End If

' Set document paths and names for Pack and Go
status = swPackAndGo.SetDocumentSaveToNames(pgFileNames)

' Override path where to save documents
myPath = "c:\temp\"
status = swPackAndGo.SetSaveToName(True, myPath)

' Pack and Go both SOLIDWORKS Design and non-SOLIDWORKS-Design files
statuses = swModelDocExt.SavePackAndGo(swPackAndGo)

End Sub

```

Pack and Go Part and Linked Equation Example (VB.NET)

This example shows how to determine if a part document includes any equations and whether those equations are linked files. The example also shows how to add the part document and any linked equations to Pack and Go.

```
'-----
' Preconditions:
' 1. Verify that the specified part and equation documents
'    exist.
' 2. Verify that c:\temp exists.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Adds the part and linked equation documents
'    to Pack and Go and copies the files to
'    c:\temp.
' 3. To verify, examine c:\temp.
'
' NOTE: Because the model is used elsewhere,
' do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swEqnMgr As EquationMgr
        Dim swPackAndGo As PackAndGo
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim i As Integer
        Dim nCount As Integer
        Dim eqnLinked As Boolean
        Dim pgFileNames As Object = Nothing
        Dim status As Boolean
        Dim statuses As Object
        Dim myPath As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\microphonehousing.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" ")

        ' Get Equation manager object
        swEqnMgr = swModel.GetEquationMgr

        ' List the equations, get whether they're linked
        ' to files, make sure any equations are up to date,
        ' and get the paths where they're stored
        nCount = swEqnMgr.GetCount
        For i = 0 To nCount - 1
            Debug.Print(" Equation #" & i & " = " & swEqnMgr.Equation(i))

            eqnLinked = swEqnMgr.LinkToFile
            Debug.Print(" Equation linked to file? " & eqnLinked)

            If eqnLinked Then

                'Make sure equations are up to date
                swEqnMgr.UpdateValuesFromExternalEquationFile()

                'Get path and name of linked equation file
                Debug.Print(" Path and file name of linked equation: " & swEqnMgr.FilePath)

            End If
        Next i

        Debug.Print(" ")

        ' Get Pack and Go object
        Debug.Print(" Pack and Go")
        swPackAndGo = swModelDocExt.GetPackAndGo
```

```

' Get current paths and names of the documents
status = swPackAndGo.GetDocumentNames(pgFileNames)
Debug.Print("  Add these SOLIDWORKS Design files to Pack and Go: ")
If Not pgFileNames Is Nothing Then
    For i = 0 To UBound(pgFileNames)
        Debug.Print("    The file to pack up is: " & pgFileNames(i))
    Next i
End If

' Set document paths and names for Pack and Go
status = swPackAndGo.SetDocumentSaveToNames(pgFileNames)

' Override path where to save documents
myPath = "c:\temp\"
status = swPackAndGo.SetSaveToName(True, myPath)

' Pack and Go both SOLIDWORKS Design and non-SOLIDWORKS-Design files
statuses = swModelDocExt.SavePackAndGo(swPackAndGo)

```

End Sub

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Place Note Behind Drawing Sheet Example (C#)

This example shows how to place a note behind a drawing sheet.

```
//-----
// Preconditions:
// 1. Verify that the specified drawing file to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Places the selected note, 2012-sm in the drawing template,
//    behind the drawing sheet.
// 2. To verify:
//    a. Examine the Immediate window.
//    b. Right-click the drawing and click
//       Edit Sheet Format.
//    c. Right-click 2012-sm and examine the
//       the short-cut menu to verify that Display
//       Note Behind Sheet is selected.
//    d. Exit drawing sheet edit mode.
//
// NOTE: Because this drawing is used elsewhere, do not
// save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace NoteBehindSheetCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            DrawingDoc swDrawing = default(DrawingDoc);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Note swNote = default(Note);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            // Open drawing
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2024\\samples\\tutorial\\api\\2012-
sm.sldrw";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_
e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swDrawing = (DrawingDoc)swModel;

            // Put drawing template and sheet in edit mode
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Sheet1", "SHEET", 0.0399580396732789, 0.20594194865811, 0, false, 0,
null, 0);

            swDrawing.EditTemplate();
            swDrawing.EditSheet2();

            swModel.ClearSelection2(true);

            // Select note to place behind the sheet
            status = swModelDocExt.SelectByID2("DetailItem3@Sheet Format1", "NOTE", 0.155548914819136, 0.017885845974
329, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swNote = (Note)swSelectionMgr.GetSelectedObject6(1, -1);
            swNote.BehindSheet = true;
            Debug.Print("Was the selected note placed behind the sheet? " + status);
        }
    }
}
```

```
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>
```

```
public SldWorks swApp;
```

```
    }  
}
```

Place Note Behind Drawing Sheet Example (VBA)

This example shows how to place a note behind a drawing sheet.

```
' -----  
' Preconditions:  
' 1. Verify that the specified drawing file to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Places the selected note, 2012-sm in the drawing template,  
'    behind the drawing sheet.  
' 2. To verify:  
'    a. Examine the Immediate window.  
'    b. Right-click the drawing and click  
'       Edit Sheet Format.  
'    c. Right-click 2012-sm and examine the  
'       the short-cut menu to verify that Display  
'       Note Behind Sheet is selected.  
'    d. Exit drawing sheet edit mode.  
'  
' NOTE: Because this drawing is used elsewhere, do not  
' save changes.  
' -----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swDrawing As SldWorks.DrawingDoc  
Dim swSelectionMgr As SldWorks.SelectionMgr  
Dim swNote As SldWorks.Note  
Dim fileName As String  
Dim status As Boolean  
Dim errors As Long, warnings As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
  
    ' Open drawing  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2024\samples\tutorial\api\2012-sm.slddrw"  
    Set swModel = swApp.OpenDoc6(fileName, swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)  
    Set swDrawing = swModel  
  
    ' Put drawing template and sheet in edit mode  
    Set swModelDocExt = swModel.Extension  
    status = swModelDocExt.SelectByID2("Sheet1", "SHEET", 3.99580396732789E-02, 0.20594194865811, 0, False, 0,  
Nothing, 0)  
    swDrawing.EditTemplate  
    swDrawing.EditSheet2  
  
    swModel.ClearSelection2 True  
  
    ' Select note to place behind the sheet  
    status = swModelDocExt.SelectByID2("DetailItem3@Sheet Format1", "NOTE", 0.155548914819136, 0.017885845974329, 0,  
False, 0, Nothing, 0)  
    Set swSelectionMgr = swModel.SelectionManager  
    Set swNote = swSelectionMgr.GetSelectedObject6(1, -1)  
    swNote.BehindSheet = True  
    Debug.Print ("Was the selected note placed behind the sheet? " & status)  
  
End Sub
```

Place Note Behind Drawing Sheet Example (VB.NET)

This example shows how to place a note behind a drawing sheet.

```
'-----  
' Preconditions:  
' 1. Verify that the specified drawing file to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Places the selected note, 2012-sm in the drawing template,  
'    behind the drawing sheet.  
' 2. To verify:  
'    a. Examine the Immediate window.  
'    b. Right-click the drawing and click  
'       Edit Sheet Format.  
'    c. Right-click 2012-sm and examine the  
'       the short-cut menu to verify that Display  
'       Note Behind Sheet is selected.  
'    d. Exit drawing sheet edit mode.  
'  
' NOTE: Because this drawing is used elsewhere, do not  
' save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
    Public Sub Main()  
  
        Dim swModel As ModelDoc2  
        Dim swModelDocExt As ModelDocExtension  
        Dim swDrawing As DrawingDoc  
        Dim swSelectionMgr As SelectionMgr  
        Dim swNote As Note  
        Dim fileName As String  
        Dim status As Boolean  
        Dim errors As Integer  
        Dim warnings As Integer  
  
        ' Open drawing  
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2024\samples\tutorial\api\2012-sm.slddrw"  
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent  
, "", errors, warnings)  
        swDrawing = swModel  
  
        ' Put drawing template and sheet in edit mode  
        swModelDocExt = swModel.Extension  
        status = swModelDocExt.SelectByID2("Sheet1", "SHEET", 0.0399580396732789, 0.20594194865811, 0, False, 0, Nothing  
ing, 0)  
        swDrawing.EditTemplate()  
        swDrawing.EditSheet2()  
  
        swModel.ClearSelection2(True)  
  
        ' Select note to place behind the sheet  
        status = swModelDocExt.SelectByID2("DetailItem3@Sheet Format1", "NOTE", 0.155548914819136, 0.017885845974329,  
0, False, 0, Nothing, 0)  
        swSelectionMgr = swModel.SelectionManager  
        swNote = swSelectionMgr.GetSelectedObject6(1, -1)  
        swNote.BehindSheet = True  
        Debug.Print("Was the selected note placed behind the sheet? " & status)  
  
    End Sub
```

```
''' <summary>
```

```
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Position Document Window Example (VBA)

This example shows how to position the current document window.

'-----

' Assumes part is already set to a IModelDoc2 object

' Get the IModelView object

Set ModelView = Part.**ActiveView**

' Position the window

ModelView.**FrameLeft** = 100

ModelView.**FrameWidth** = 500

ModelView.**FrameTop** = 100

ModelView.**FrameHeight** = 400

ModelView.**FrameState** = swWindowNormal

Preset_and_Reset_Template_and_Sheet_Parameters_for_New_Drawing_Documents_Example_VB

(Topic)

SOLIDWORKS API Help

Preset and Reset Template and Sheet Parameters for New Drawing Documents Example (VBA)

This example shows how to preset the template and sheet parameters for a new drawing document so that the Sheet Format/Size dialog is not displayed when opening a new drawing document. This example also shows how to reset the template and sheet parameters back to the defaults.

```
'-----  
' Preconditions: Verify that the specified drawing document  
' template exists.  
'  
' Postconditions:  
' 1. Presets the drawing document template and sheet parameters so that  
'   the Sheet Format/Size dialog is not displayed.  
' 2. Click File > New > Drawing > OK to open a new drawing document  
'   to verify step 1.  
' 3. Press F5.  
' 4. Resets the drawing template and sheet parameters so that  
'   Sheet Format/Size dialog is displayed when opening a  
'   new drawing document.  
' 5. Open another new drawing document to verify step 4.  
' 6. Close both drawings without saving them.  
'-----
```

Option Explicit

Dim swApp As SldWorks.SldWorks

Sub main()

Set swApp = Application.SldWorks

' Preset new drawing template and sheet parameters

swApp.PresetNewDrawingParameters "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\sheetformat\portrait.slddrt", True, 0, 0

Stop

' Interactively open a new drawing document

' Sheet Format/Size dialog is not displayed

' Press F5

' Reset the drawing template and sheet parameters

' to display the Sheet Format/Size dialog the next

' time a drawing document is opened

swApp.ResetPresetDrawingParameters

End Sub

Print Drawing and Save as PDF Example (VBA)

This example shows how to print a drawing document to your default printer and save it as a PDF file.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\2012-sm.slddrw.  
' 2. Verify that C:\temp exists.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the name of your default printer.  
' 2. Sets the page orientation to landscape.  
' 3. Prints 2012-sm.slddrw to your default printer.  
' 4. Saves 2012-sm.slddrw as C:\temp\2012-sm.pdf.  
' 5. Examine the Immediate window, your default printer,  
'    and C:\temp.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim Model As ModelDoc2  
  
Sub main()  
    Set swApp = CreateObject("SldWorks.Application")  
    Set Model = swApp.ActiveDoc  
    Debug.Print Model.Printer  
  
    Dim ps As PageSetup  
    Set ps = Model.PageSetup  
    ps.Orientation = 2 ' 1=Portrait '2 = Landscape  
  
    Dim pageArray(3) As Long  
    pageArray(0) = 1  
    pageArray(1) = 1  
    pageArray(2) = 3  
    pageArray(3) = 3  
  
    Dim vPageArray As Variant  
    vPageArray = pageArray  
  
    Dim copies As Long  
    copies = 1  
    Dim collate As Boolean  
    collate = True  
  
    ' Print to default printer  
    Model.Extension.PrintOut2 vPageArray, copies, collate, "", ""  
  
    Dim e As Long  
    Dim w As Long  
  
    ' Save to PDF  
    Debug.Print Model.SaveAs4("C:\temp\2012-sm.pdf", 0, 0, e, w)  
  
End Sub
```

Print Drawing Example (C#)

This example shows how to print the active drawing document.

```
//-----  
// Preconditions:  
// 1. Verify that the specified drawing document exists.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Prints the drawing to your default printer using the  
//    specified page setup and print specification.  
// 2. Inspect the Immediate window.  
//  
// NOTE: Because the model is used elsewhere, do not save changes.  
//-----  
using Microsoft.VisualBasic;  
using System;  
using System.Collections;  
using System.Collections.Generic;  
using System.Data;  
using System.Diagnostics;  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
namespace PrintSpecification_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        ModelDoc2 swModel;  
        ModelDocExtension swModelDocExt;  
        PrintSpecification printSpec;  
        PageSetup swPageSetup;  
        string drawing;  
        int errors;  
        int warnings;  
        string footerTextNumber;  
        string footerTextCount;  
        string footerText;  
  
        public void Main()  
        {  
            drawing = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\advdrawings\\foodprocessor.slddrw";  
            swModel = swApp.OpenDoc6(drawing, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOp  
ptions_Silent, "", ref errors, ref warnings);  
  
            //Set up page's footer text  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
            swModelDocExt.UsePageSetup = (int)swPageSetupInUse_e.swPageSetupInUse_Document;  
            swPageSetup = (PageSetup)swModel.PageSetup;  
            footerTextNumber = swPageSetup.GetHeaderFooterString((int)swStandardHeaderFooterPageSetupTexts_e.swHeader  
FooterText_PageNumber);  
            footerTextCount = swPageSetup.GetHeaderFooterString((int)swStandardHeaderFooterPageSetupTexts_e.swHeaderF  
ooterText_PageCount);  
            footerText = (footerTextNumber + " of " + footerTextCount);  
            swPageSetup.CenterFooter = footerText;  
  
            //Create a print specification  
            printSpec = (PrintSpecification)swModelDocExt.GetPrintSpecification();  
            printSpec.ConvertToHighQuality = true;  
            printSpec.AddPrintRange(1, 1);  
            printSpec.FromScale = 1.5;  
            printSpec.ToScale = 0.5;  
            printSpec.ScaleMethod = (int)swPrintSelectionScaleFactor_e.swPrintSelection;  
            printSpec.PrinterQueue = "";  
            printSpec.PrintToFile = false;  
  
            Debug.Print("Printing specifications:");  
            Debug.Print("  Collate: " + printSpec.Collate);  
            Debug.Print("  Convert to high quality: " + printSpec.ConvertToHighQuality);  
            Debug.Print("  Current sheet: " + printSpec.CurrentSheet);  
            Debug.Print("  From scale: " + printSpec.FromScale);  
            Debug.Print("  To scale: " + printSpec.ToScale);  
            Debug.Print("  Number of copies: " + printSpec.NumberOfCopies);  
            Debug.Print("  Print background: " + printSpec.PrintBackground);  
            Debug.Print("  Print cross hatch on out-of-date views: " + printSpec.PrintCrossHatchOnOutOfDateViews);  
            Debug.Print("  Printer queue: " + printSpec.PrinterQueue);  
            Debug.Print("  Print white items black: " + printSpec.PrintWhiteItemsBlack);  
            Debug.Print("  Selection as defined in swPrintSelectionScaleFactor_e: " + printSpec.ScaleMethod);  
            Debug.Print("  Total sheet count: " + printSpec.SheetCount);  
        }  
    }  
}
```

```
Debug.Print(" X origin: " + printSpec.XOrigin);  
Debug.Print(" Y origin: " + printSpec.YOrigin);  
  
//Print the drawing to your default printer  
swModelDocExt.PrintOut4("", "", printSpec);  
  
printSpec.RestoreDefaults();  
printSpec.ResetPrintRange();  
}
```

```
public SldWorks swApp;
```

```
}
```

```
}
```

Print Drawing Example (VBA)

This example shows how to print the active drawing document.

```
'-----  
' Preconditions:  
' 1. Verify that the specified drawing exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Prints the drawing to your default printer using the  
'    specified page setup and print specification.  
' 2. Inspect the Immediate window.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
'-----
```

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swPageSetup As SldWorks.PageSetup  
Dim printSpec As SldWorks.PrintSpecification  
Dim drawing As String  
Dim errors As Long  
Dim warnings As Long  
Dim footerTextNumber As String  
Dim footerTextCount As String  
Dim footerText as String  
  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
    drawing = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\advdrawings\foodprocessor.slddrw"  
    Set swModel = swApp.OpenDoc6(drawing, swDocDRAWING, swOpenDocOptions_Silent, "", errors,  
warnings)  
  
    'Set up page's footer text  
    Set swModelDocExt = swModel.Extension  
    swModelDocExt.UsePageSetup = swPageSetupInUse_e.swPageSetupInUse_Document  
    Set swPageSetup = swModel.PageSetup  
    footerTextNumber =  
swPageSetup.GetHeaderFooterString(swStandardHeaderFooterPageSetupTexts_e.swHeaderFooterText_PageNumber)  
    footerTextCount =  
swPageSetup.GetHeaderFooterString(swStandardHeaderFooterPageSetupTexts_e.swHeaderFooterText_PageCount)  
    footerText = (footerTextNumber & " of " & footerTextCount)  
    swPageSetup.CenterFooter = footerText  
  
    'Create a print specification  
    Set printSpec = swModelDocExt.GetPrintSpecification  
    printSpec.ConvertToHighQuality = True  
    printSpec.AddPrintRange 1, 1  
    printSpec.FromScale = 1.5  
    printSpec.ToScale = 0.5  
    printSpec.ScaleMethod = swPrintSelection  
    printSpec.PrinterQueue = ""  
    printSpec.PrintToFile = False  
  
    Debug.Print "Printing specifications:"  
    Debug.Print "  Collate: " & printSpec.Collate  
    Debug.Print "  Convert to high quality: " & printSpec.ConvertToHighQuality  
    Debug.Print "  Current sheet: " & printSpec.CurrentSheet  
    Debug.Print "  From scale: " & printSpec.FromScale  
    Debug.Print "  To scale: " & printSpec.ToScale  
    Debug.Print "  Number of copies: " & printSpec.NumberOfCopies  
    Debug.Print "  Print background: " & printSpec.PrintBackground  
    Debug.Print "  Print cross hatch on out-of-date views: " &  
printSpec.PrintCrossHatchOnOutOfDateViews
```

```
Debug.Print "   Printer queue: " & printSpec.PrinterQueue
Debug.Print "   File: " & printSpec.PrintFile
Debug.Print "   Print white items black: " & printSpec.PrintWhiteItemsBlack
Debug.Print "   Selection as defined in swPrintSelectionScaleFactor_e: " & printSpec.ScaleMethod
Debug.Print "   Total sheet count: " & printSpec.SheetCount
Debug.Print "   X origin: " & printSpec.XOrigin
Debug.Print "   Y origin: " & printSpec.YOrigin
```

```
' Print the drawing to your default printer
swModelDocExt.PrintOut4 "", "", printSpec
```

```
printSpec.RestoreDefaults
printSpec.ResetPrintRange
```

```
End Sub
```

Print Drawing Example (VB.NET)

This example shows how to print the active drawing document.

```
'-----  
' Preconditions:  
' 1. Verify that the specified drawing exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Prints the drawing to your default printer using the  
'    specified page setup and print specification.  
' 2. Inspect the Immediate window.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
    Dim swModel As ModelDoc2  
    Dim swModelDocExt As ModelDocExtension  
    Dim swPageSetup As PageSetup  
    Dim printSpec As PrintSpecification  
    Dim drawing As String  
    Dim errors As Integer  
    Dim warnings As Integer  
    Dim footerTextNumber As String  
    Dim footerTextCount As String  
    Dim footerText As String  
  
    Sub main()  
  
        drawing = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\advdrawings\foodprocessor.slddrw"  
        swModel = swApp.OpenDoc6(drawing, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent,  
        "", errors, warnings)  
  
        'Set up page's footer text  
        swModelDocExt = swModel.Extension  
        swModelDocExt.UsePageSetup = swPageSetupInUse_e.swPageSetupInUse_Document  
        swPageSetup = swModel.PageSetup  
        footerTextNumber = swPageSetup.GetHeaderFooterString(swStandardHeaderFooterPageSetupTexts_e.swHeaderFooterTex  
t_PageNumber)  
        footerTextCount = swPageSetup.GetHeaderFooterString(swStandardHeaderFooterPageSetupTexts_e.swHeaderFooterText  
_PageCount)  
        footerText = (footerTextNumber & " of " & footerTextCount)  
        swPageSetup.CenterFooter = footerText  
  
        'Create a print specification  
        printSpec = swModelDocExt.GetPrintSpecification  
        printSpec.ConvertToHighQuality = True  
        printSpec.AddPrintRange(1, 1)  
        printSpec.FromScale = 1.5  
        printSpec.ToScale = 0.5  
        printSpec.ScaleMethod = swPrintSelectionScaleFactor_e.swPrintSelection  
        printSpec.PrinterQueue = ""  
        printSpec.PrintToFile = False  
  
        Debug.Print("Printing specifications:")  
        Debug.Print("  Collate: " & printSpec.Collate)  
        Debug.Print("  Convert to high quality: " & printSpec.ConvertToHighQuality)  
        Debug.Print("  Current sheet: " & printSpec.CurrentSheet)  
        Debug.Print("  From scale: " & printSpec.FromScale)  
        Debug.Print("  To scale: " & printSpec.ToScale)  
        Debug.Print("  Number of copies: " & printSpec.NumberOfCopies)  
        Debug.Print("  Print background: " & printSpec.PrintBackground)  
        Debug.Print("  Print cross hatch on out-of-date views: " & printSpec.PrintCrossHatchOnOutOfDateViews)  
        Debug.Print("  Printer queue: " & printSpec.PrinterQueue)  
        Debug.Print("  Print white items black: " & printSpec.PrintWhiteItemsBlack)  
        Debug.Print("  Selection as defined in swPrintSelectionScaleFactor_e: " & printSpec.ScaleMethod)
```

```
Debug.Print("  Total sheet count: " & printSpec.SheetCount)
Debug.Print("  X origin: " & printSpec.XOrigin)
Debug.Print("  Y origin: " & printSpec.YOrigin)
```

```
'Print the drawing to your default printer
swModelDocExt.PrintOut4("", "", printSpec)
```

```
printSpec.RestoreDefaults()
printSpec.ResetPrintRange()
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Print Drawing Document to File Example (VBA)

This example shows how to print a drawing document to a print file.

```
'-----
' Preconditions:
' 1. Copy public_documents\samples\tutorial\api\2012-sm.slddrw
'    and 2012-sm.sldprt to C:\temp.
' 2. Open C:\temp\2012-sm.slddrw.
' 3. Substitute the name of your printer for GoToMyPc.
'
' Postconditions:
' 1. Prints 2012-sm0.prn to C:\temp.
' 2. Examine C:\temp.
'-----
Option Explicit

Sub main()

    ' Printer
    Const SWPrinter As String = "GoToMyPC"

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDocExt As SldWorks.ModelDocExtension
    Dim swDraw As SldWorks.DrawingDoc
    Dim PathName As String
    Dim PrintFileName As String
    Dim nPrintSheets(1) As Long
    Dim vPrintSheets As Variant
    Dim DefPrinter As String
    Dim i As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDocExt = swModel.Extension
    Set swDraw = swModel

    ' Strip off SOLIDWORKS Design file extension
    PathName = swModel.GetPathName
    PathName = Left(PathName, Len(PathName) - 7)

    For i = 0 To swDraw.GetSheetCount - 1
        ' Generate print file name based on number of sheets
        ' For example, because 2012-sm.slddrw contains
        ' one sheet, then the print file name is 2012-sm0.prn
        PrintFileName = PathName & Trim(Str(i)) & ".prn"

        ' Print out one sheet at a time
        nPrintSheets(0) = i + 1
        vPrintSheets = nPrintSheets
        swDocExt.PrintOut2 (vPrintSheets), 1, False, SWPrinter, PrintFileName
    Next i

End Sub
```

Process Body Example (C#)

This example shows how create to a new part with an imported body from the processed body of the original part.

```
//-----  
// Preconditions:  
// 1. Verify that the specified part document template  
//    exists.  
// 2. Open a part containing only one solid body.  
// 3. Open the Immediate window.  
// 4. Run the macro.  
//  
// Postconditions:  
// 1. Creates a new part with an imported body  
//    from the processed body of the original part.  
// 2. Examine the Immediate window.  
//  
// NOTE: Differences are best seen in wireframe and with parts  
// that contain curved, circular, or both types of faces.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace ProcessbodyCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
  
            // 1 radian = 180°/p = 57.295779513° or approximately 57.3°  
            const double RadPerDeg = 1.0 / 57.3;  
            const double MaxUAngle = 1.0 * RadPerDeg;  
            const double MaxVAngle = 1.0 * RadPerDeg;  
  
            ModelDoc2 swModel = default(ModelDoc2);  
            Body2 swBody = default(Body2);  
            Body2 swProcBody = default(Body2);  
            PartDoc swPart = default(PartDoc);  
            PartDoc swNewPart = default(PartDoc);  
            Feature swFeat = default(Feature);  
            object[] vBodies = null;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swPart = (PartDoc)swModel;  
            vBodies = (object[])swPart.GetBodies2((int)swBodyType_e.swSolidBody, false);  
            swBody = (Body2)vBodies[0];  
            swProcBody = (Body2)swBody.GetProcessedBody2(MaxUAngle, MaxVAngle);  
            swNewPart = (PartDoc)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2013\\templates\\part.prt  
dot", 0, 0, 0);  
            swFeat = (Feature)swNewPart.CreateFeatureFromBody3(swProcBody, false, (int)swCreateFeatureBodyOpts_e.swCr  
eateFeatureBodyCheck);  
            Debug.Print("File = " + swModel.GetPathName());  
            Debug.Print("  Body faces      = " + swBody.GetFaceCount());  
            Debug.Print("  Processed body faces = " + swProcBody.GetFaceCount());  
  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
    }  
}
```

Process Body Example (VBA)

This example shows how create to a new part with an imported body from the processed body of the original part.

```
' -----  
' Preconditions:  
' 1. Verify that the specified part document template  
'    exists.  
' 2. Open a part containing only one solid body.  
' 3. Open the Immediate window.  
' 4. Run the macro.  
'  
' Postconditions:  
' 1. Creates a new part with an imported body  
'    from the processed body of the original part.  
' 2. Examine the Immediate window.  
'  
' NOTE: Differences are best seen in wireframe and with parts  
' that contain curved, circular, or both types of faces.  
' -----
```

Option Explicit

Sub main()

```
    ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°  
    Const RadPerDeg As Double = 1# / 57.3  
    Const MaxUAngle As Double = 1# * RadPerDeg  
    Const MaxVAngle As Double = 1# * RadPerDeg  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swBody As SldWorks.Body2  
    Dim swProcBody As SldWorks.Body2  
    Dim swPart As SldWorks.PartDoc  
    Dim swNewPart As SldWorks.PartDoc  
    Dim swFeat As SldWorks.Feature  
    Dim vBodies As Variant  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
    vBodies = swPart.GetBodies2(swSolidBody, False)  
    Set swBody = vBodies(0)  
    Set swProcBody = swBody.GetProcessedBody2(MaxUAngle, MaxVAngle)  
  
    Set swNewPart = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2013\templates\part.prtdot", 0, 0, 0)  
    Set swFeat = swNewPart.CreateFeatureFromBody3(swProcBody, False, swCreateFeatureBodyCheck)  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print " Body faces = " & swBody.GetFaceCount  
    Debug.Print " Processed body faces = " & swProcBody.GetFaceCount
```

End Sub

Process Body Example (VB.NET)

This example shows how create to a new part with an imported body from the processed body of the original part.

```
'-----
' Preconditions:
' 1. Verify that the specified part document template
'    exists.
' 2. Open a part containing only one solid body.
' 3. Open the Immediate window.
' 4. Run the macro.
'
' Postconditions:
' 1. Creates a new part with an imported body
'    from the processed body of the original part.
' 2. Examine the Immediate window.
'
' NOTE: Differences are best seen in wireframe and with parts
'       that contain curved, circular, or both types of faces.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        ' 1 radian = 180°/p = 57.295779513° or approximately 57.3
        Const RadPerDeg As Double = 1.0# / 57.3
        Const MaxUAngle As Double = 1.0# * RadPerDeg
        Const MaxVAngle As Double = 1.0# * RadPerDeg

        Dim swModel As ModelDoc2
        Dim swBody As Body2
        Dim swProcBody As Body2
        Dim swPart As PartDoc
        Dim swNewPart As PartDoc
        Dim swFeat As Feature
        Dim vBodies As Object

        swModel = swApp.ActiveDoc
        swPart = swModel
        vBodies = swPart.GetBodies2(swBodyType_e.swSolidBody, False)
        swBody = vBodies(0)
        swProcBody = swBody.GetProcessedBody2(MaxUAngle, MaxVAngle)
        swNewPart = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2013\templates\part.prtdot", 0, 0, 0)
        swFeat = swNewPart.CreateFeatureFromBody3(swProcBody, False, swCreateFeatureBodyOpts_e.swCreateFeatureBodyChe
ck)

        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print("  Body faces      = " & swBody.GetFaceCount)
        Debug.Print("  Processed body faces = " & swProcBody.GetFaceCount)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Publish Text and Custom Properties from Theme to MBD 3D PDF Example (C#)

This example shows how to publish text and custom properties from a theme to SOLIDWORKS MBD 3D PDF.

```
//-----
// Preconditions:
// 1. Verify that:
//    * specified part,
//    * SOLIDWORKS MBD 3D PDF theme, and
//    * c:\temp exist.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part.
// 2. Gets the MBD3DPdfData object.
// 3. Sets the path and file name for the SOLIDWORKS MBD 3D PDF.
// 4. Sets to display SOLIDWORKS MBD 3D PDF after publishing.
// 5. Sets the theme for the SOLIDWORKS MBD 3D PDF.
// 6. Sets standard views for the SOLIDWORKS MBD 3D PDF.
// 7. Creates and sets custom views for the SOLIDWORKS MBD 3D PDF.
// 8. Gets the text and custom properties from the specified
//    theme for the SOLIDWORKS MBD 3D PDF and sets the custom
//    property names.
// 9. Creates the SOLIDWORKS MBD 3D PDF.
// 10. Examine the Immediate window and c:\temp\MBDPart1.PDF.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            MBD3DPdfData swMBDPdfData = default(MBD3DPdfData);
            TextAndCustomProperty swTextAndCustomProperty = default(TextAndCustomProperty);
            string fileName = null;
            object standardViews = null;
            int[] viewIDs = new int[3];
            string[] viewNames = new string[2];
            object[] textAndCustomProperties = null;
            int nbrTextAndCustomProperties = 0;
            int status = 0;
            int errors = 0;
            int warnings = 0;
            string custProp = null;
            int i = 0;

            fileName = "C:\\temp\\box.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.s
wOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Get MBD3DPdfData object
            swMBDPdfData = (MBD3DPdfData)swModelDocExt.GetMBD3DPdfData();

            //Set path and file name for SOLIDWORKS MBD 3D PDF
            swMBDPdfData.FilePath = "c:\\temp\\MBDPart1.PDF";

            //Display SOLIDWORKS MBD 3D PDF
            swMBDPdfData.ViewPdfAfterSaving = true;

            //Set SOLIDWORKS MBD 3D PDF theme
            swMBDPdfData.ThemeName = "C:\\Program Files\\SolidWorks Corp\\SOLIDWORKS\\data\\themes\\simple part (a4,
portrait)\\theme.xml";

            //Set standard views for SOLIDWORKS MBD 3D PDF
            viewIDs[0] = (int)swStandardViews_e.swFrontView;
            viewIDs[1] = (int)swStandardViews_e.swTopView;
            viewIDs[2] = (int)swStandardViews_e.swDimetricView;
            standardViews = viewIDs;
            swMBDPdfData.SetStandardViews(standardViews);
        }
    }
}
```

```

//Create and set custom views for SOLIDWORKS MBD 3D PDF
swModel.NameView("CustomView1");
swModel.NameView("CustomView2");
viewNames[0] = "CustomView1";
viewNames[1] = "CustomView2";
swMBDPdfData.SetMoreViews(viewNames);

//Get text and custom properties from theme for SOLIDWORKS MBD
//3D PDF
textAndCustomProperties = (object[])swMBDPdfData.GetTextAndCustomProperties();
if ((textAndCustomProperties != null))
{
    nbrTextAndCustomProperties = textAndCustomProperties.GetUpperBound(0);
    Debug.Print("Text and custom properties from theme:");
    for (i = 0; i <= nbrTextAndCustomProperties; i++)
    {
        swTextAndCustomProperty = (TextAndCustomProperty)textAndCustomProperties[i];
        if (swTextAndCustomProperty.IsCustomProperty == false)
        {
            Debug.Print(" Text:");
            Debug.Print(" Description: " + swTextAndCustomProperty.Description);
            Debug.Print(" Field name: " + swTextAndCustomProperty.Name);
        }
        else
        {
            Debug.Print(" Custom property:");
            custProp = "CustomProperty " + Convert.ToString(i);
            swTextAndCustomProperty.CustomPropertyName = custProp;
            Debug.Print(" Custom property name: " + swTextAndCustomProperty.CustomPropertyName);
            Debug.Print(" Description: " + swTextAndCustomProperty.Description);
            Debug.Print(" Field name: " + swTextAndCustomProperty.Name);
            Debug.Print(" Value: " + swTextAndCustomProperty.Value);
        }
    }
}
else
{
    Debug.Print("No text or custom properties in theme.");
}

//Create SOLIDWORKS MBD 3D PDF
status = swModelDocExt.PublishTo3DPDF(swMBDPdfData);
Debug.Print("Status of publishing part to SOLIDWORKS MBD 3D PDF (0 = success): " + status);
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Publish Text and Custom Properties from Theme to MBD 3D PDF Example (VBA)

This example shows how to publish text and custom properties from a theme to SOLIDWORKS MBD 3D PDF.

```
'-----  
' Preconditions:  
' 1. Verify that:  
'    * specified part,  
'    * SOLIDWORKS MBD 3D PDF theme, and  
'    * c:\temp exist.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified part.  
' 2. Gets the MBD3DPdfData object.  
' 3. Sets the path and file name for the SOLIDWORKS MBD 3D PDF.  
' 4. Sets to display SOLIDWORKS MBD 3D PDF after publishing.  
' 5. Sets the theme for the SOLIDWORKS MBD 3D PDF.  
' 6. Sets standard views for the SOLIDWORKS MBD 3D PDF.  
' 7. Creates and sets custom views for the SOLIDWORKS MBD 3D PDF.  
' 8. Gets the text and custom properties from the specified  
'    theme for the SOLIDWORKS MBD 3D PDF and sets the custom  
'    property names.  
' 9. Creates the SOLIDWORKS MBD 3D PDF.  
' 10. Examine the Immediate window and c:\temp\MBDPart1.PDF.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swMBDPdfData As SldWorks.MBD3DPdfData  
Dim swTextAndCustomProperty As SldWorks.TextAndCustomProperty  
Dim fileName As String  
Dim standardViews As Variant  
Dim viewIDs(2) As Long  
Dim viewNames(1) As String  
Dim textAndCustomProperties As Variant  
Dim nbrTextAndCustomProperties As Long  
Dim status As Long  
Dim errors As Long  
Dim warnings As Long  
Dim custProp As String  
Dim i As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\temp\box.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent,  
    "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
  
    'Get MBD3DPdfData object  
    Set swMBDPdfData = swModelDocExt.GetMBD3DPdfData  
  
    'Set path and file name for SOLIDWORKS MBD 3D PDF  
    swMBDPdfData.filePath = "c:\temp\MBDPart1.PDF"  
  
    'Display SOLIDWORKS MBD 3D PDF  
    swMBDPdfData.ViewPdfAfterSaving = True  
  
    'Set SOLIDWORKS MBD 3D PDF theme  
    swMBDPdfData.ThemeName = "C:\Program Files\SolidWorks Corp\SOLIDWORKS\data\themes\simple part (a4,  
    portrait)\theme.xml"  
  
    'Set standard views for SOLIDWORKS MBD 3D PDF  
    viewIDs(0) = swStandardViews_e.swFrontView  
    viewIDs(1) = swStandardViews_e.swTopView  
    viewIDs(2) = swStandardViews_e.swDimetricView
```

```

standardViews = viewIDs
swMBDPdfData.SetStandardViews (standardViews)

'Create and set custom views for SOLIDWORKS MBD 3D PDF
swModel.NameView ("CustomView1")
swModel.NameView ("CustomView2")
viewNames(0) = "CustomView1"
viewNames(1) = "CustomView2"
swMBDPdfData.SetMoreViews (viewNames)

'Get text and custom properties from theme for SOLIDWORKS MBD
'3D PDF
textAndCustomProperties = swMBDPdfData.GetTextAndCustomProperties
If Not IsEmpty(textAndCustomProperties) Then
    nbrTextAndCustomProperties = UBound(textAndCustomProperties)
    Debug.Print "Text and custom properties from theme:"
    For i = 0 To nbrTextAndCustomProperties
        Set swTextAndCustomProperty = textAndCustomProperties(i)
        If swTextAndCustomProperty.IsCustomProperty = False Then
            Debug.Print (" Text:")
            Debug.Print (" Description: " & swTextAndCustomProperty.Description)
            Debug.Print (" Field name: " & swTextAndCustomProperty.Name)
        Else
            Debug.Print (" Custom property:")
            custProp = "CustomProperty " + CStr(i)
            swTextAndCustomProperty.CustomPropertyName = custProp
            Debug.Print (" Custom property name: " & swTextAndCustomProperty.CustomPropertyName)
            Debug.Print (" Description: " & swTextAndCustomProperty.Description)
            Debug.Print (" Field name: " & swTextAndCustomProperty.Name)
            Debug.Print (" Value: " & swTextAndCustomProperty.Value)
        End If
    Next i
Else
    Debug.Print "No text or custom properties in theme."
End If

'Create SOLIDWORKS MBD 3D PDF
status = swModelDocExt.PublishTo3DPDF(swMBDPdfData)
Debug.Print ("Status of publishing part to SOLIDWORKS MBD 3D PDF (0 = success): " & status)

```

End Sub

Publish Text and Custom Properties from Theme to MBD 3D PDF Example (VB.NET)

This example shows how to publish text and custom properties from a theme to SOLIDWORKS MBD 3D PDF.

```

'-----
' Preconditions:
' 1. Verify that:
'    * specified part,
'    * SOLIDWORKS MBD 3D PDF theme, and
'    * c:\temp exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part.
' 2. Gets the MBD3DPdfData object.
' 3. Sets the path and file name for the SOLIDWORKS MBD 3D PDF.
' 4. Sets to display SOLIDWORKS MBD 3D PDF after publishing.
' 5. Sets the theme for the SOLIDWORKS MBD 3D PDF.
' 6. Sets standard views for the SOLIDWORKS MBD 3D PDF.
' 7. Creates and sets custom views for the SOLIDWORKS MBD 3D PDF.
' 8. Gets the text and custom properties from the specified
'    theme for the SOLIDWORKS MBD 3D PDF and sets the custom
'    property names.
' 9. Creates the SOLIDWORKS MBD 3D PDF.
' 10. Examine the Immediate window and c:\temp\MBDPart1.PDF.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

Public Sub main()

```

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swMBDPdfData As MBD3DPdfData
    Dim swTextAndCustomProperty As TextAndCustomProperty
    Dim fileName As String
    Dim standardViews As Object
    Dim viewIDs(2) As Integer
    Dim viewNames(1) As String
    Dim textAndCustomProperties As Object
    Dim nbrTextAndCustomProperties As Integer
    Dim status As Integer
    Dim errors As Integer
    Dim warnings As Integer
    Dim custProp As String
    Dim i As Integer

    fileName = "C:\temp\box.sldprt"
    swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "
", errors, warnings)
    swModelDocExt = swModel.Extension

    'Get MBD3DPdfData object
    swMBDPdfData = swModelDocExt.GetMBD3DPdfData

    'Set path and file name for SOLIDWORKS MBD 3D PDF
    swMBDPdfData.FilePath = "c:\temp\MBDPart1.PDF"

    'Display SOLIDWORKS MBD 3D PDF
    swMBDPdfData.ViewPdfAfterSaving = True

    'Set SOLIDWORKS MBD 3D PDF theme
    swMBDPdfData.ThemeName = "C:\Program Files\SolidWorks Corp\SOLIDWORKS\data\themes\simple part (a4, portrait)\
theme.xml"

    'Set standard views for SOLIDWORKS MBD 3D PDF
    viewIDs(0) = swStandardViews_e.swFrontView
    viewIDs(1) = swStandardViews_e.swTopView
    viewIDs(2) = swStandardViews_e.swDimetricView
    standardViews = viewIDs
    swMBDPdfData.SetStandardViews(standardViews)

    'Create and set custom views for SOLIDWORKS MBD 3D PDF
    swModel.NameView("CustomView1")
    swModel.NameView("CustomView2")

```

```

viewNames(0) = "CustomView1"
viewNames(1) = "CustomView2"
swMBDPdfData.SetMoreViews(viewNames)

'Get text and custom properties from theme for SOLIDWORKS MBD
'3D PDF
textAndCustomProperties = swMBDPdfData.GetTextAndCustomProperties
If Not IsNothing(textAndCustomProperties) Then
    nbrTextAndCustomProperties = UBound(textAndCustomProperties)
    Debug.Print("Text and custom properties from theme:")
    For i = 0 To nbrTextAndCustomProperties
        swTextAndCustomProperty = textAndCustomProperties(i)
        If swTextAndCustomProperty.IsCustomProperty = False Then
            Debug.Print(" Text:")
            Debug.Print(" Description: " & swTextAndCustomProperty.Description)
            Debug.Print(" Field name: " & swTextAndCustomProperty.Name)
        Else
            Debug.Print(" Custom property:")
            custProp = "CustomProperty " + CStr(i)
            swTextAndCustomProperty.CustomPropertyName = custProp
            Debug.Print(" Custom property name: " & swTextAndCustomProperty.CustomPropertyName)
            Debug.Print(" Description: " & swTextAndCustomProperty.Description)
            Debug.Print(" Field name: " & swTextAndCustomProperty.Name)
            Debug.Print(" Value: " & swTextAndCustomProperty.Value)
        End If
    Next i
Else
    Debug.Print("No text or custom properties in theme.")
End If

'Create SOLIDWORKS MBD 3D PDF
status = swModelDocExt.PublishTo3DPDF(swMBDPdfData)
Debug.Print("Status of publishing part to SOLIDWORKS MBD 3D PDF (0 = success): " & status)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Select the Midpoint of an Edge Example (C#)

This example shows how to select the midpoint of an edge.

```
//-----  
// Preconditions:  
// 1. Open a part document.  
// 2. Select an edge.  
// 3. Open the Immediate window.  
//  
// Postconditions:  
// 1. Puts a midpoint on the selected edge.  
// 2. Place the cursor on the selected edge to see the  
//    midpoint.  
// 3. Examine the Immediate window.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace PlaceMidPointOnEdge_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        const long swSelEDGES = 1;  
        const long swSelREFERENCECURVES = 26;  
        const long swSelPOINTREFS = 41;  
        const long swSelREFEDGES = 51;  
  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            PartDoc swPart = default(PartDoc);  
            SelectionMgr swSelMgr = default(SelectionMgr);  
            long nSelType = 0;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swPart = (PartDoc)swModel;  
            swSelMgr = (SelectionMgr)swModel.SelectionManager;  
  
            nSelType = swSelMgr.GetSelectedObjectType3(1, -1);  
            Debug.Print("SelType (before) = " + nSelType);  
  
            swModel.SelectMidpoint();  
  
            nSelType = swSelMgr.GetSelectedObjectType3(1, -1);  
            Debug.Print("SelType (after ) = " + nSelType);  
        }  
  
        public SldWorks swApp;  
    }  
}
```

Put_a_Midpoint_on_an_Edge_Example_VB (Topic)

SOLIDWORKS API Help

Select the Midpoint of an Edge Example (VBA)

This example shows how to select the midpoint of an edge.

```
'-----  
' Preconditions:  
' 1. Open a part document.  
' 2. Select an edge.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Puts a midpoint on the selected edge.  
' 2. Place the cursor on the selected edge to see the  
'    midpoint.  
' 3. Examine the Immediate window.  
'-----
```

Option Explicit

```
Const swSelEDGES As Long = 1  
Const swSelREFERENCECURVES As Long = 26  
Const swSelPOINTREFS As Long = 41  
Const swSelREFEDGES As Long = 51
```

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swPart As SldWorks.PartDoc  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim nSelType As Long  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
    Set swSelMgr = swModel.SelectionManager  
  
    nSelType = swSelMgr.GetSelectedObjectType3(1, -1)  
    Debug.Print "SelType (before) = " + Str(nSelType)  
  
    swModel.SelectMidpoint  
  
    nSelType = swSelMgr.GetSelectedObjectType3(1, -1)  
    Debug.Print "SelType (after ) = " + Str(nSelType)
```

End Sub

Select the Midpoint of an Edge Example (VB.NET)

This example shows how to select the midpoint of an edge.

```
' -----  
' Preconditions:  
' 1. Open a part document.  
' 2. Select an edge.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Puts a midpoint on the selected edge.  
' 2. Place the cursor on the selected edge to see the  
'    midpoint.  
' 3. Examine the Immediate window.  
' -----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Const swSelEDGES As Long = 1  
    Const swSelREFERENCECURVES As Long = 26  
    Const swSelPOINTREFS As Long = 41  
    Const swSelREFEDGES As Long = 51  
  
    Sub main()  
  
        Dim swModel As ModelDoc2  
        Dim swPart As PartDoc  
        Dim swSelMgr As SelectionMgr  
        Dim nSelType As Long  
  
        swModel = swApp.ActiveDoc  
        swPart = swModel  
        swSelMgr = swModel.SelectionManager  
  
        nSelType = swSelMgr.GetSelectedObjectType3(1, -1)  
        Debug.Print("SelType (before) = " + Str(nSelType))  
  
        swModel.SelectMidpoint()  
  
        nSelType = swSelMgr.GetSelectedObjectType3(1, -1)  
        Debug.Print("SelType (after ) = " + Str(nSelType))  
  
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Put Assembly Components in Drawing View on Different Layers Example (VBA)

This example shows how to put all assembly components in a drawing view on different layers.

```
' -----  
' Preconditions:  
' 1. Open a drawing document of an assembly with multiple  
'    components.  
' 2. Select a drawing view.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Creates a new drawing layer for each assembly component.  
' 2. Puts each component on its own layer.  
' 3. To verify, right-click the drawing view selected in  
'    Preconditions step 2 and click Change Layer > down-arrow  
'    button on the Change document layer dialog.  
' 4. Examine the Immediate window.  
'  
' NOTE: Illegal characters are replaced with legal characters  
' when creating a new drawing layer.  
' -----
```

Option Explicit

```
Private Sub ChangeComponentLayer(swApp As SldWorks.SldWorks, swDraw As SldWorks.DrawingDoc, swDrawComp As  
SldWorks.DrawingComponent, sLayerName As String)
```

```
    Dim bRet As Boolean
```

```
    ' Form a legal layer name by replacing backslash
```

```
    ' ( / ) and at sign ( @ ) symbols with underscores
```

```
    sLayerName = Replace(sLayerName, "/", "_")
```

```
    sLayerName = Replace(sLayerName, "@", "_")
```

```
    bRet = swDraw.CreateLayer(sLayerName, "layer for " & sLayerName, 0, swLineCONTINUOUS, swLW_NORMAL, True):
```

```
    Debug.Assert bRet
```

```
    swDrawComp.Layer = sLayerName
```

```
End Sub
```

```
Sub ProcessDrawingComponent(swApp As SldWorks.SldWorks, swDraw As SldWorks.DrawingDoc, swDrawComp As  
SldWorks.DrawingComponent, sPadStr As String)
```

```
    Dim vDrawCompChildArr As Variant
```

```
    Dim vDrawCompChild As Variant
```

```
    Dim swDrawCompChild As SldWorks.DrawingComponent
```

```
    Debug.Print sPadStr & swDrawComp.Name
```

```
    ChangeComponentLayer swApp, swDraw, swDrawComp, swDrawComp.Name
```

```
    vDrawCompChildArr = swDrawComp.GetChildren
```

```
    If Not IsEmpty(vDrawCompChildArr) Then
```

```
        For Each vDrawCompChild In vDrawCompChildArr
```

```
            Set swDrawCompChild = vDrawCompChild
```

```
            ProcessDrawingComponent swApp, swDraw, swDrawCompChild, sPadStr + " "
```

```
        Next
```

```
    End If
```

```
End Sub
```

```
Sub main()
```

```
    Dim swApp As SldWorks.SldWorks
```

```
    Dim swModel As SldWorks.ModelDoc2
```

```
    Dim swDraw As SldWorks.DrawingDoc
```

```
    Dim swSelMgr As SldWorks.SelectionMgr
```

```
    Dim swView As SldWorks.View
```

```
    Dim swDrawComp As SldWorks.DrawingComponent
```

```
    Dim bRet As Boolean
```

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swDraw = swModel
Set swSelMgr = swModel.SelectionManager
Set swView = swSelMgr.GetSelectedObject6(1, -1)
Set swDrawComp = swView.RootDrawingComponent

Debug.Print "File = " & swModel.GetPathName
Debug.Print " " & swView.Name & " [" & swView.Type & "]"

ProcessDrawingComponent swApp, swDraw, swDrawComp, " "
```

End Sub